

Infrastructure de communication

S. Mancini



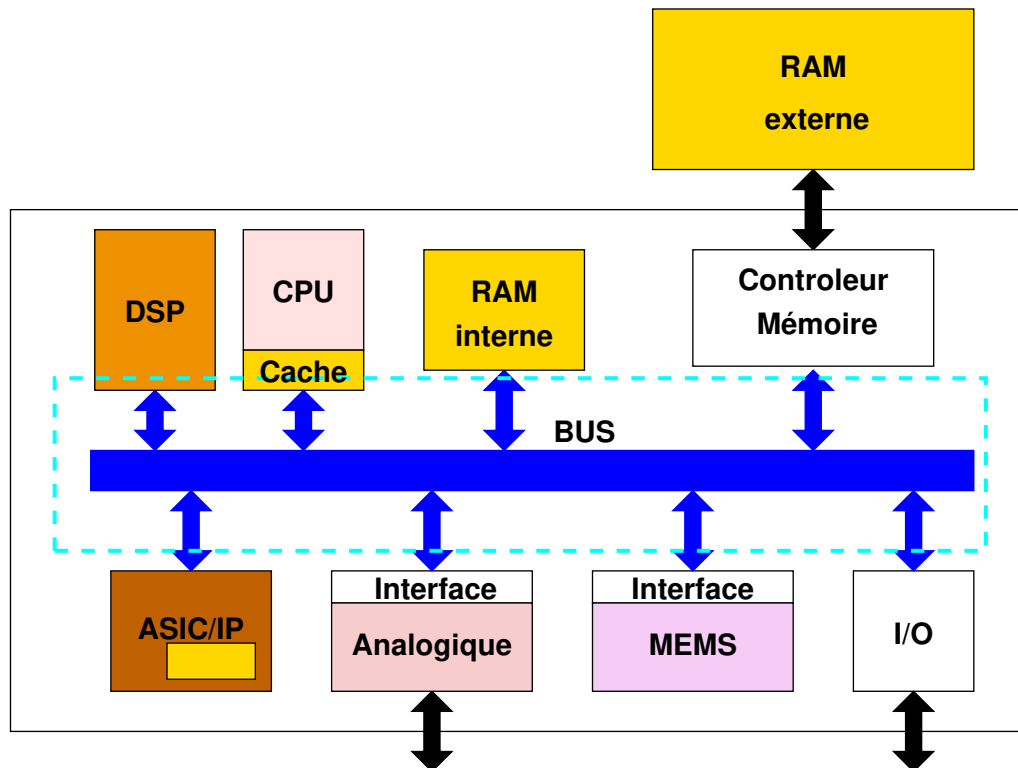
Plan

- ✖ Bus système
- ☐ Éléments d'un bus système
- ☐ Infrastructure de communication
- ☐ Bus systèmes
- ☐ Network on Chip
- ☐ Conclusion

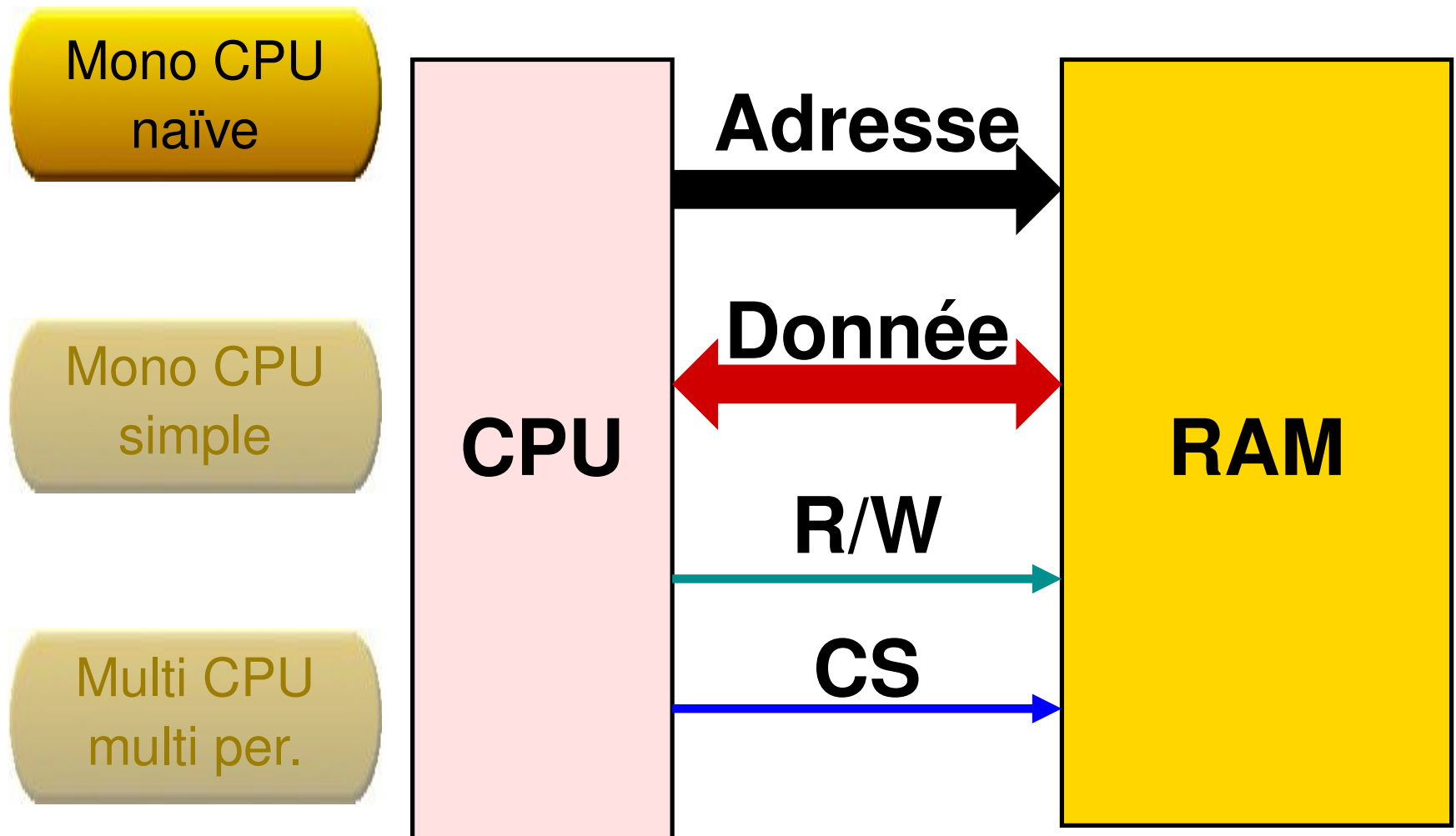
Objectifs

Les bus systèmes servent à assurer des transferts de données entre les modules

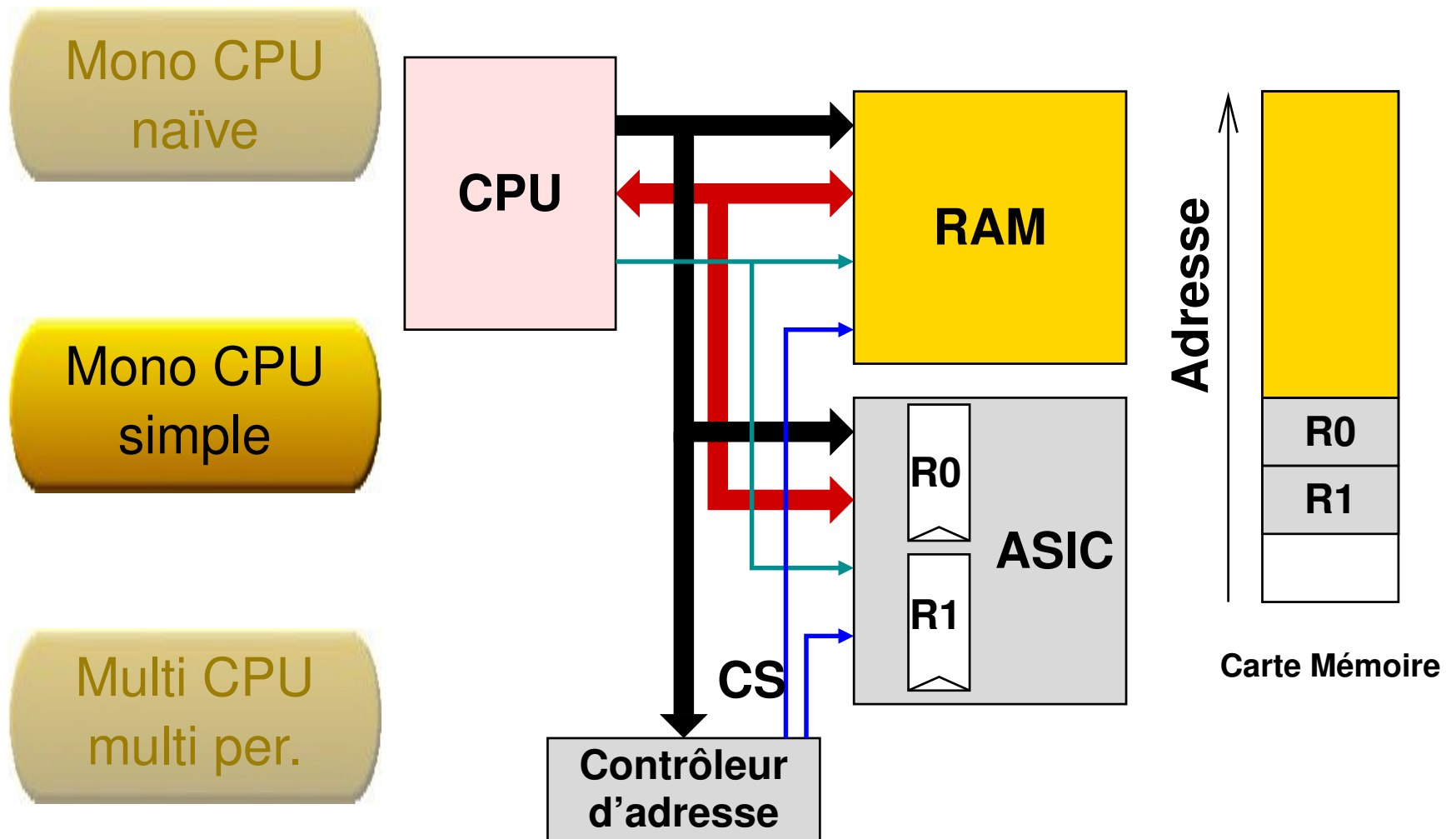
En général, les bus systèmes servent à établir des communications dont on **ne connaît pas à-priori** ni la topologie ni le séquençement exacts.



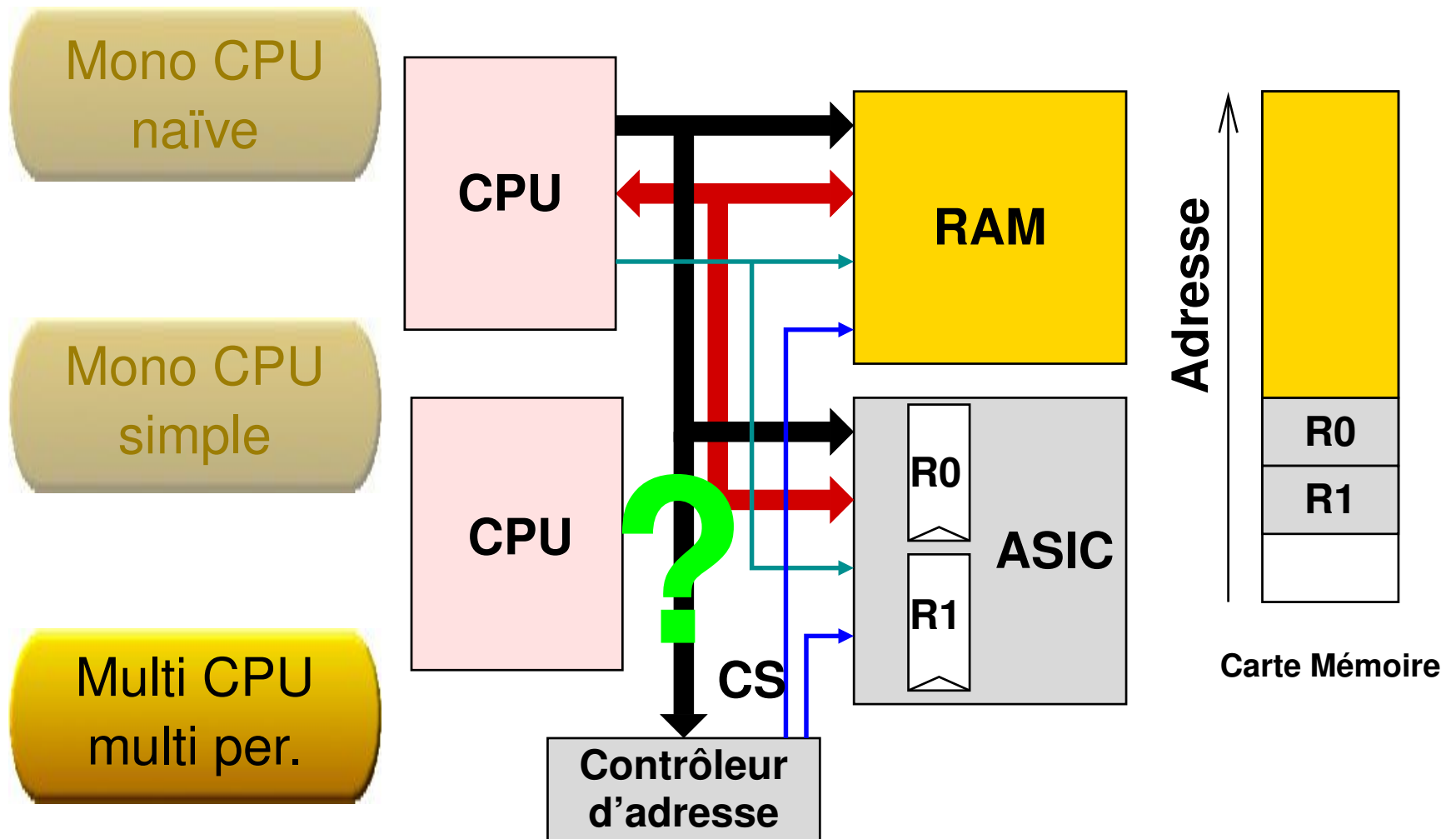
Problématique

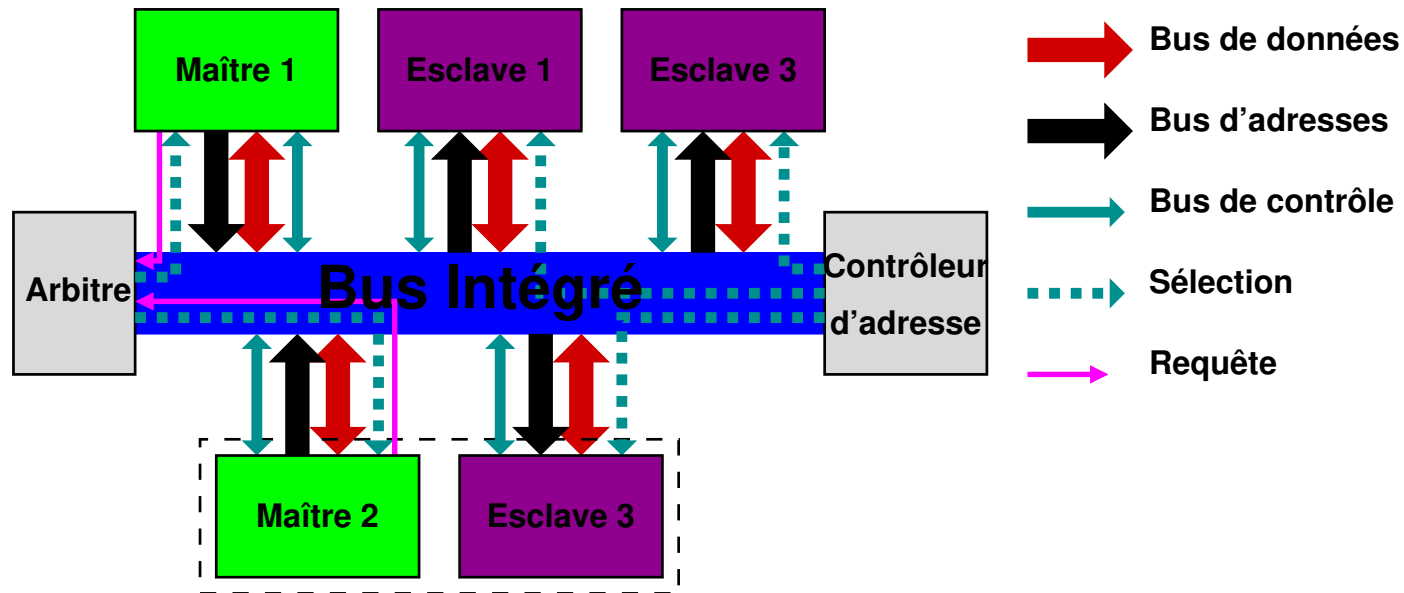


Problématique



Problématique





Caractéristiques d'un bus système:

❑ Connectique

- Données
- Adresses
- Contrôle

❑ Caractéristiques physiques

❑ Protocole

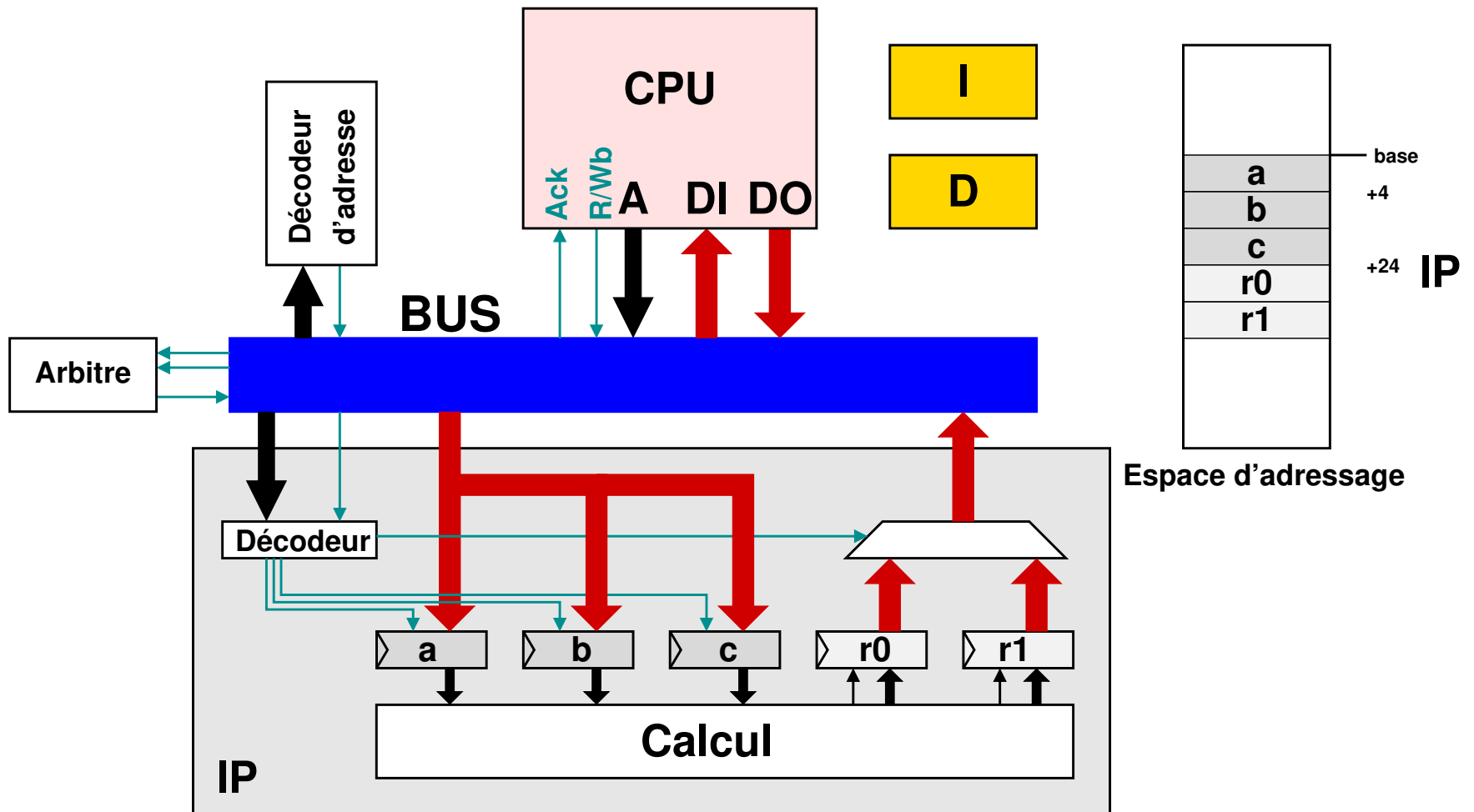
Exemple : communication CPU/IP

On souhaite faire communiquer un microprocesseur avec un module matériel (IP).

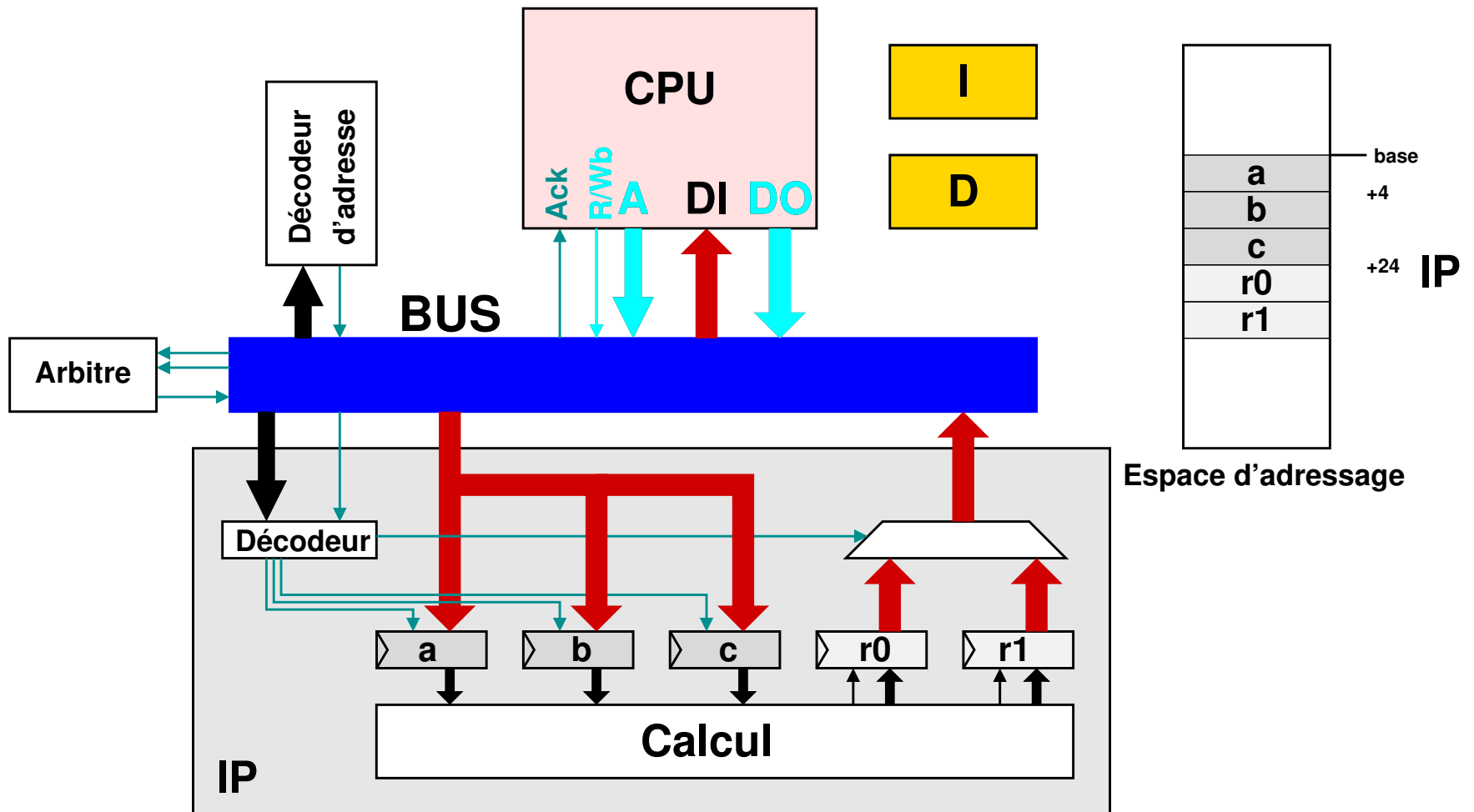
L'IP:

- ❑ A besoin de différents paramètres pour réaliser le calcul
- ❑ Envoie une interruption au CPU la fin des calculs
- ❑ Stocke les résultats dans des registres internes auxquels il faut accéder depuis le processeur

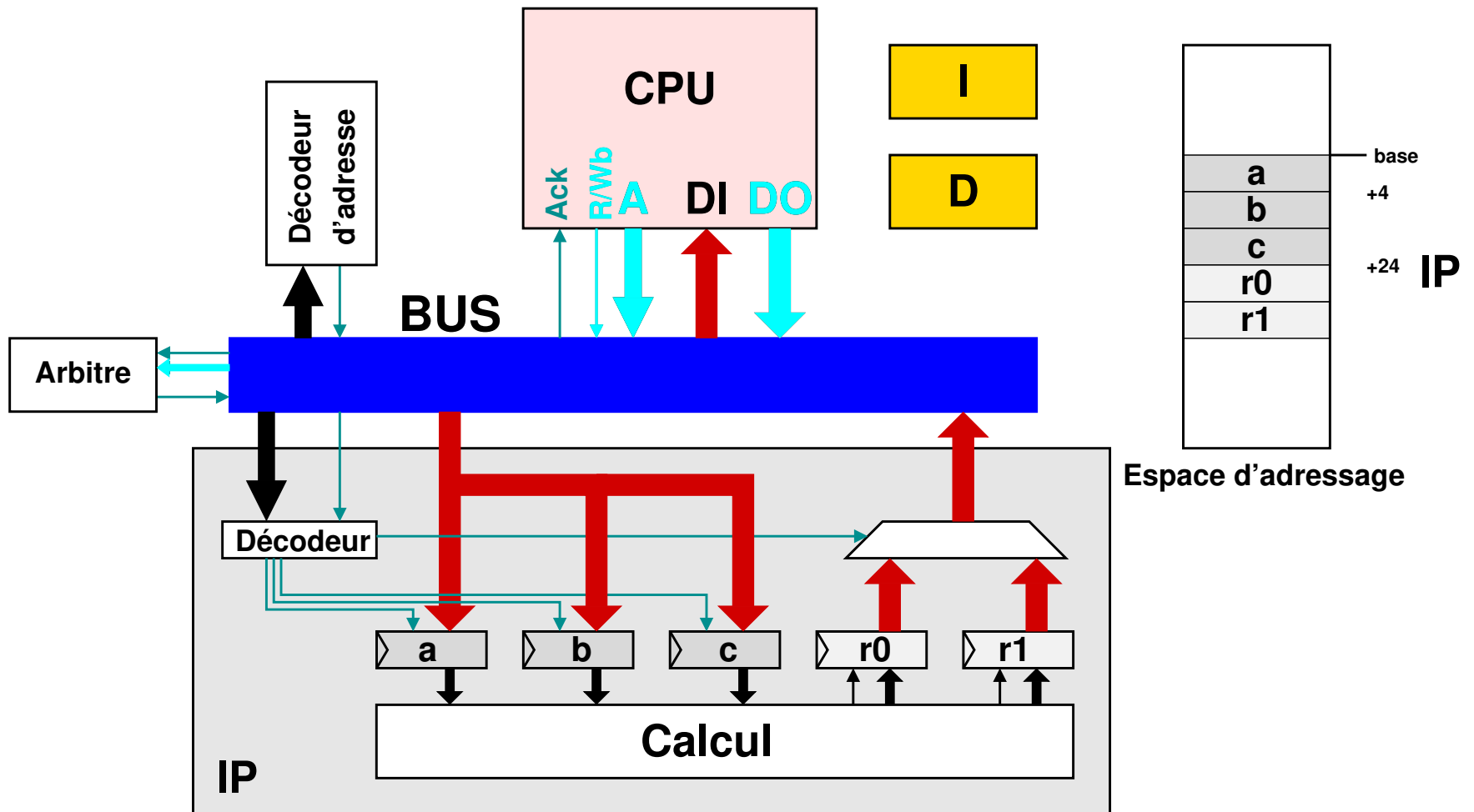
Exemple : communication CPU/IP



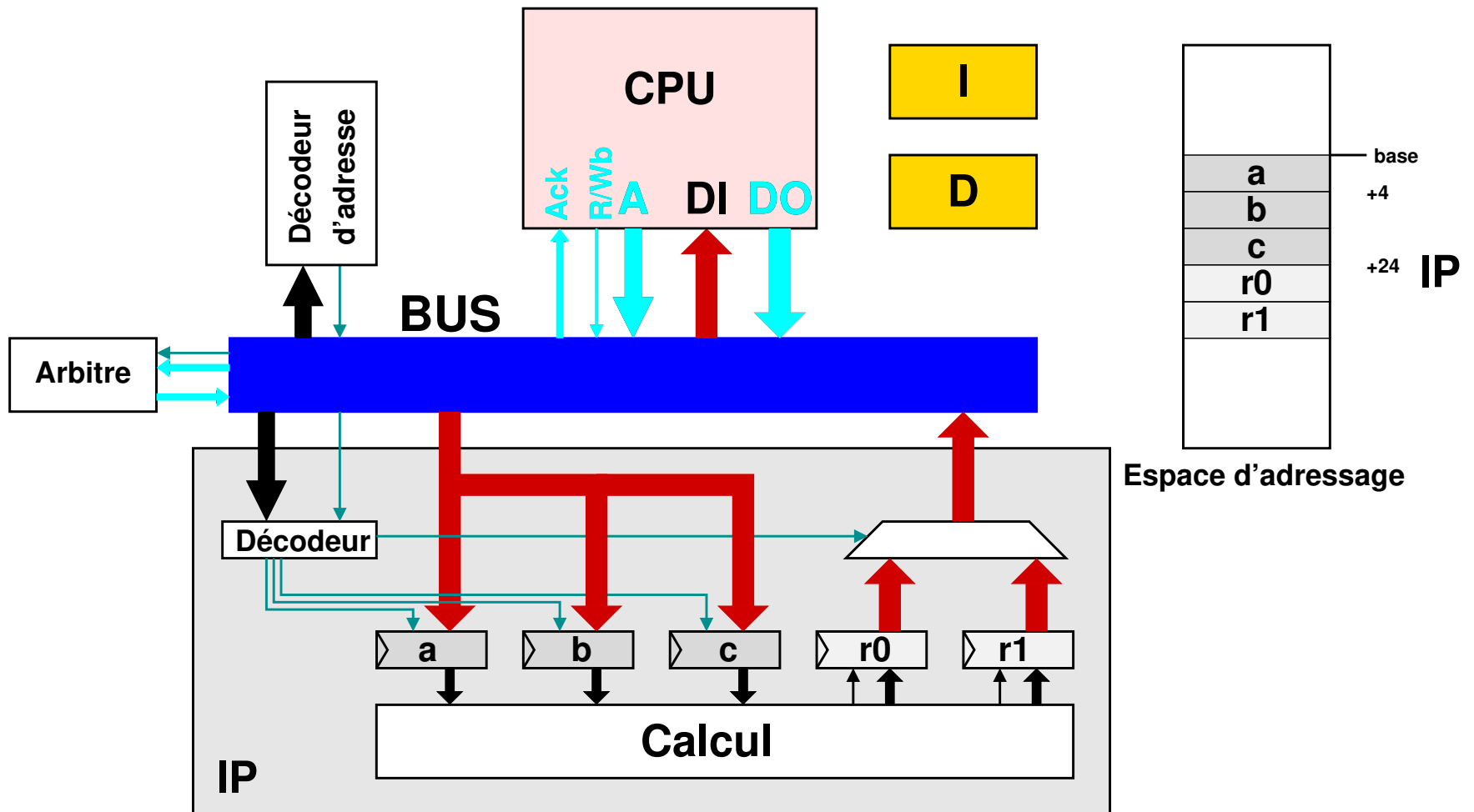
Exemple : communication CPU/IP



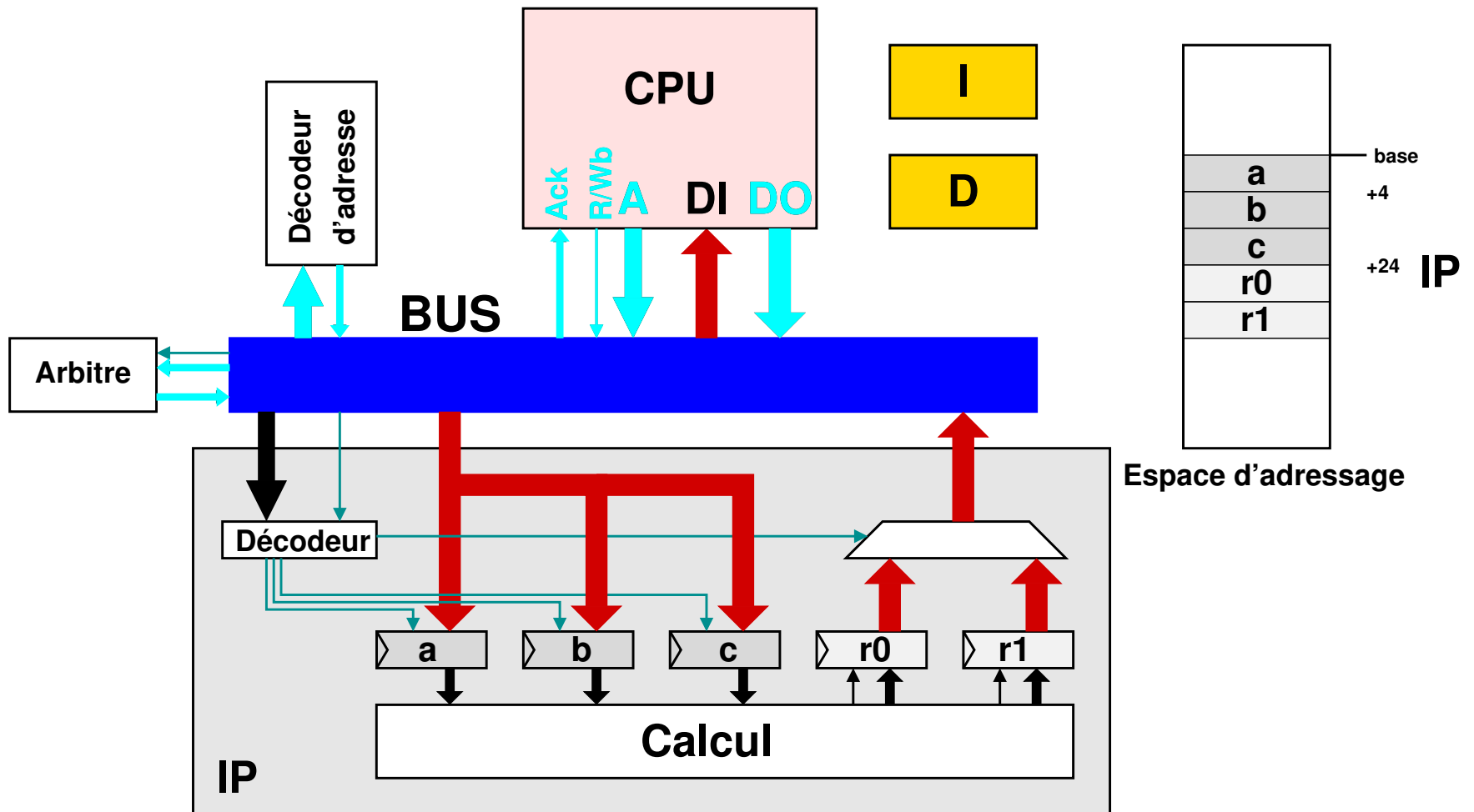
Exemple : communication CPU/IP



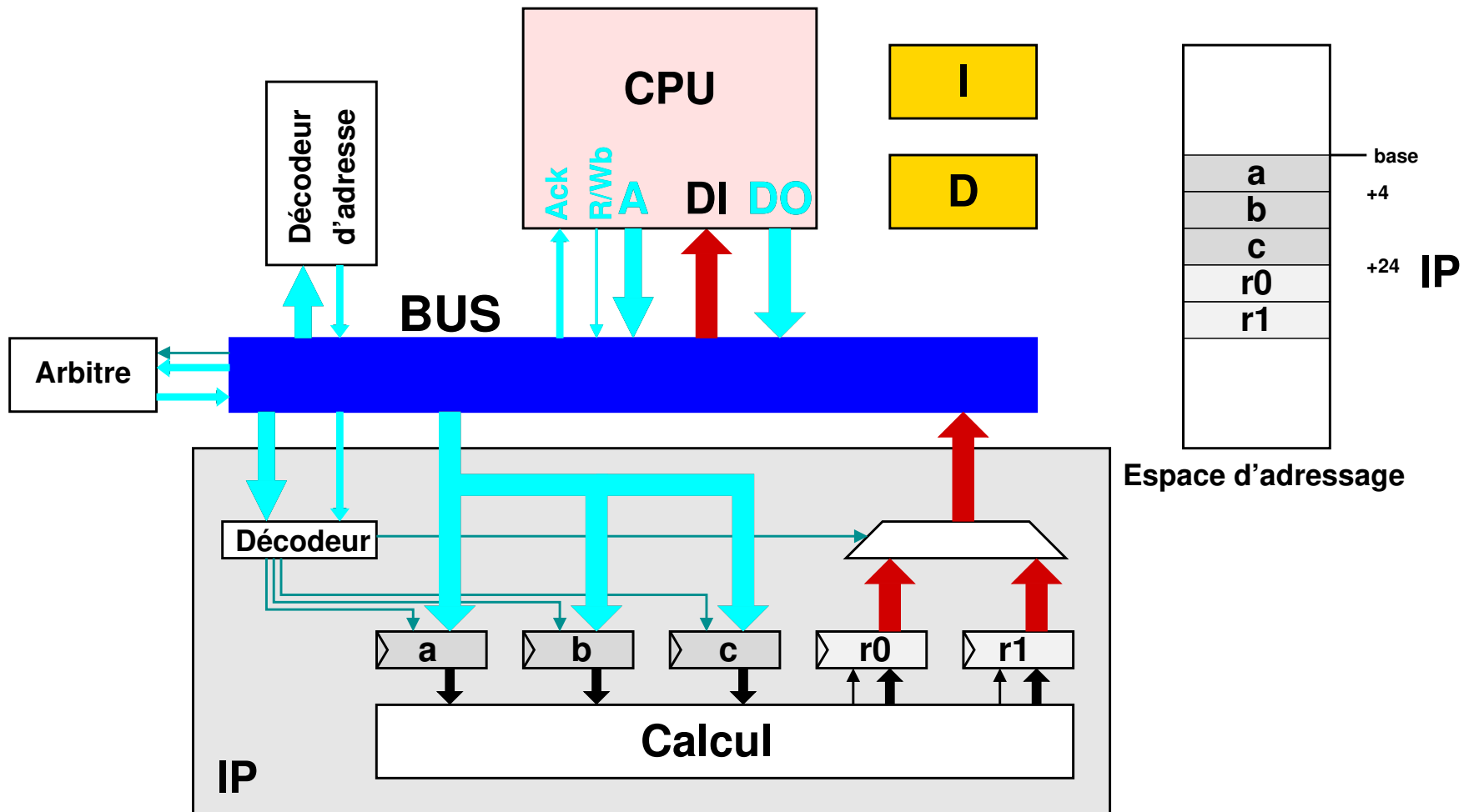
Exemple : communication CPU/IP



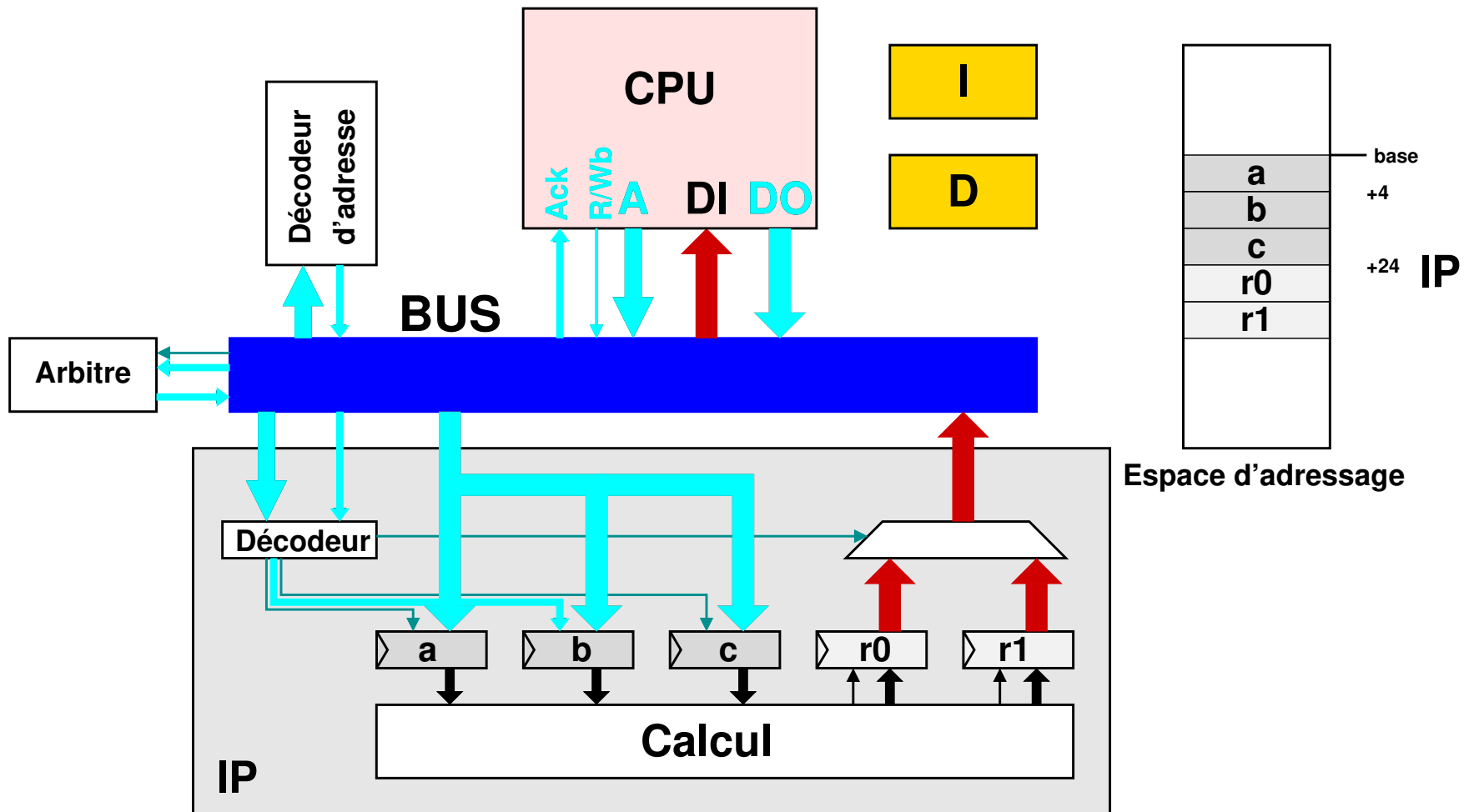
Exemple : communication CPU/IP



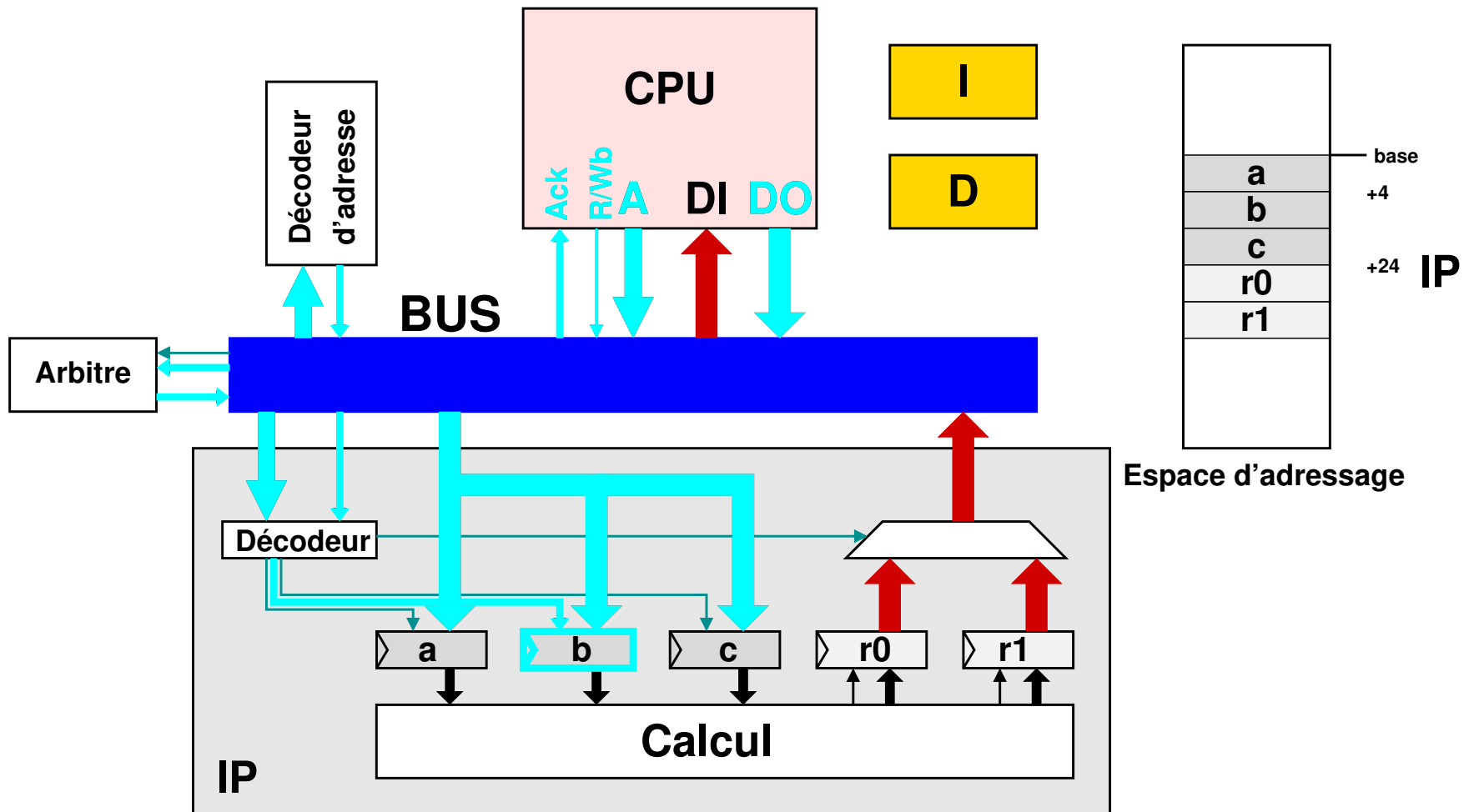
Exemple : communication CPU/IP



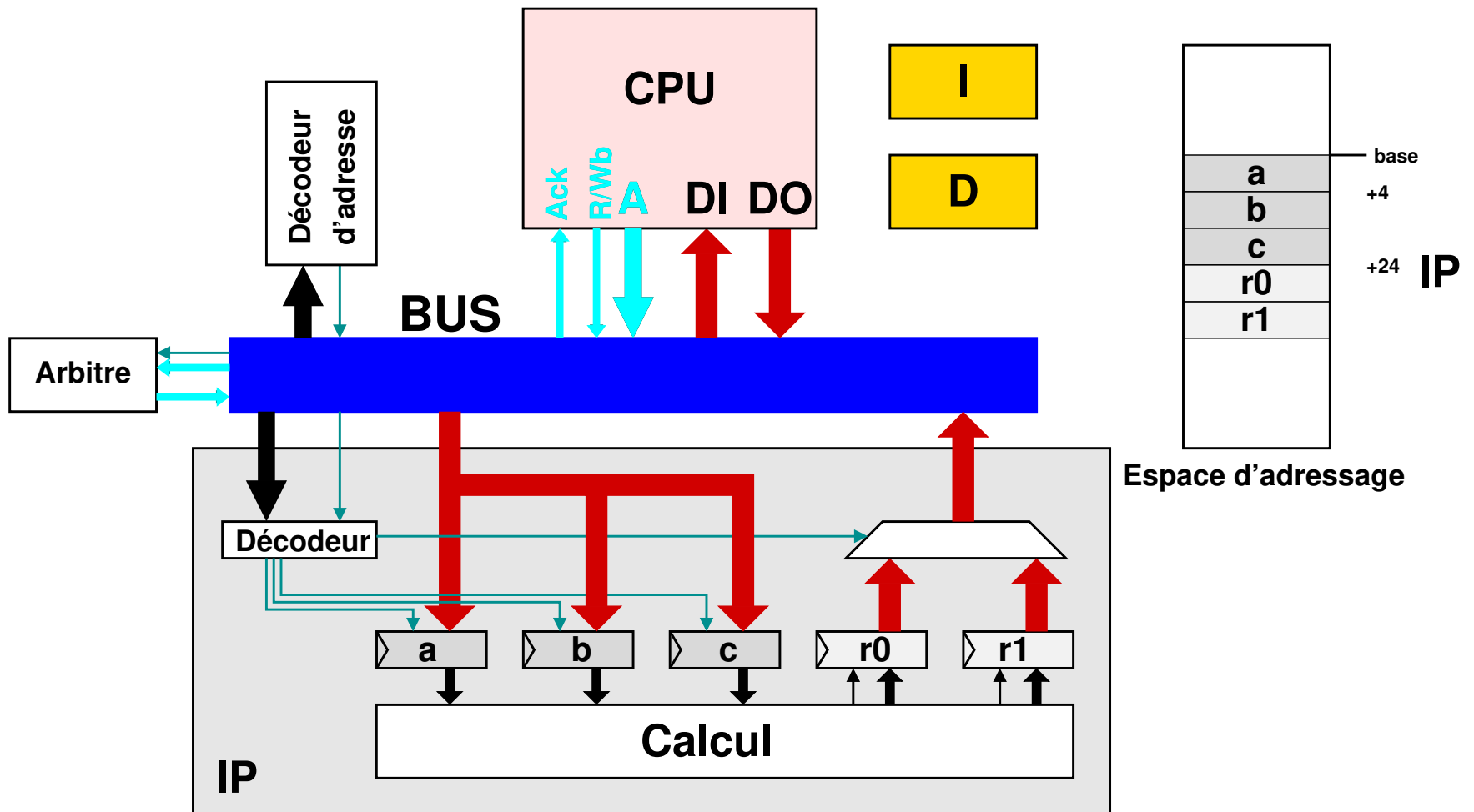
Exemple : communication CPU/IP



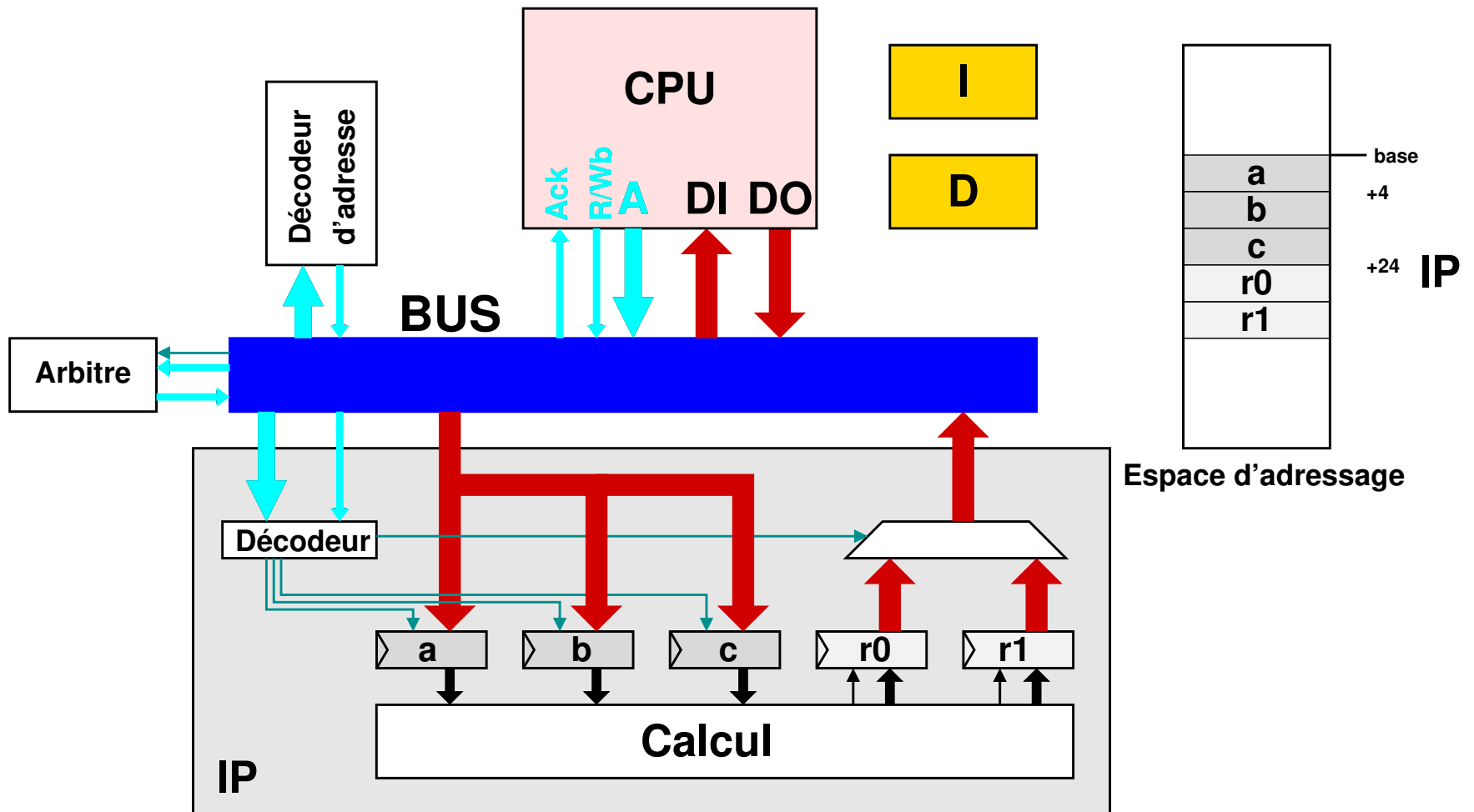
Exemple : communication CPU/IP



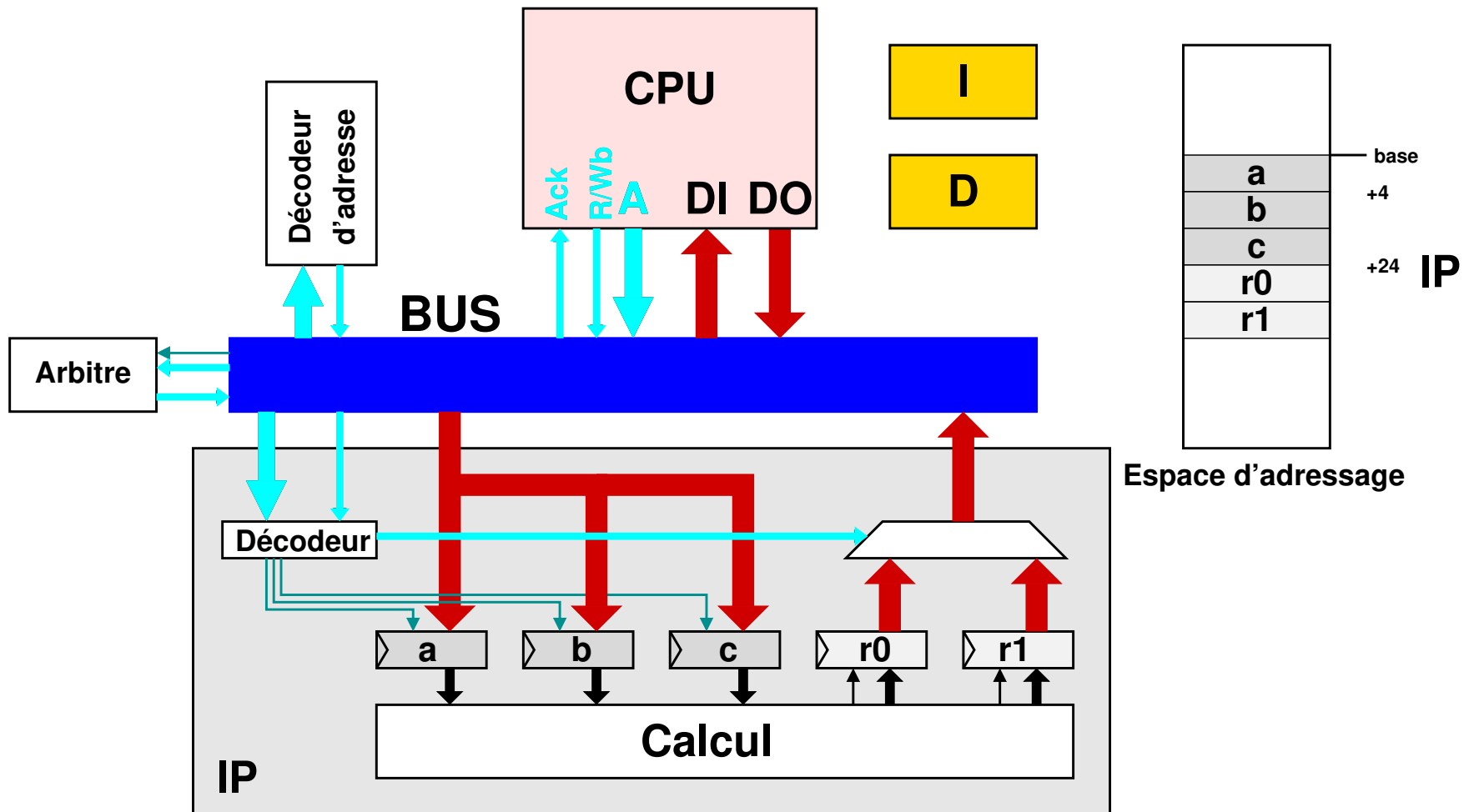
Exemple : communication CPU/IP



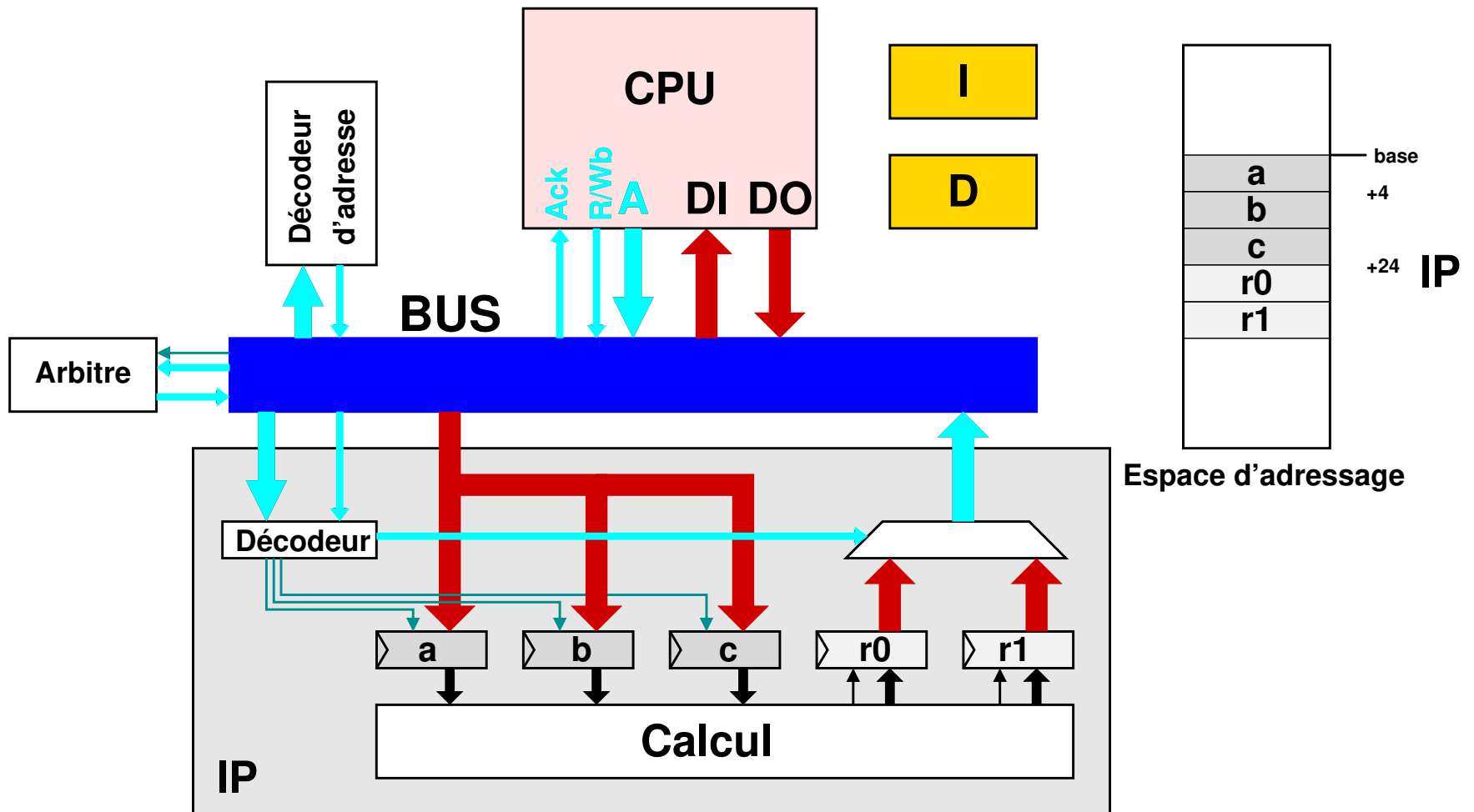
Exemple : communication CPU/IP



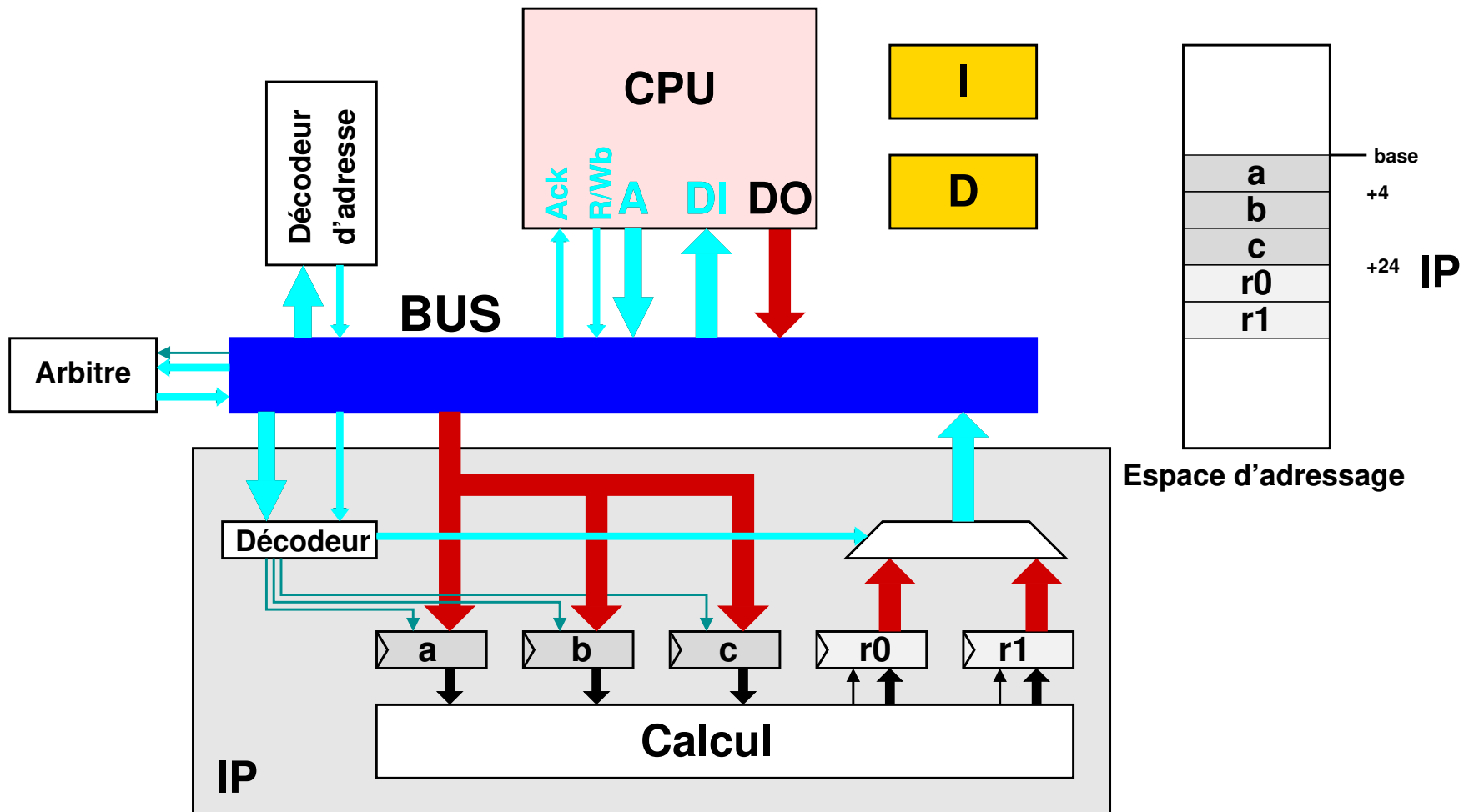
Exemple : communication CPU/IP



Exemple : communication CPU/IP



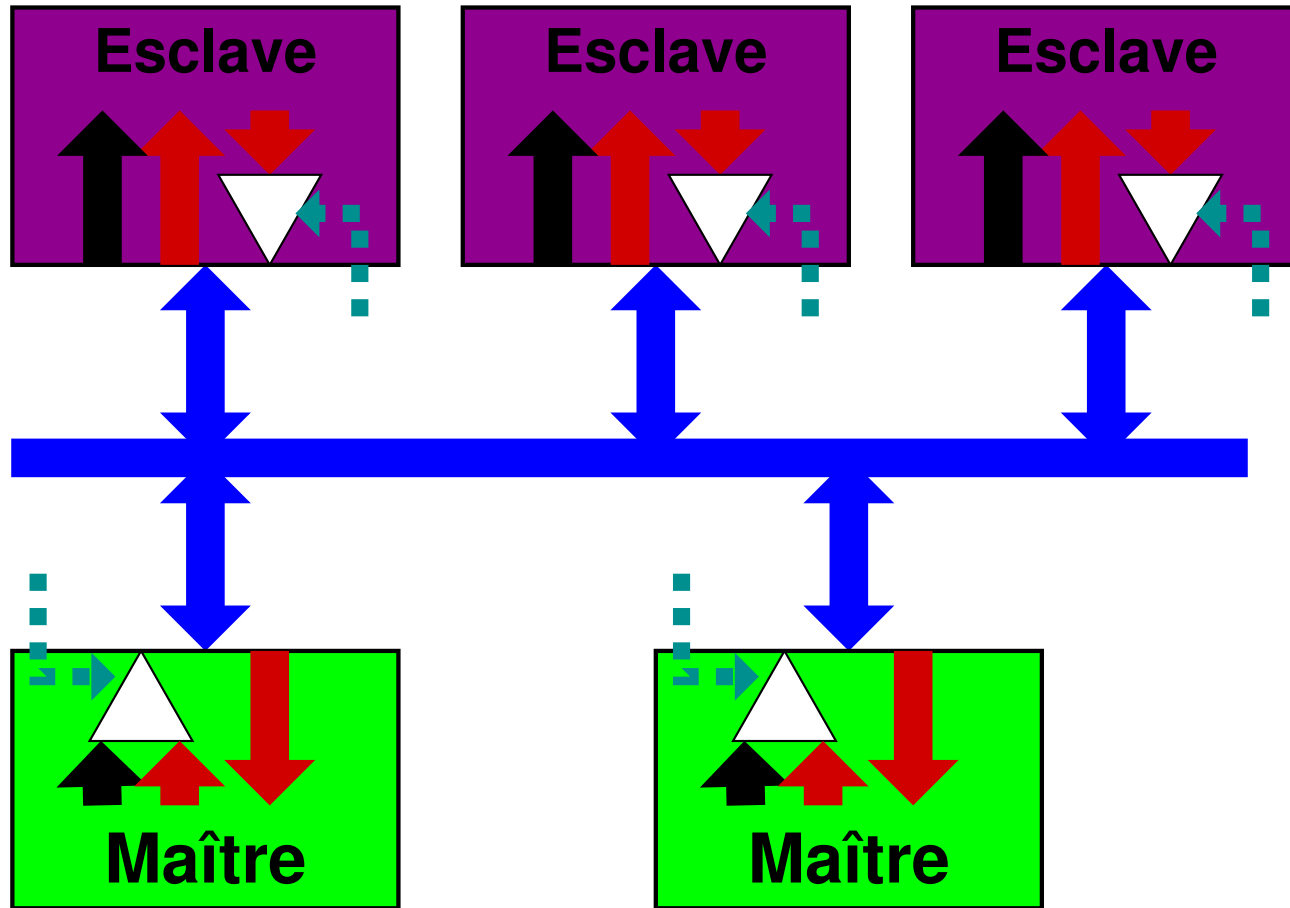
Exemple : communication CPU/IP



Plan

- Bus système
- ✗ Éléments d'un bus système
 - Connectique
 - Contrôleur de bus
- Infrastructure de communication
- Bus systèmes
- Network on Chip
- Conclusion

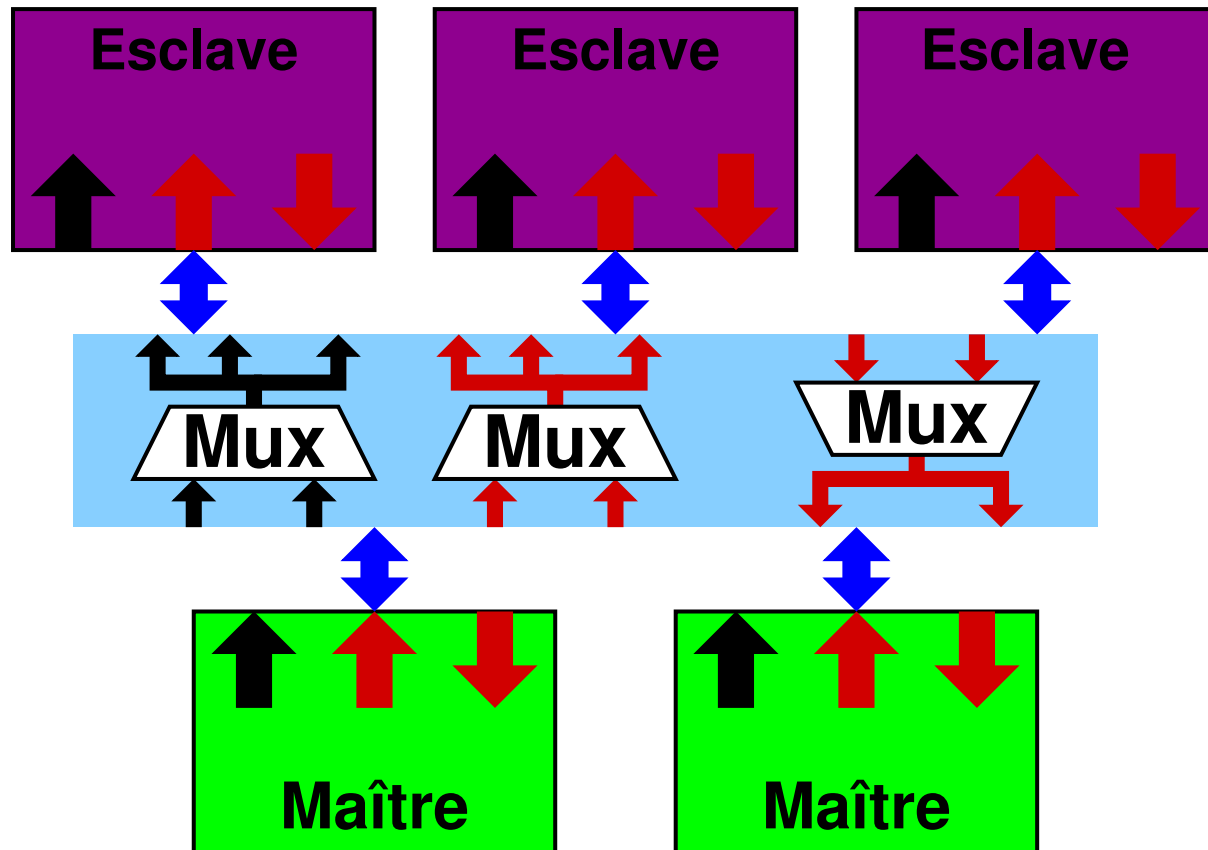
Bus Trois Etats



- Grande modularité
- Connectivité maître-esclave simple
- Observable simplement

- Fréquence faible
- Test complexe (buffer)
- Grande longueur des connexions
- Layout délicat (découplage)

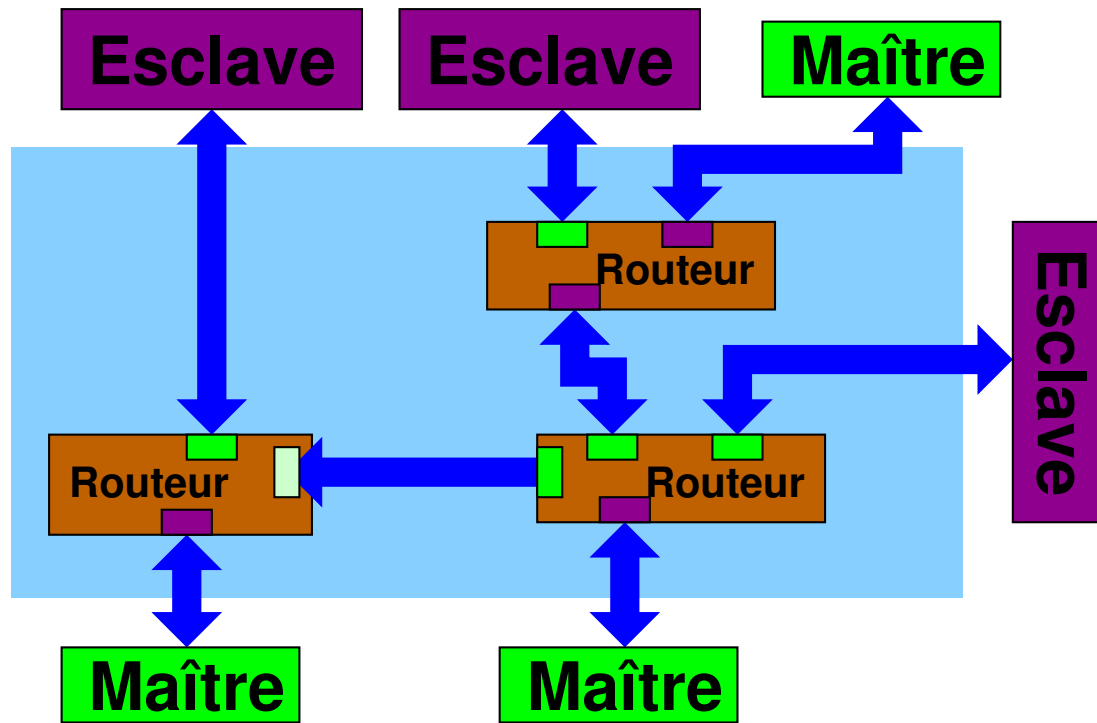
Bus “Cross bar”



- Fréquence élevée
- Accès simultanés
- Conçu avec des cellules standards

- Nombreux fils
- Layout complexe pour de nombreux maîtres
- Peu modulaire
- Difficile à observer

Bus "NOC"



Les bus Network on Chip propagent les requêtes dans une infrastructure semblable à un réseau. Des routeurs redirigent les requêtes jusqu'au destinataire.

- Fréquence élevée
- Transferts simultanés entre plusieurs M&S
- Conçu avec des cellules standards
- Maîtrise des conflits de routage
- Nombreux fils
- Layout complexe pour de nombreux maîtres

Contrôleur de bus

Le contrôleur de bus répond aux requêtes des maîtres. Il donne accès au bus à un des demandeurs selon :

- ❑ Une politique de priorité
- ❑ Une gestion temporelle du bus

Les principaux critères d'efficacité sont :

- ❑ La garantie de bande passante
- ❑ Le temps d'attente d'accès maximum
- ❑ La préemptabilité

Arbitrage

- ❑ Les politiques pré-définies
 - Séquencement à-priori
 - Découpage temporel fixe
- ❑ Les politiques en-ligne
 - Priorités (statiques/dynamiques)
 - Tourniquet (Round-Robin)
 - Mixte

Exercice: arbitre de bus

On s'intéresse à un système de m maîtres, numérotés de 0 à $m - 1$. Chacun des maîtres émet un signal de requête $r[0..m - 1]$ et reçoit un signal d'acquitement $a[0..m - 1]$. A tout instant, un seul des bits de a est valide. C'est celui dont le maître reçoit l'accès au bus.

Dans un premier temps, on s'intéresse à la conception du bloc `fix_prio`, qui implémente une priorité fixe, le maître n° 0 étant le plus prioritaire. `fix_prio` a en entrée un vecteur de bit $e[0..m - 1]$ et en sortie $s[0..m - 1]$. Seul le bit $s[j]$ vaut 1 si $e[j]$ est l'entrée valide la plus prioritaire.



- Dessinez le schéma porte d'un arbitre simple
- Comment le temps de propagation évolue-t-il ?

Exercice: arbitre de bus

Afin d'éviter le phénomène de famine, les priorités sont maintenant tournantes. Pour rester simple, à chaque fois qu'une des requêtes est valide, les priorités sont toutes décalées d'un cran vers la gauche: $e[0]$ commence par la priorité maximum, puis prend la priorité de $e[m - 1]$, puis de $e[m - 2]$, etc ...

?

- En réutilisant `fix_prio`, donner un schéma du système
- Quelle est la nouvelle chaîne critique ?

Exercice: arbitre de bus

Pour ajouter de la souplesse dans la gestion des priorités, un maître peut maintenant demander une priorité complémentaire codée sur 3 bits, $p_j[2..0]$.

- ❑ La valeur de p_j est prioritaire sur la priorité usuelle (j étant le n° de maître).
- ❑ Si deux maîtres i et j ont des valeurs p_i et p_j identiques la résolution se fait d'après la priorité usuelle entre i et j .

?

?

- En réutilisant `fix_prio`, donner un schéma du système
- Quelle est la nouvelle chaîne critique ?
- Donnez un schéma du système avec priorité dynamique et tournante.

Amortissement du coût de l'arbitrage: burst

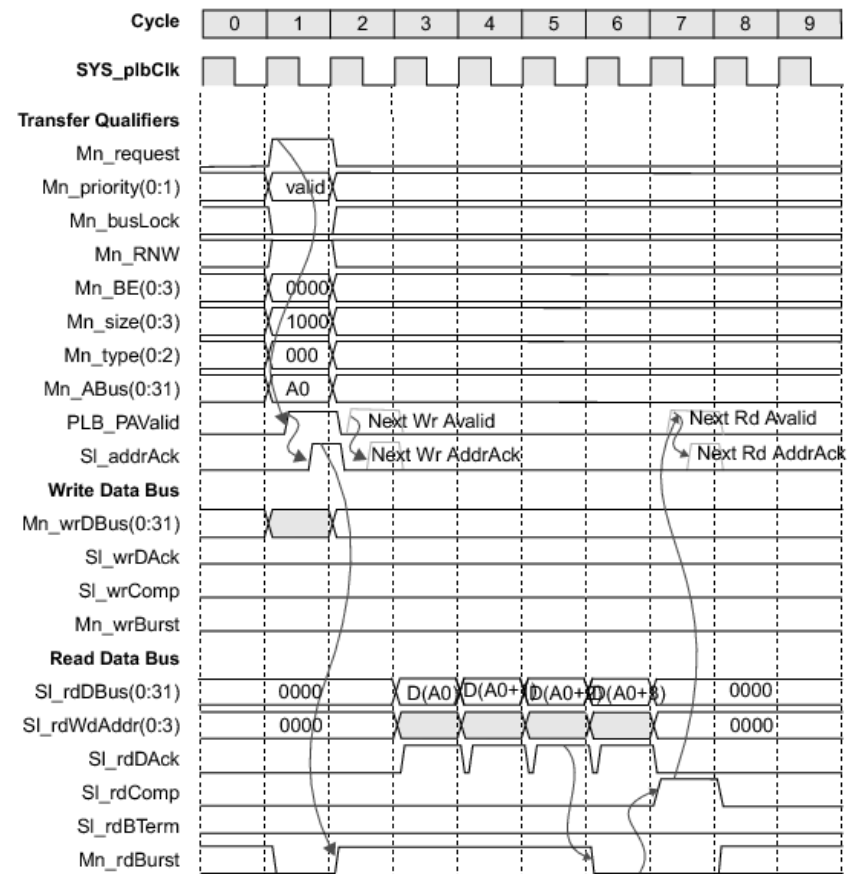
- 👍 L'arbitrage rend (quasi)impossible la réalisation d'un transfert d'une donnée en un seul cycle d'horloge. Les performances sont drastiquement réduites.

Le coût de l'arbitrage est amorti en faisant plusieurs transferts pour un seul arbitrage.

➡ C'est la notion de **burst**.

Lors de la requête, le maître spécifie la longueur du transfert.

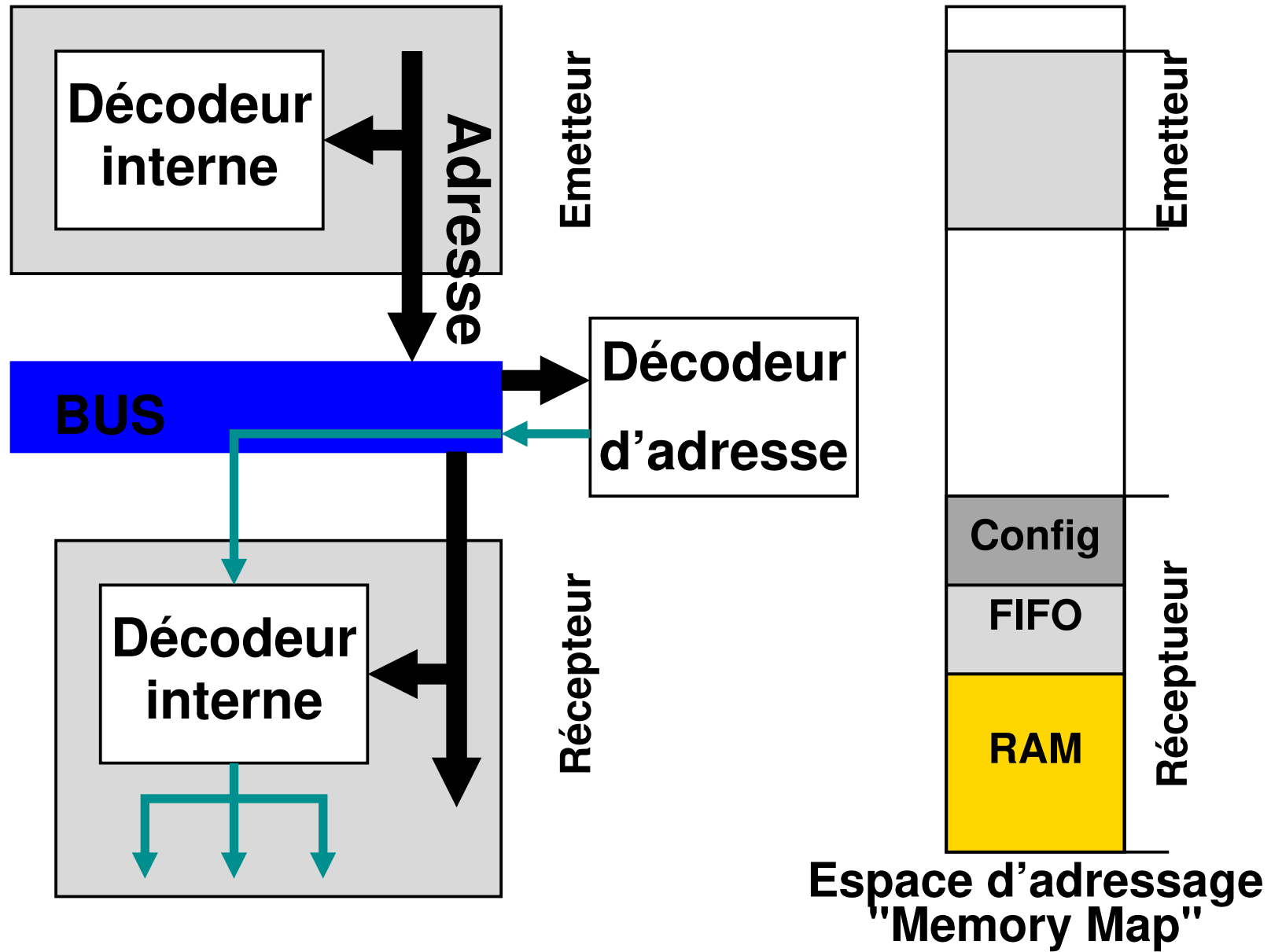
Certains bus autorisent des burst de longueur indéfinie, la fin étant gérée par un protocole de contrôle.



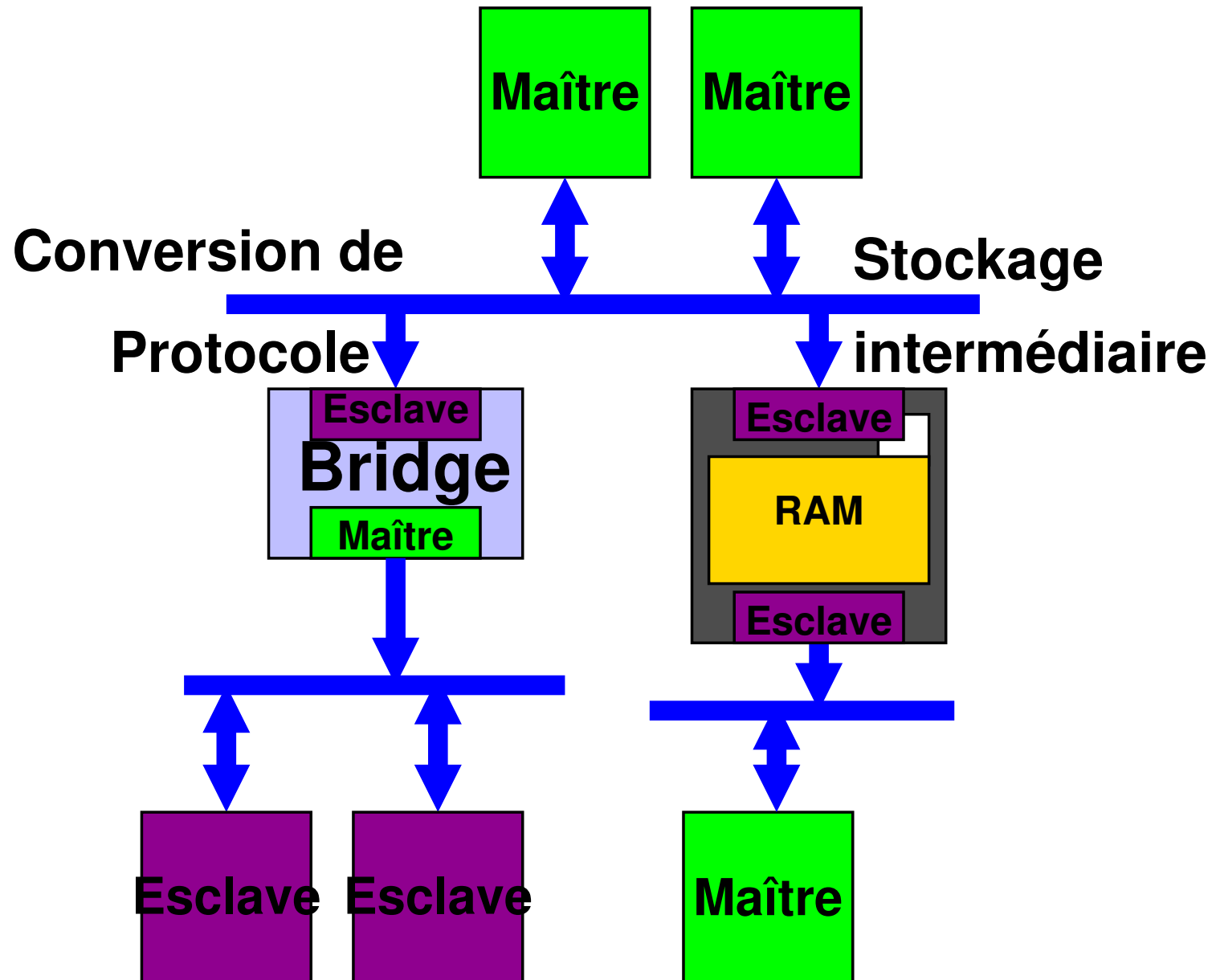
Plan

- ❑ Bus système
- ❑ Éléments d'un bus système
- ✖ Infrastructure de communication
 - ✓ Topologie
 - ❑ DMA
- ❑ Bus systèmes
- ❑ Network on Chip
- ❑ Conclusion

Espace d'adressage



Hiérarchie de bus



Gestion des interblocages

Il y a interblocage lorsqu'il y a :

- ❑ Demandes simultanées
- ❑ Attente circulaire
- ❑ Absence de préemption

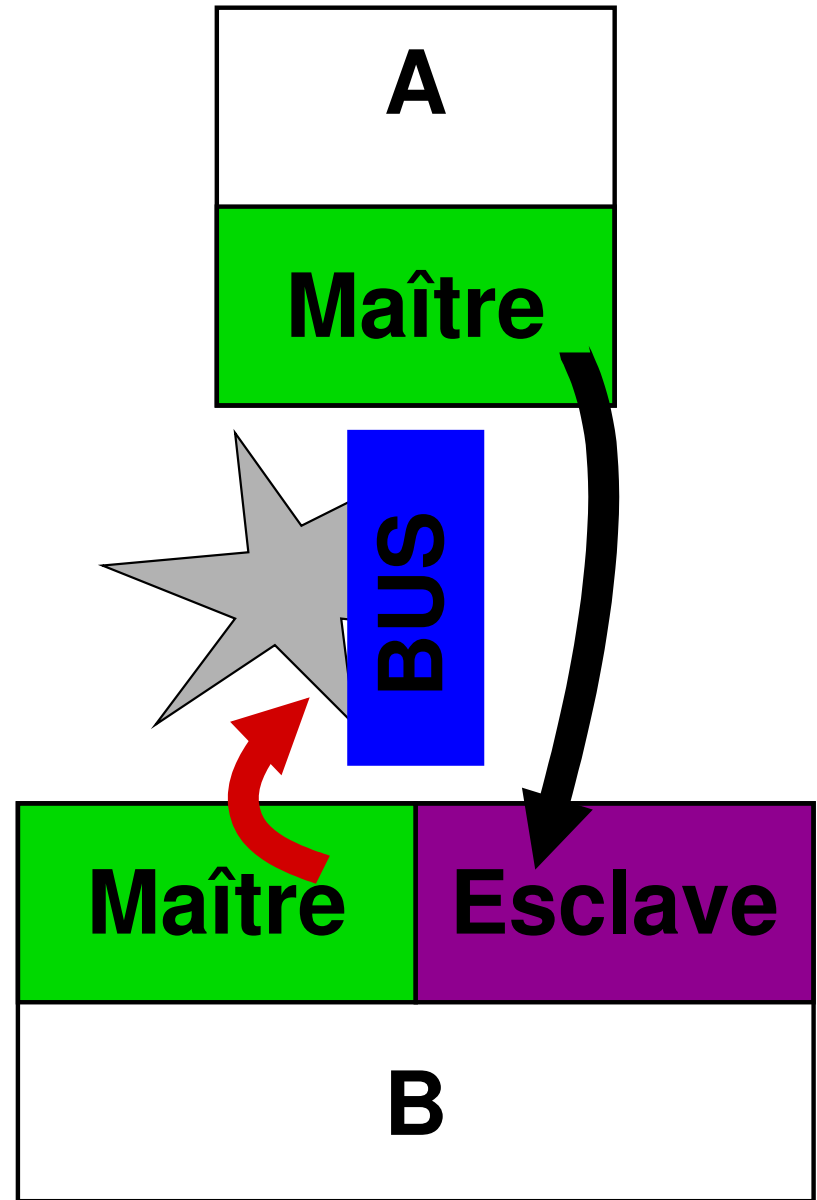
Les solutions à l'interblocage sont:

La prévention

- Algorithme
- Dimensionnement
- Topologie

La guérison

- Détection
- Préemption



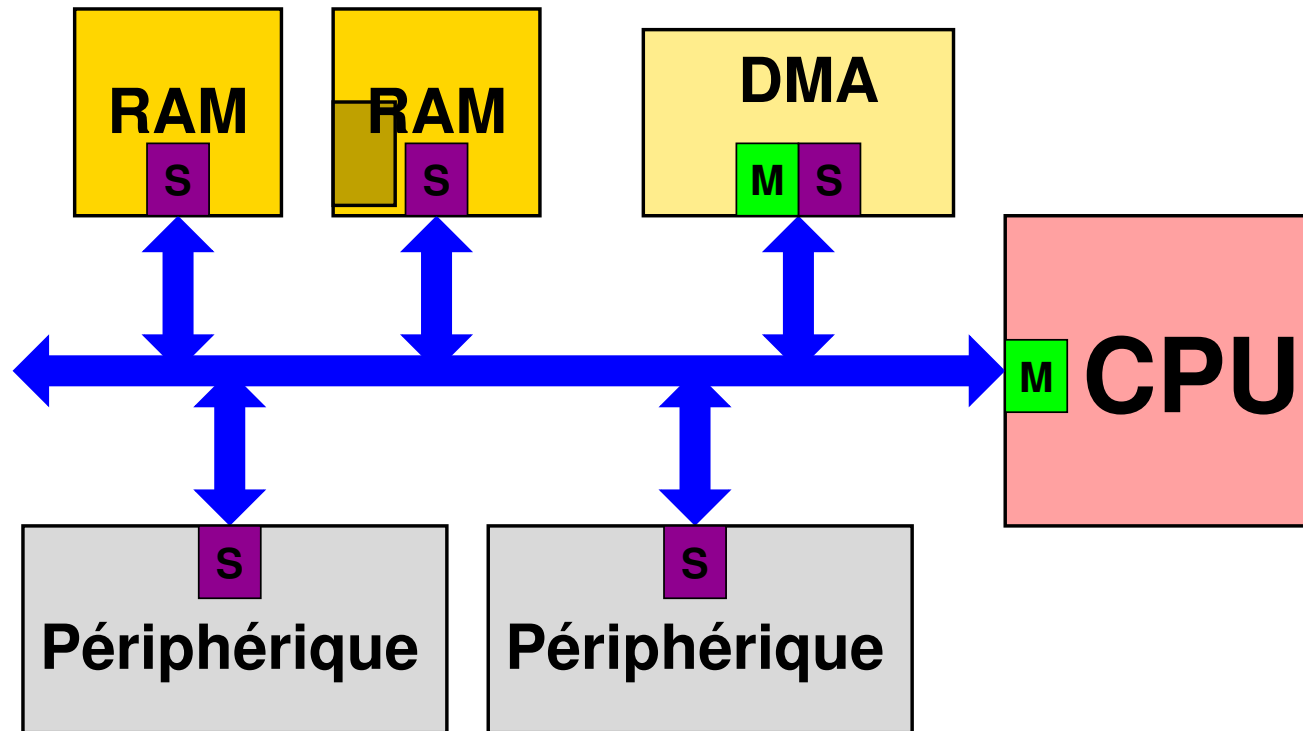
Plan

- ❏ Bus système
- ❏ Éléments d'un bus système
- ✖ Infrastructure de communication
 - ❏ Topologie
 - ✓ DMA
- ❏ Bus systèmes
- ❏ Network on Chip
- ❏ Conclusion

DMA: “Direct Memory Access”

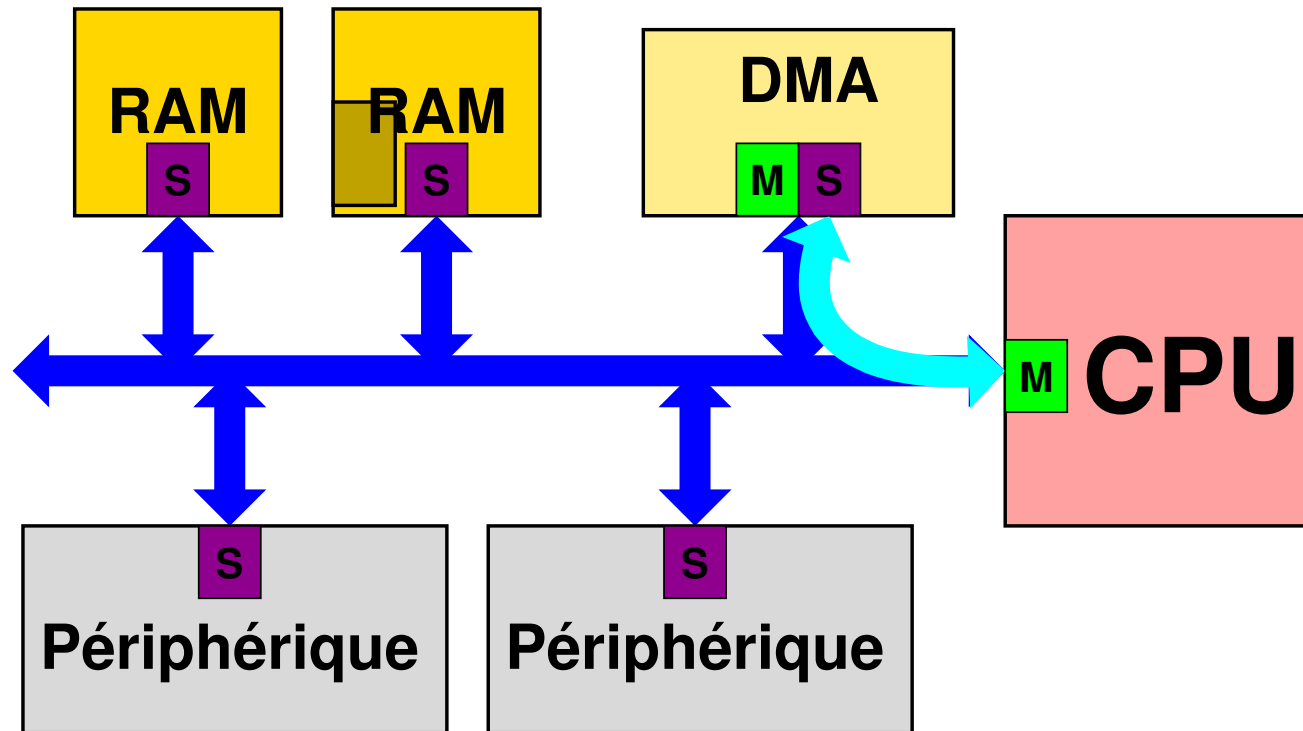
Un DMA est un automate de transfert automatique de données. Il organise les transferts massifs entre deux esclaves.

Il est programmé et configuré par affectation de registres.



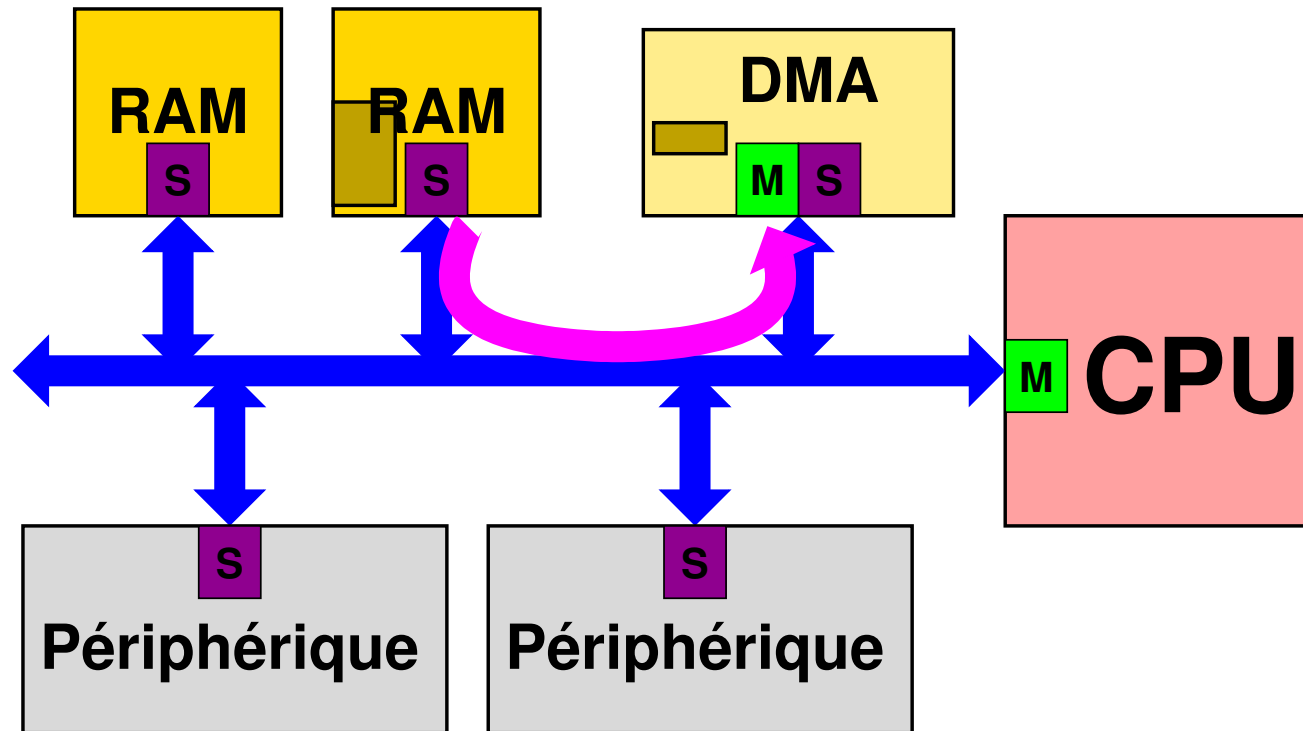
DMA: “Direct Memory Access”

Le logiciel applicatif configure le DMA, par des registres



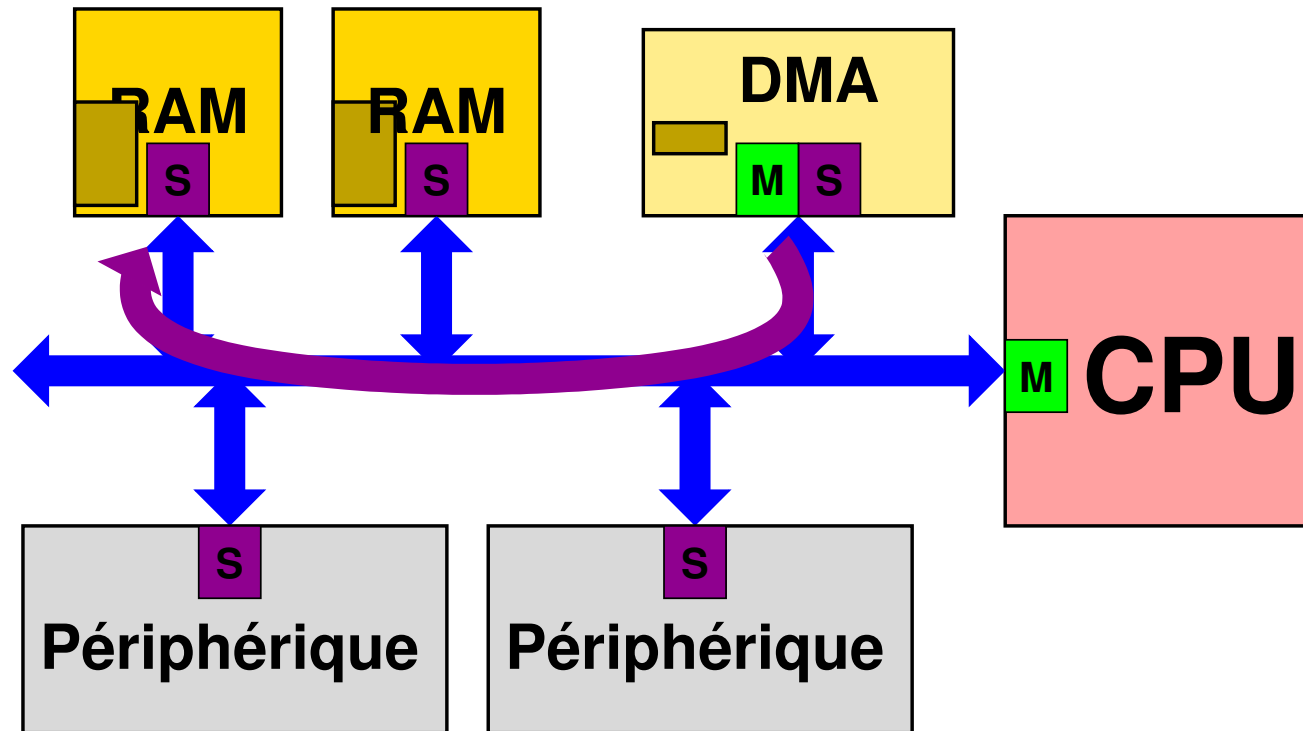
DMA: “Direct Memory Access”

Le DMA lit un premier burst depuis l'adresse source, stocke les données en interne



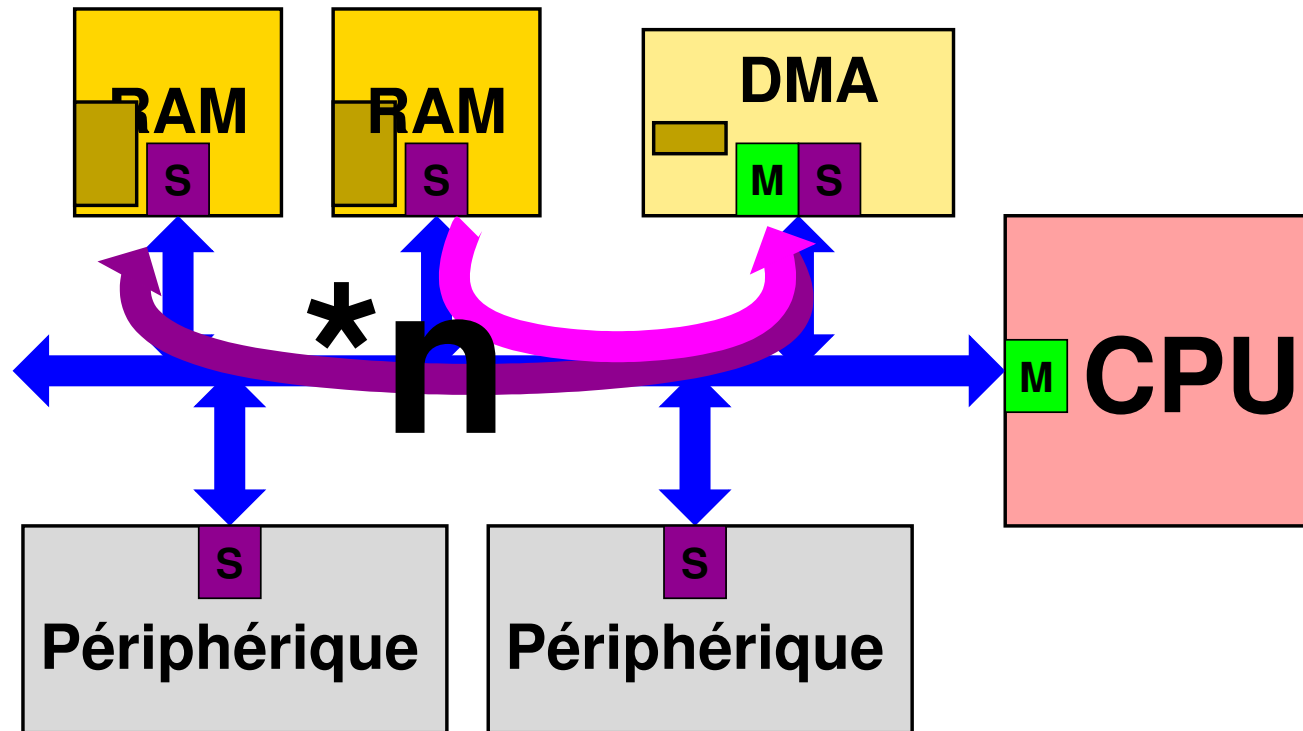
DMA: “Direct Memory Access”

Le burst est écrit vers l'adresse destination



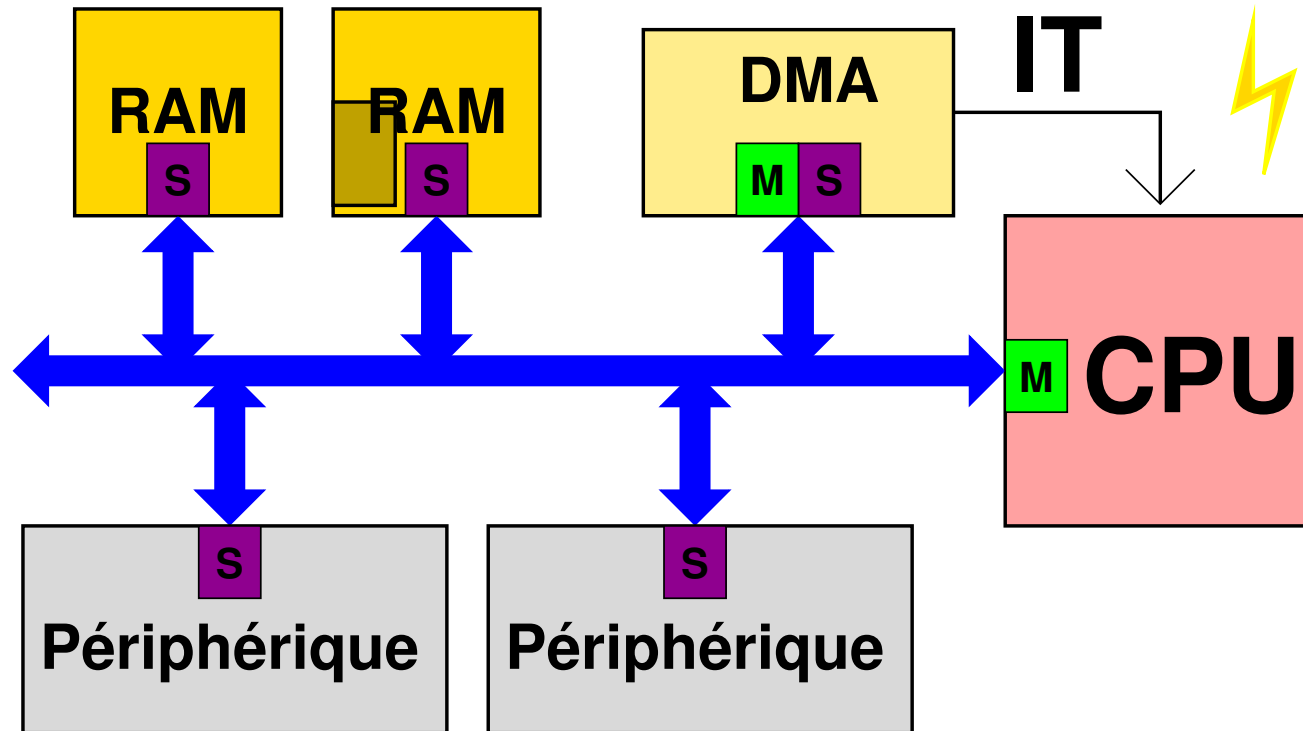
DMA: “Direct Memory Access”

... jusqu’au transfert de tout le paquet.



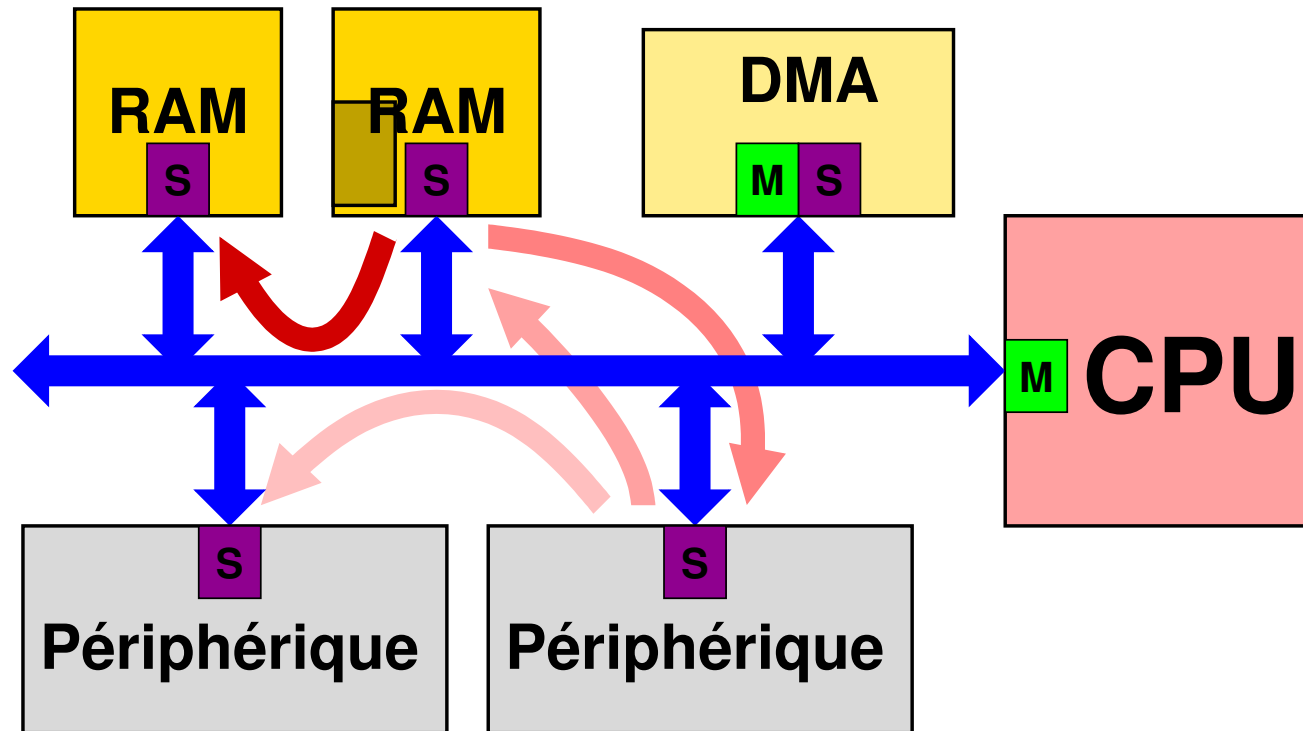
DMA: “Direct Memory Access”

Une interruption signale la fin.



DMA: “Direct Memory Access”

Par l’intermédiaire du DMA, les transferts peuvent se faire entre toutes sortes de périphériques!



Caractéristiques d'un DMA

❑ Nombre de canaux

❑ Gestion du bus

❑ Transferts

- Groupés (burst)
- Auto-incrémentés
- Canaux chaînés

❑ Priorité des canaux

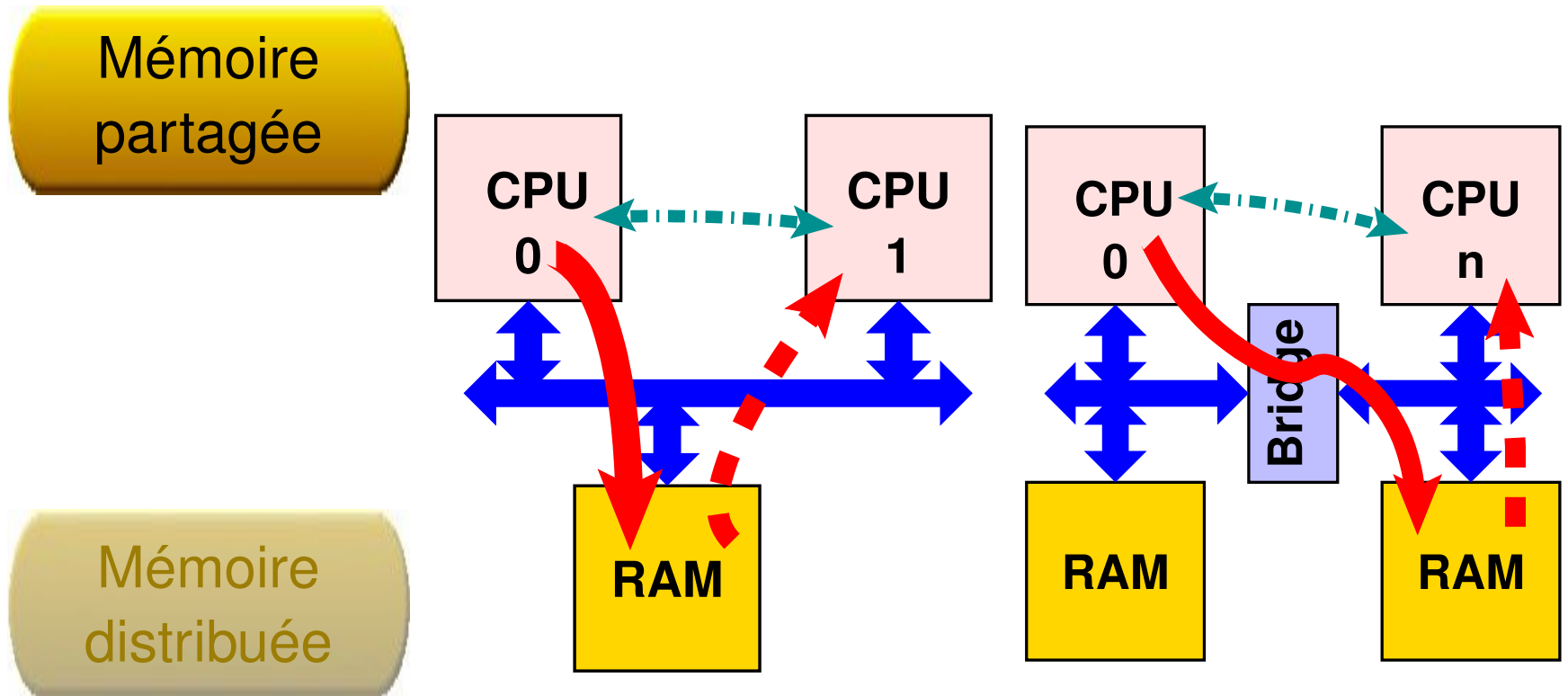
❑ Mode de transfert

- FIFO
- Taille des données

❑ Interruptions

- Fin de transfert
- Erreur

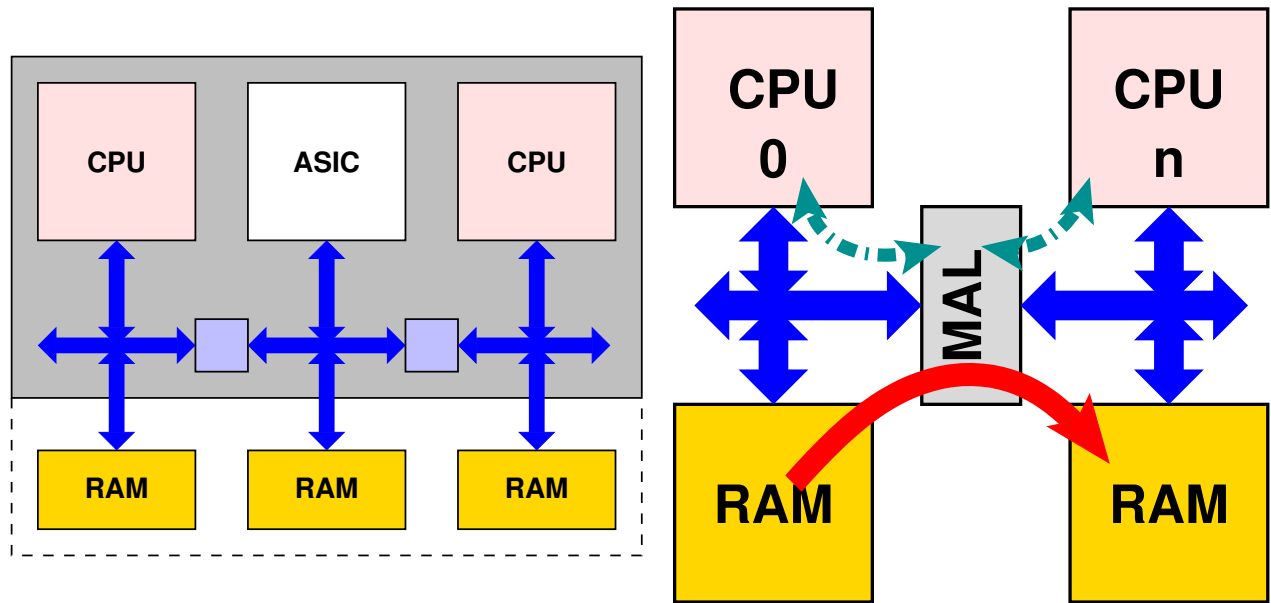
De CPU à CPU



De CPU à CPU

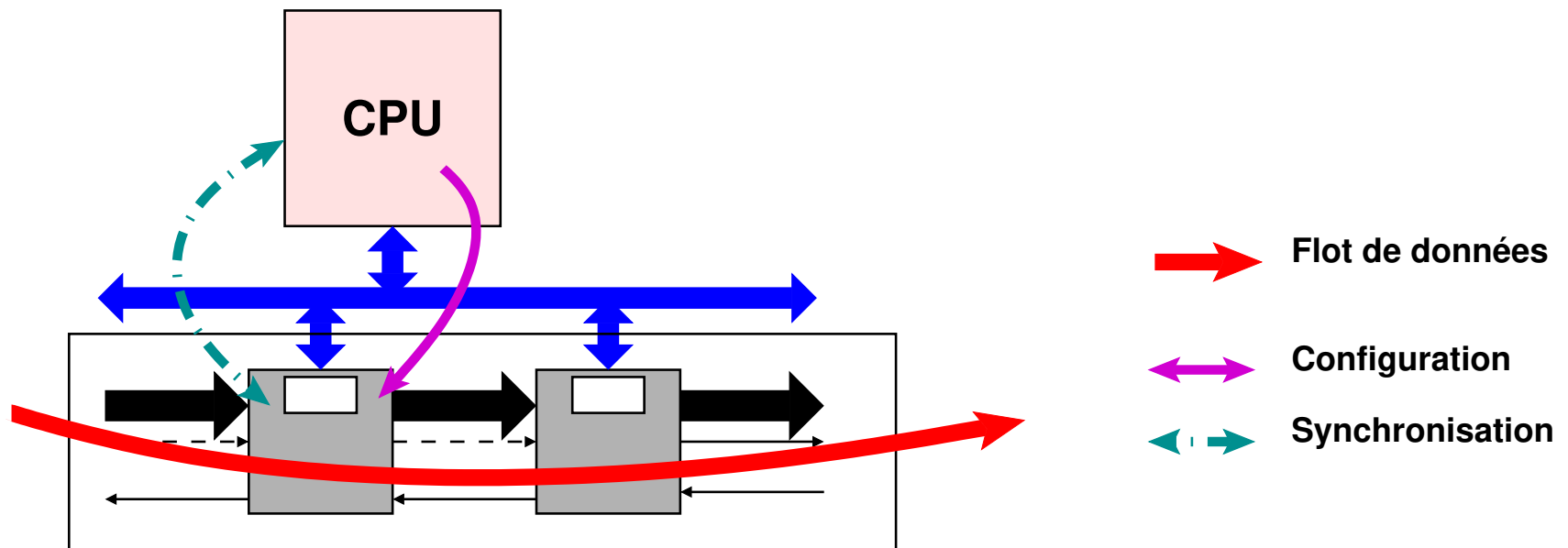
Mémoire partagée

Mémoire distribuée



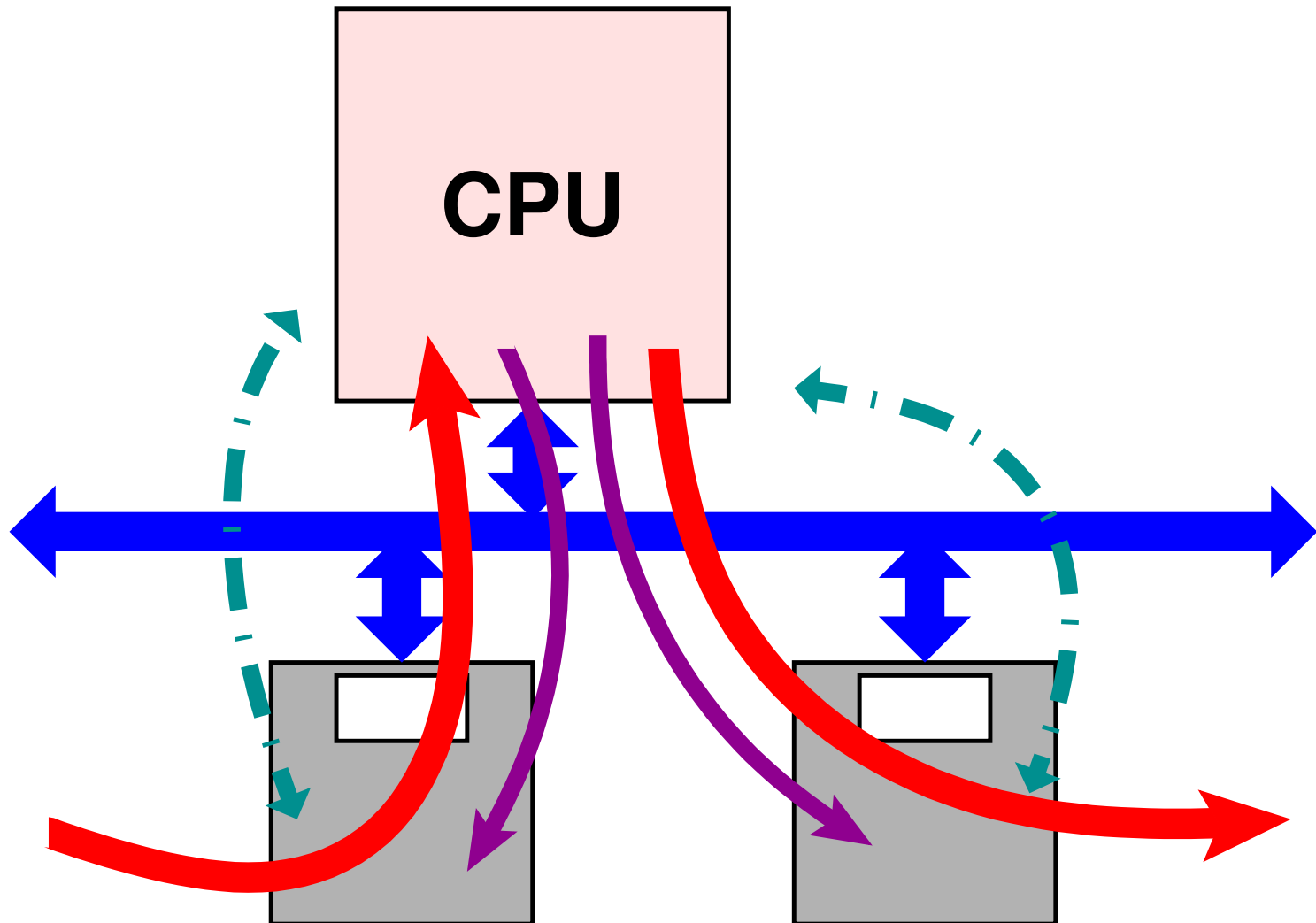
Contrôle d'un chemin de données

Un CPU contrôle un chemin de donnée en le configurant dynamiquement.



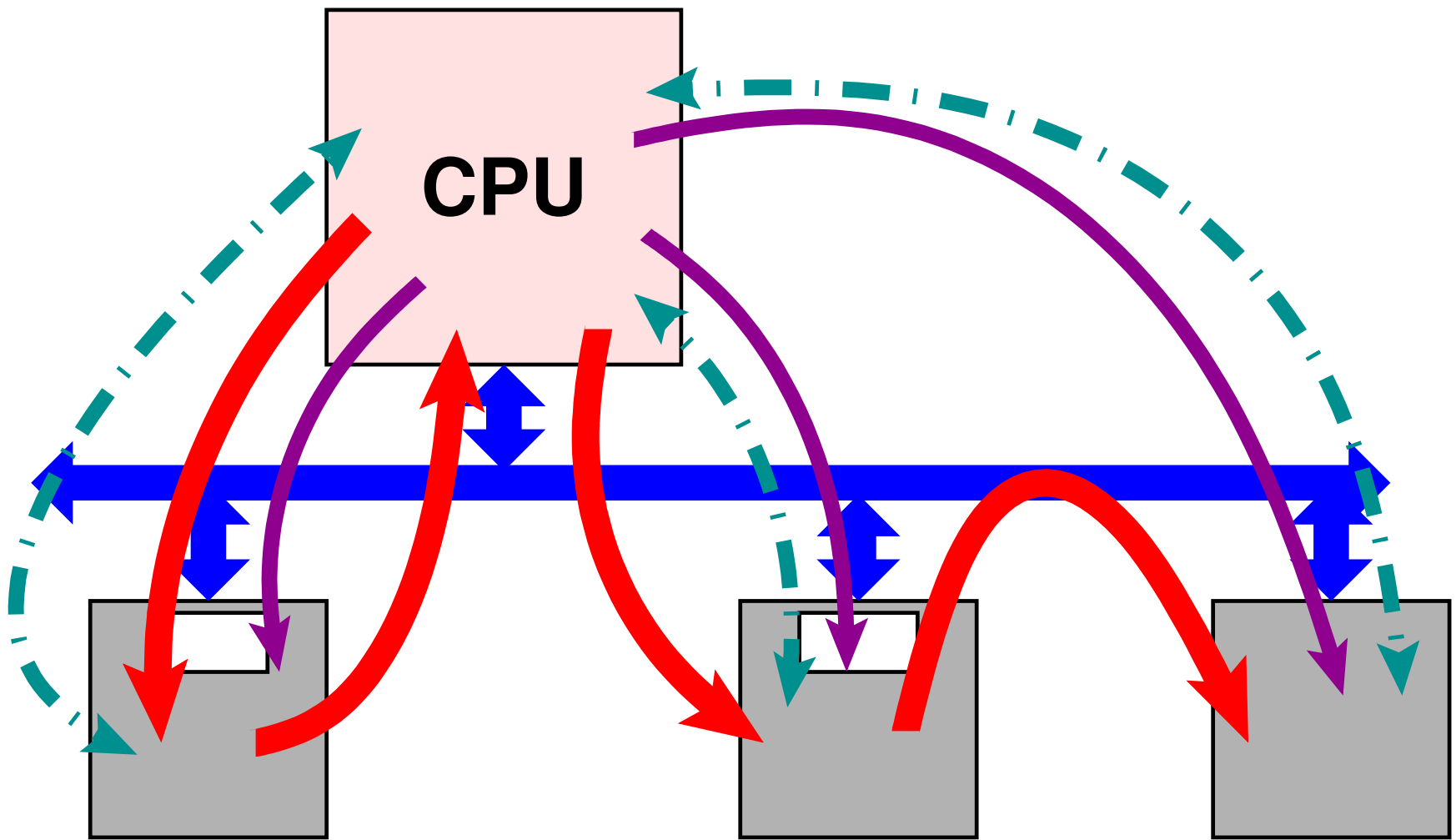
Terminal

Un CPU communique vers l'extérieur à travers un chemin de données.



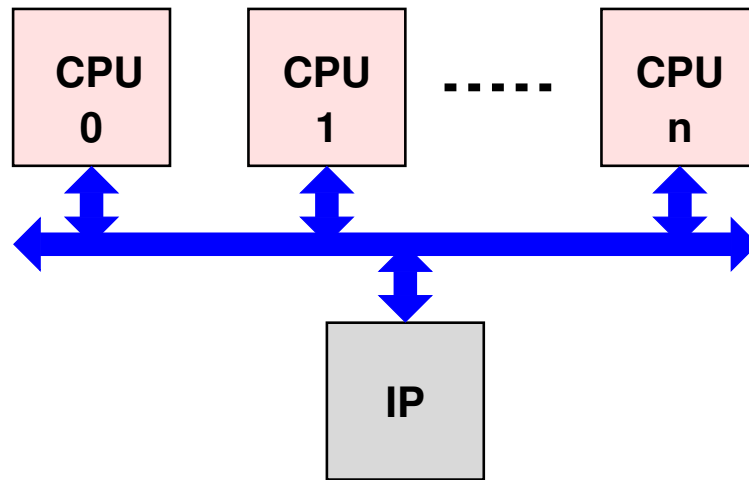
Coopération

Un CPU participe aux calculs. Il se sert des calculateurs comme des accélérateurs.

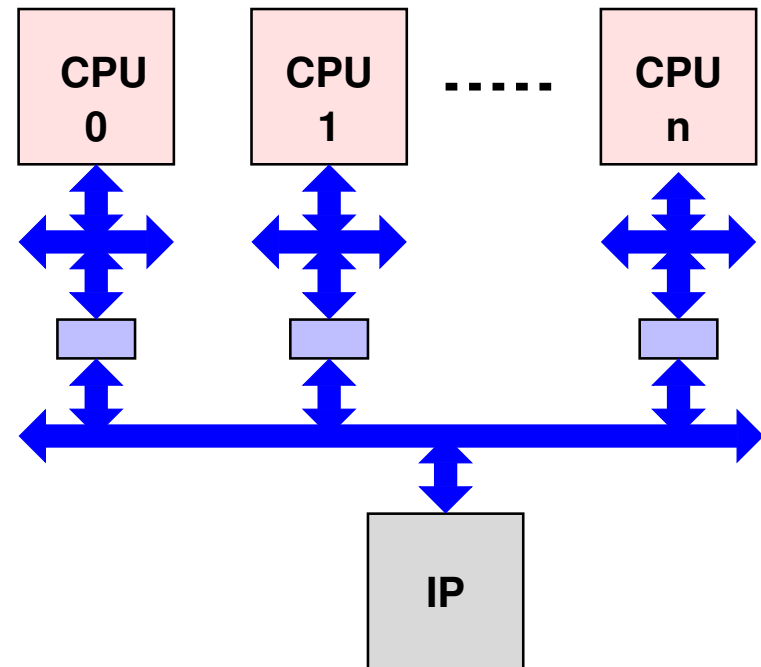


Accès à une ressource commune

Plusieurs CPUs accèdent à un accélérateur commun au travers des bus



Mémoire partagée



Mémoire distribuée

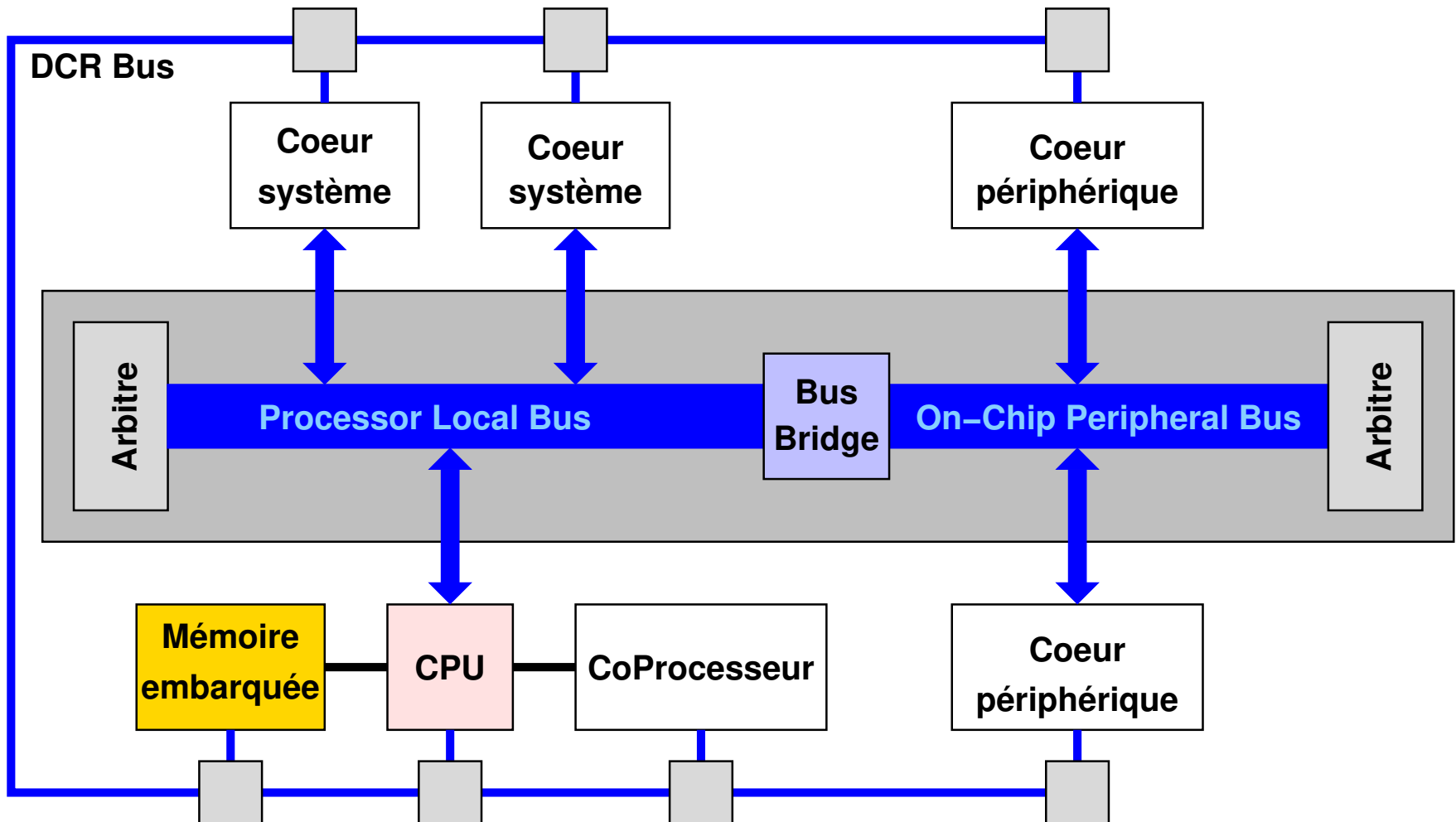
💣 La ressource doit être visible dans tous les espaces d'adressage.

Plan

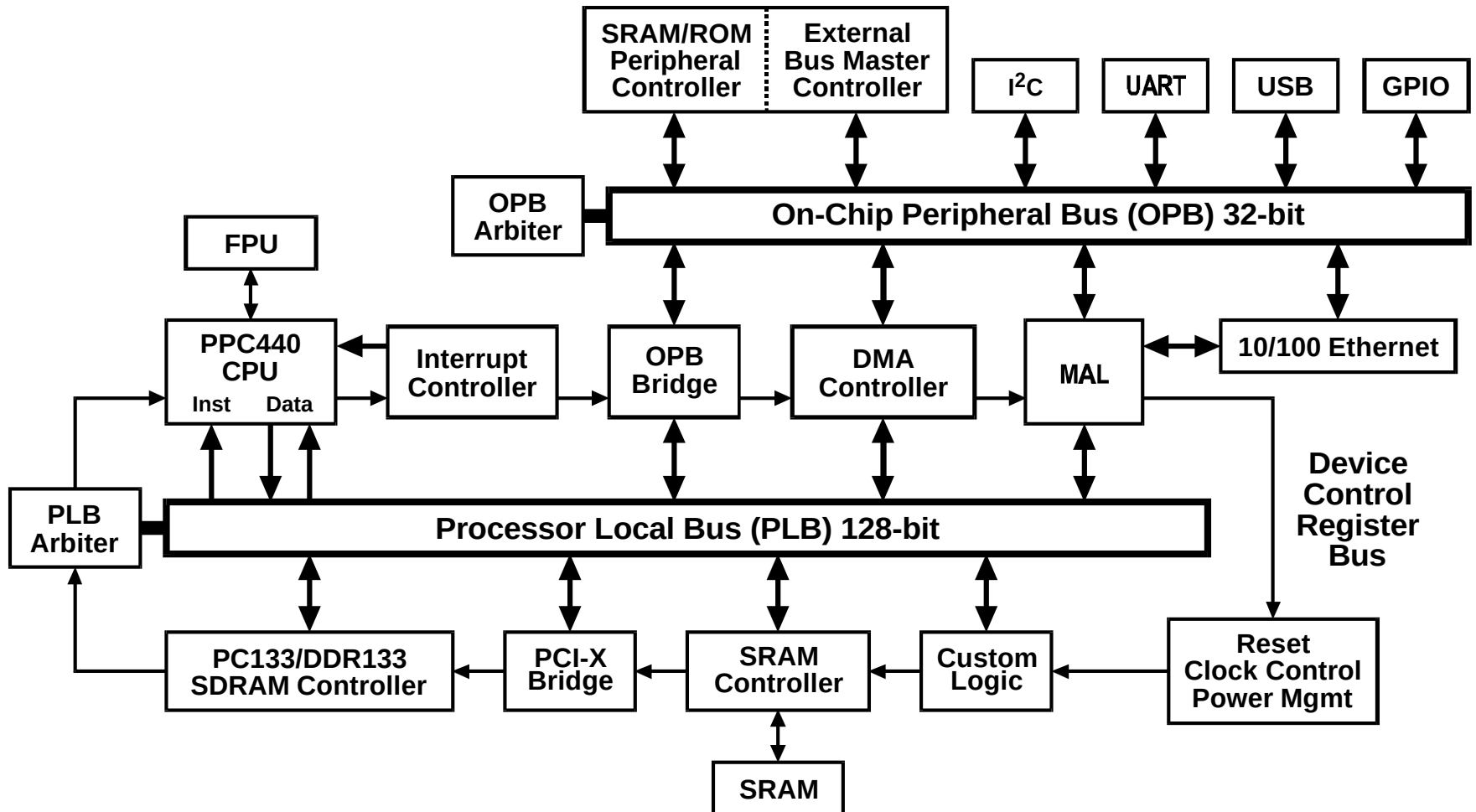
- ❑ Bus système
- ❑ Éléments d'un bus système
- ❑ Infrastructure de communication
- ✖ Bus systèmes
 - ✓ CoreConnect
 - ❑ AMBA
- ❑ Network on Chip
- ❑ Conclusion

Architecture

CoreConnect est une architecture de bus proposé par IBM



CoreConnect - Exemple

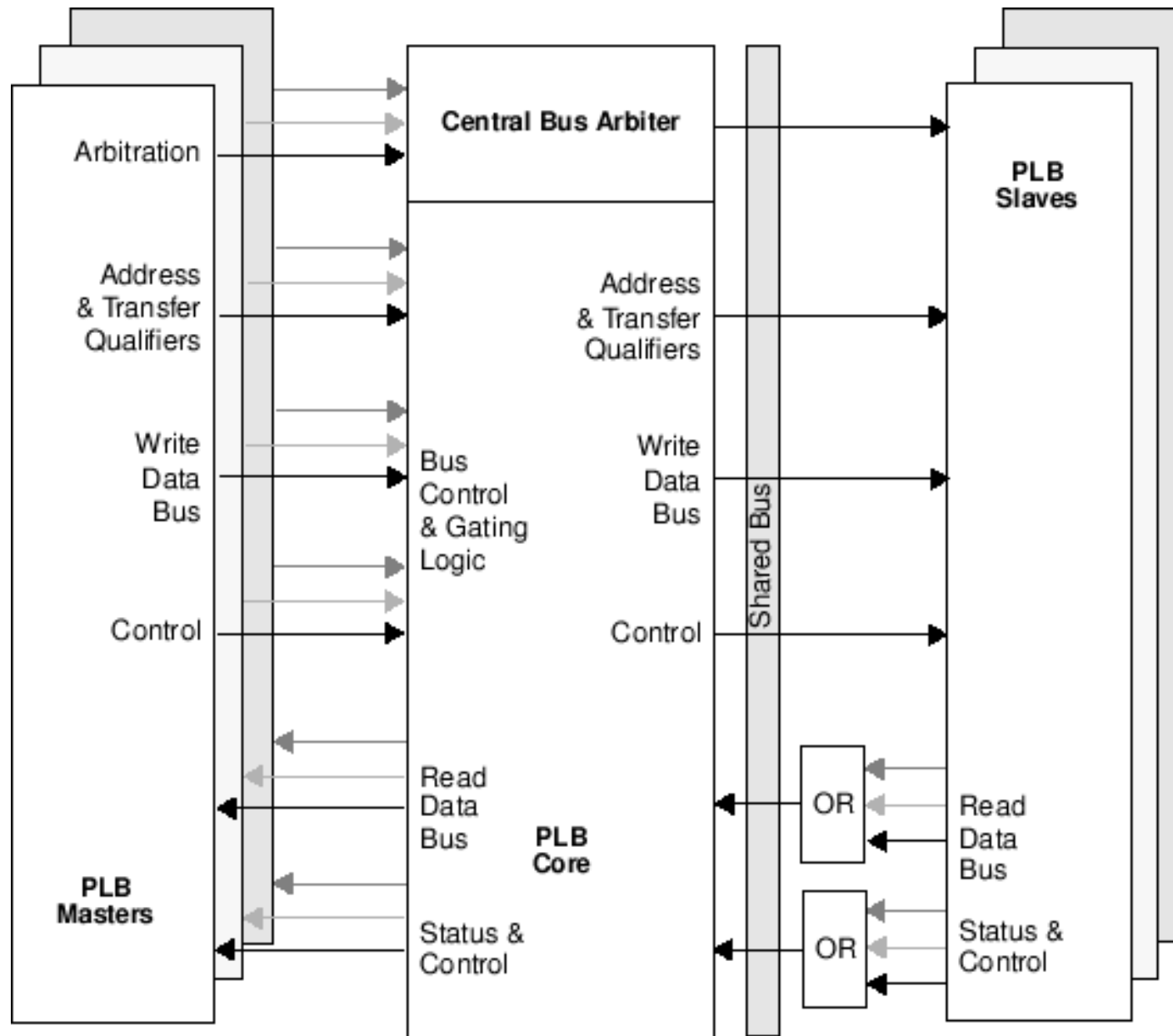


CoreConnect propose un coeur d'arbitre de bus PLB

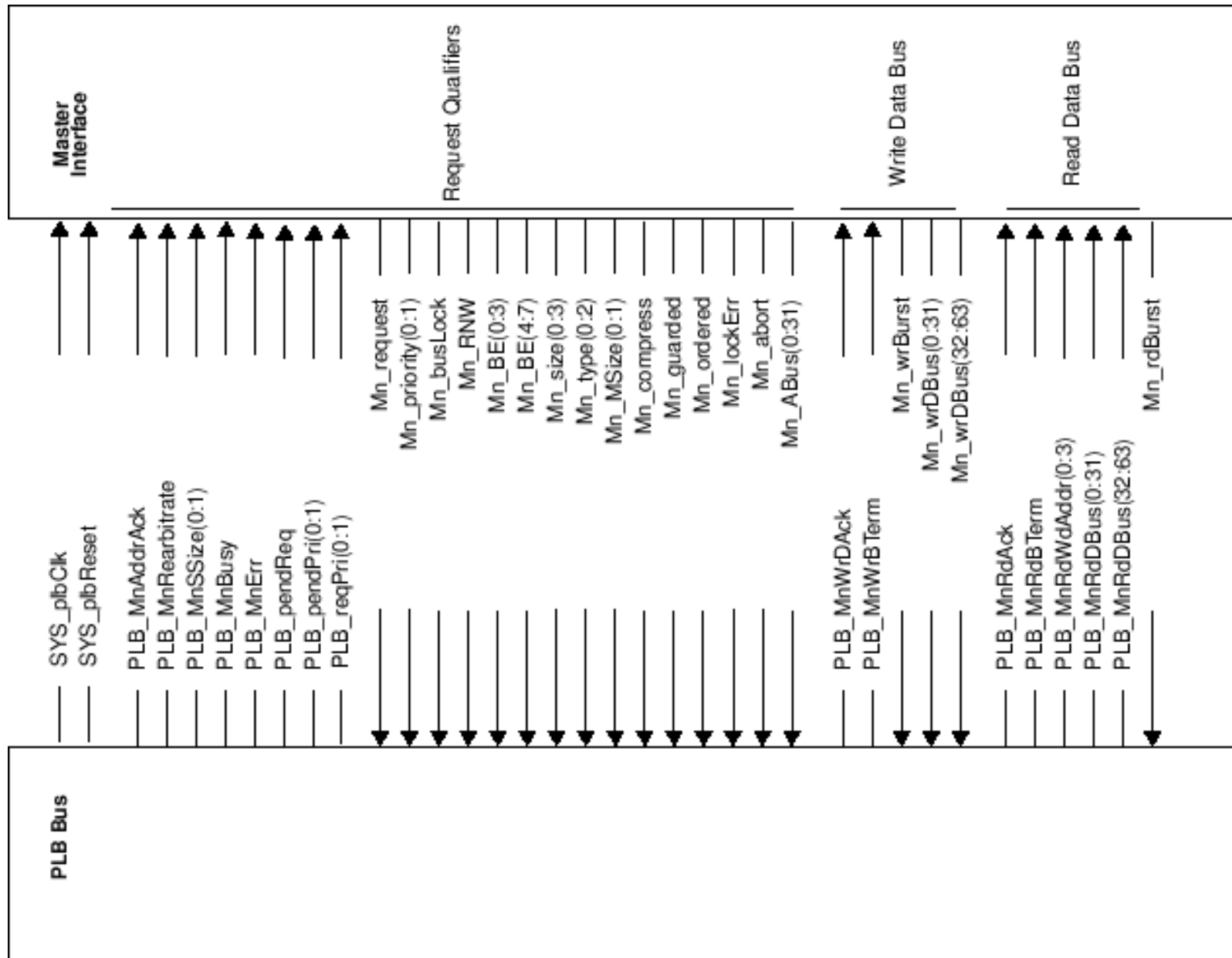
Caractéristiques :

- 8/16 Maîtres
- 8/16 jeux de priorité, sélectionnables dynamiquement
- 2 déclinaisons d'arbitrage
 - Fixe
 - “Juste”

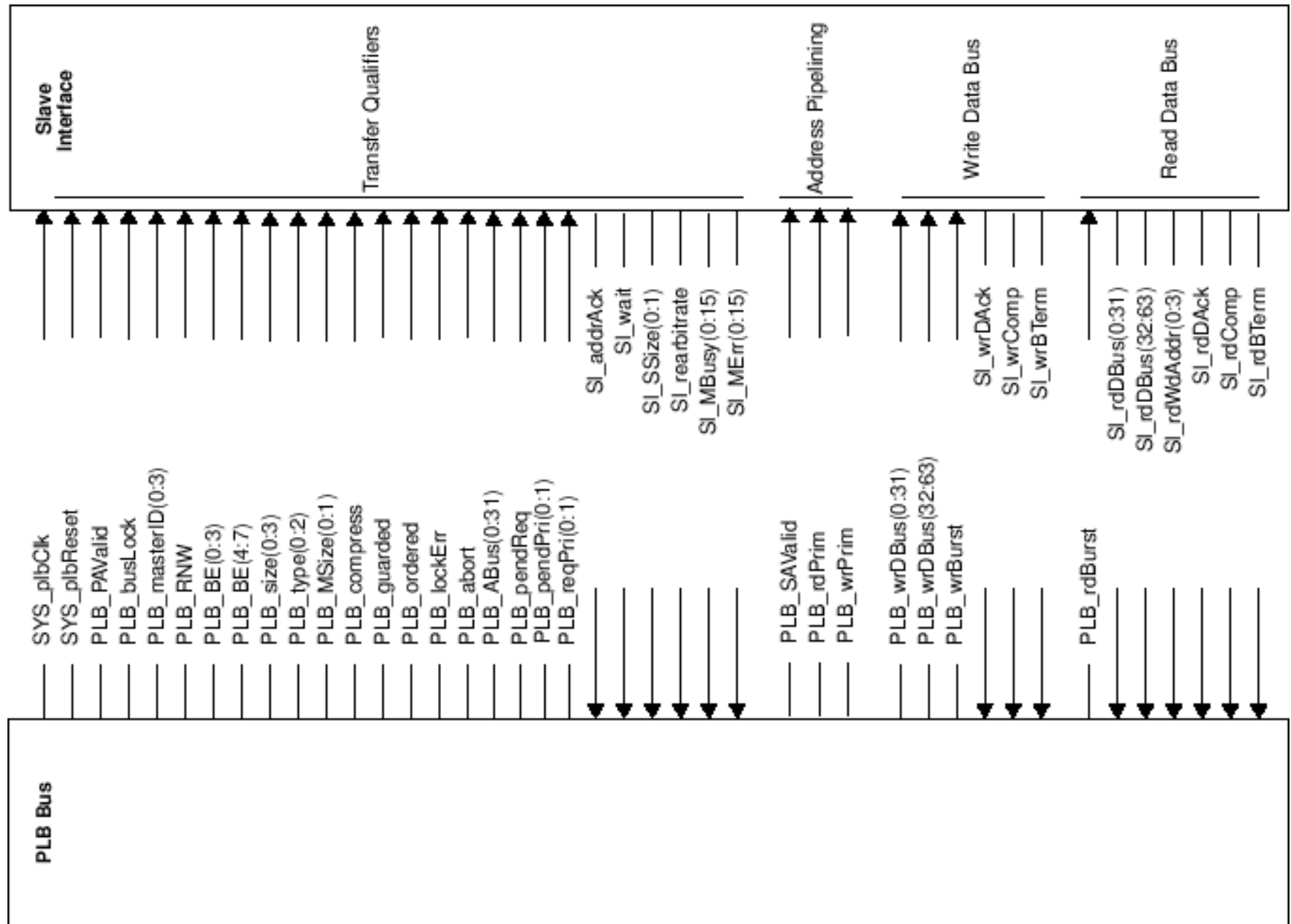
PLB Signaux



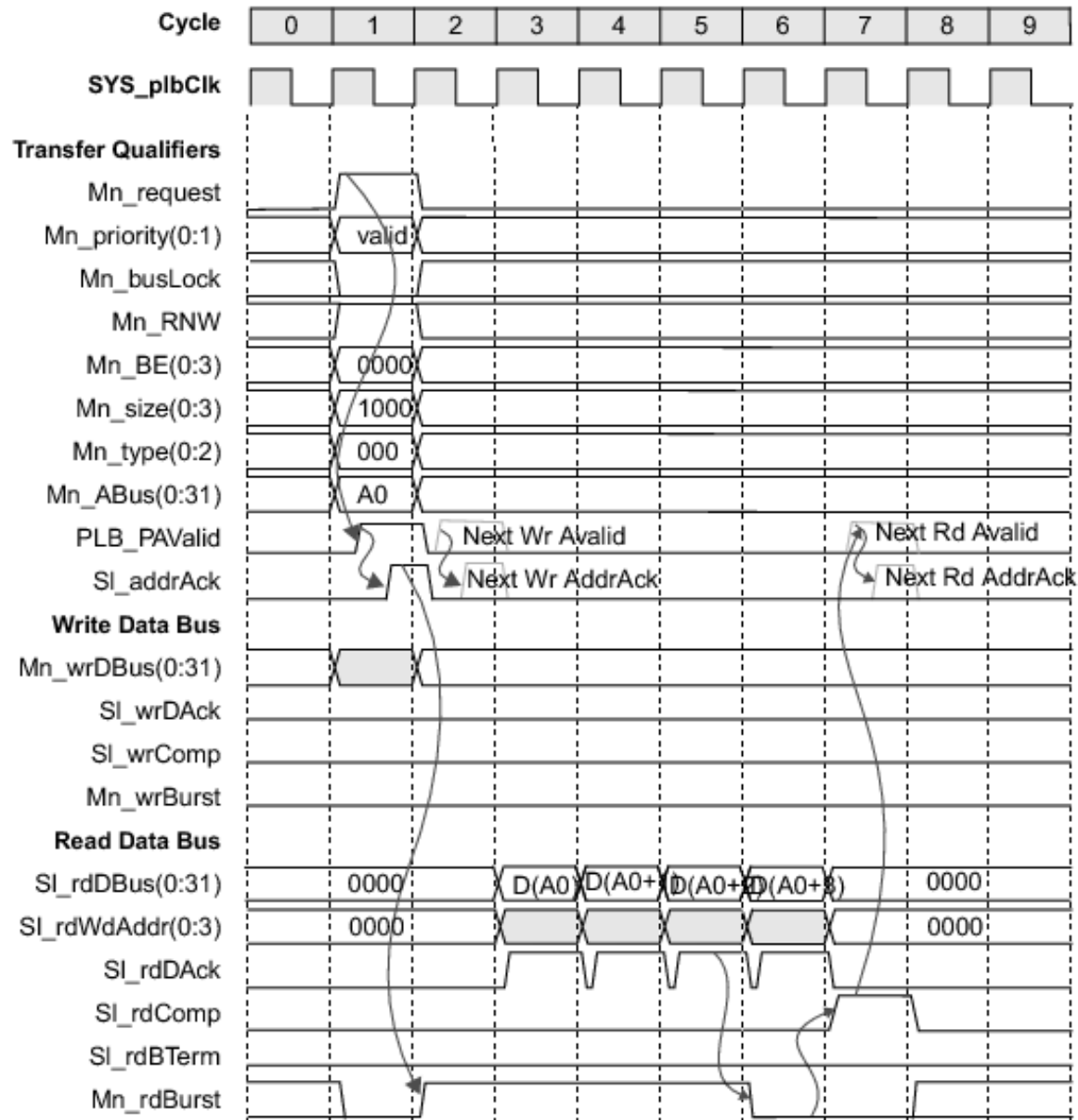
PLB Master



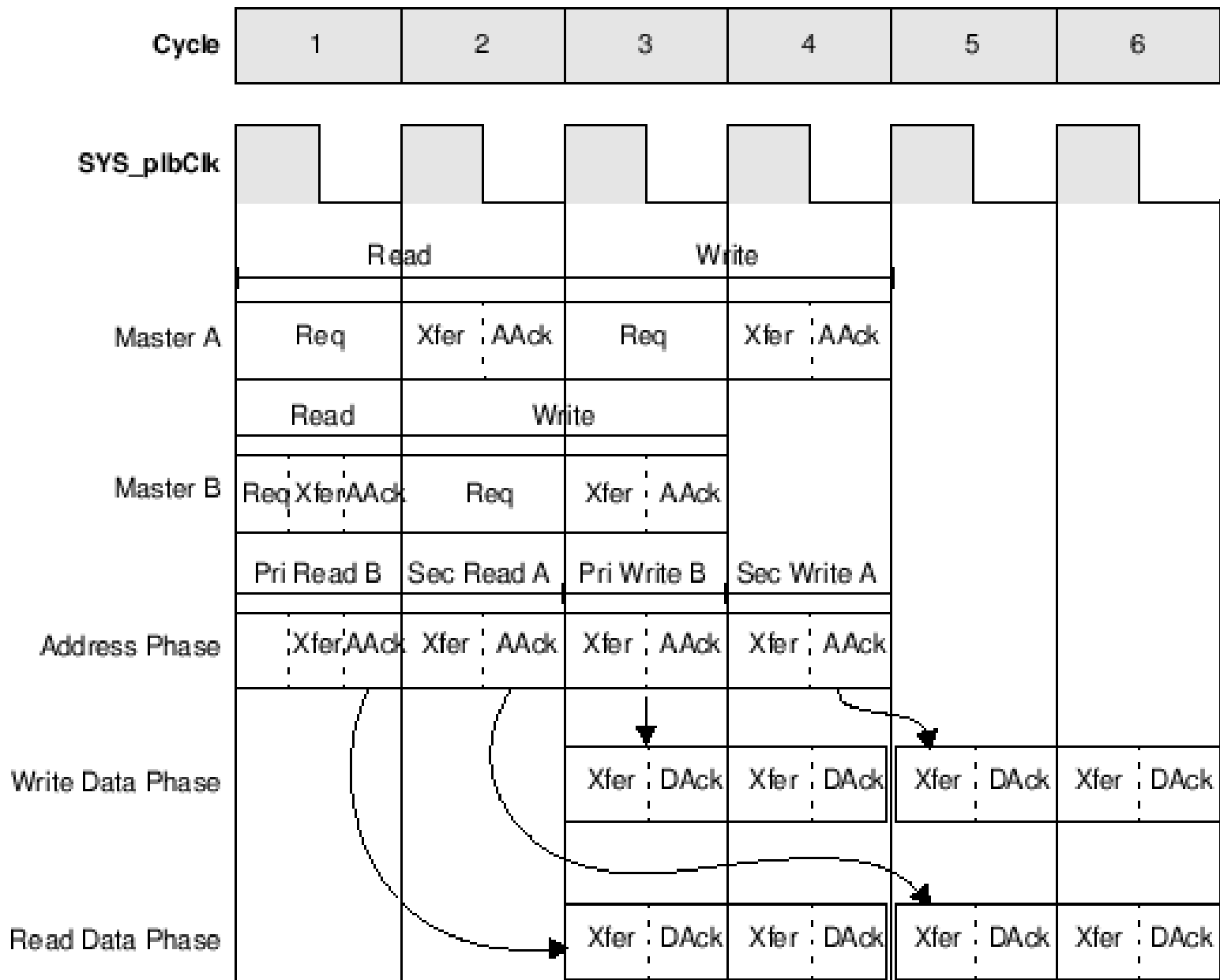
PLB Slave



Exemple de transaction: PLB burst read



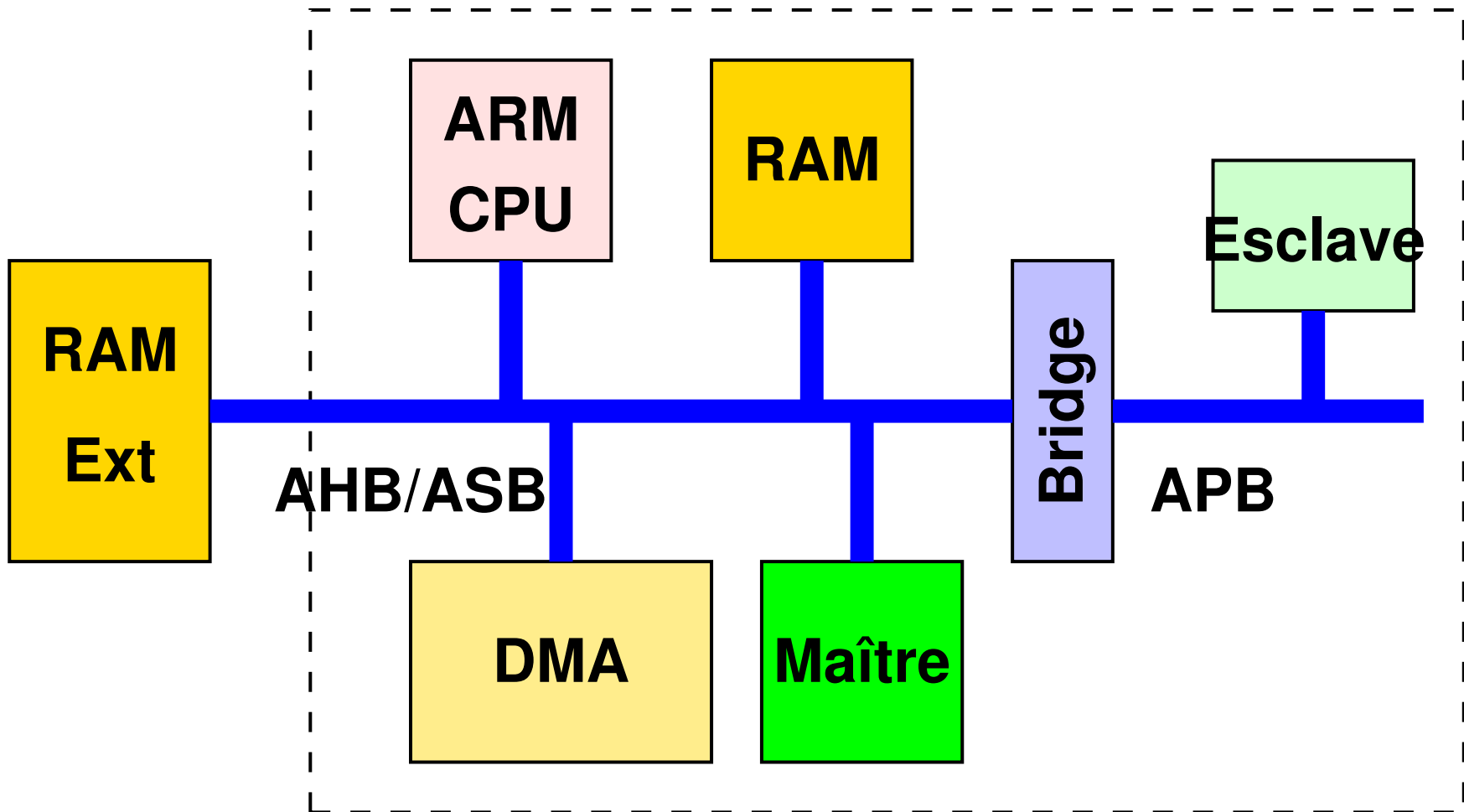
PLB & pipeline de transferts



Plan

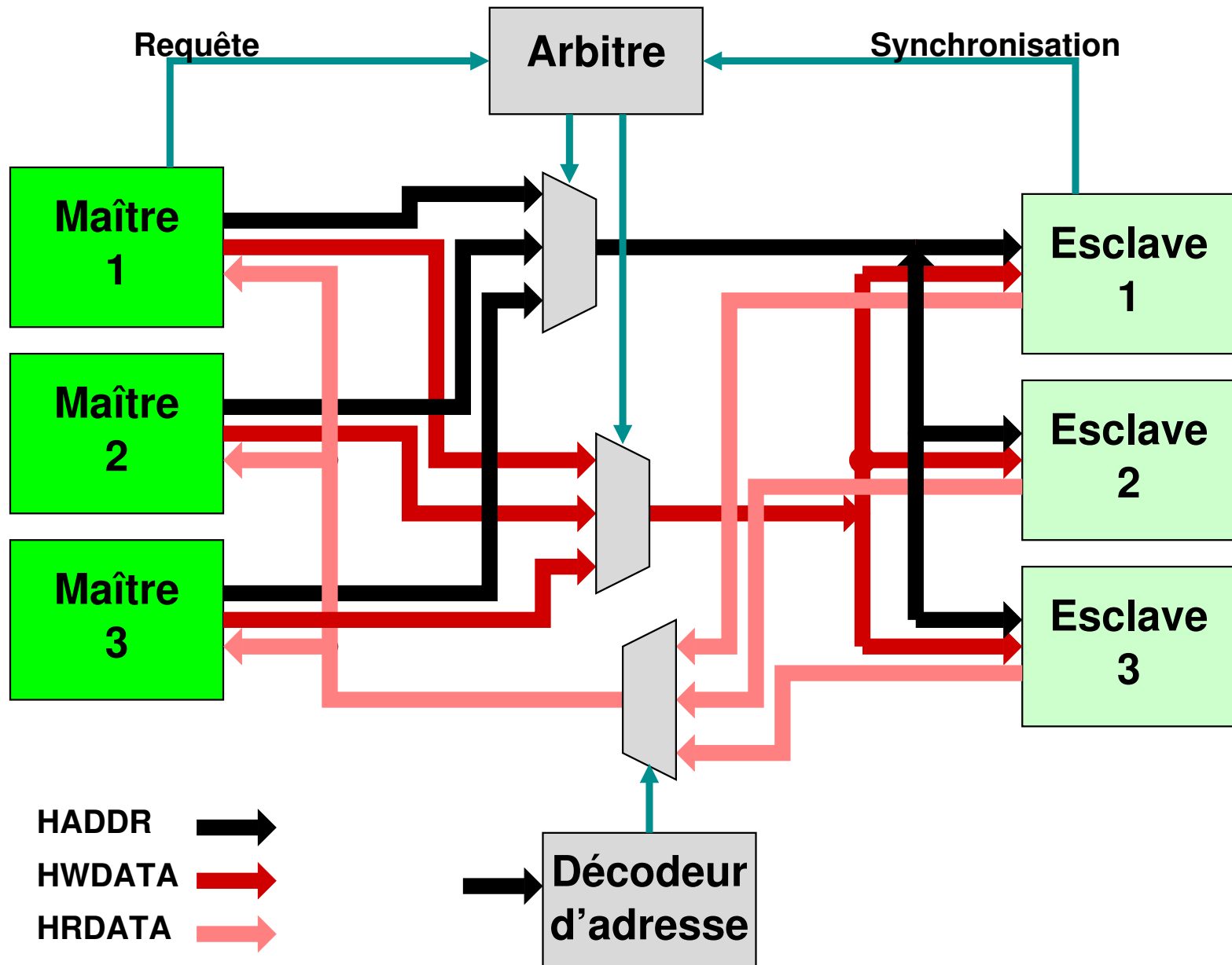
- ❏ Bus système
- ❏ Éléments d'un bus système
- ❏ Infrastructure de communication
- ✖ Bus systèmes
 - ❏ CoreConnect
 - ✓ AMBA
- ❏ Network on Chip
- ❏ Conclusion

AMBA - Architecture



- ❑ Transfert de données
- ❑ Contrôle
 - Type de transfert
 - Niveau de protection
 - Gestion des caches
- ❑ Arbitrage
 - Requête
 - Temporisation des réponses

Connectique AHB

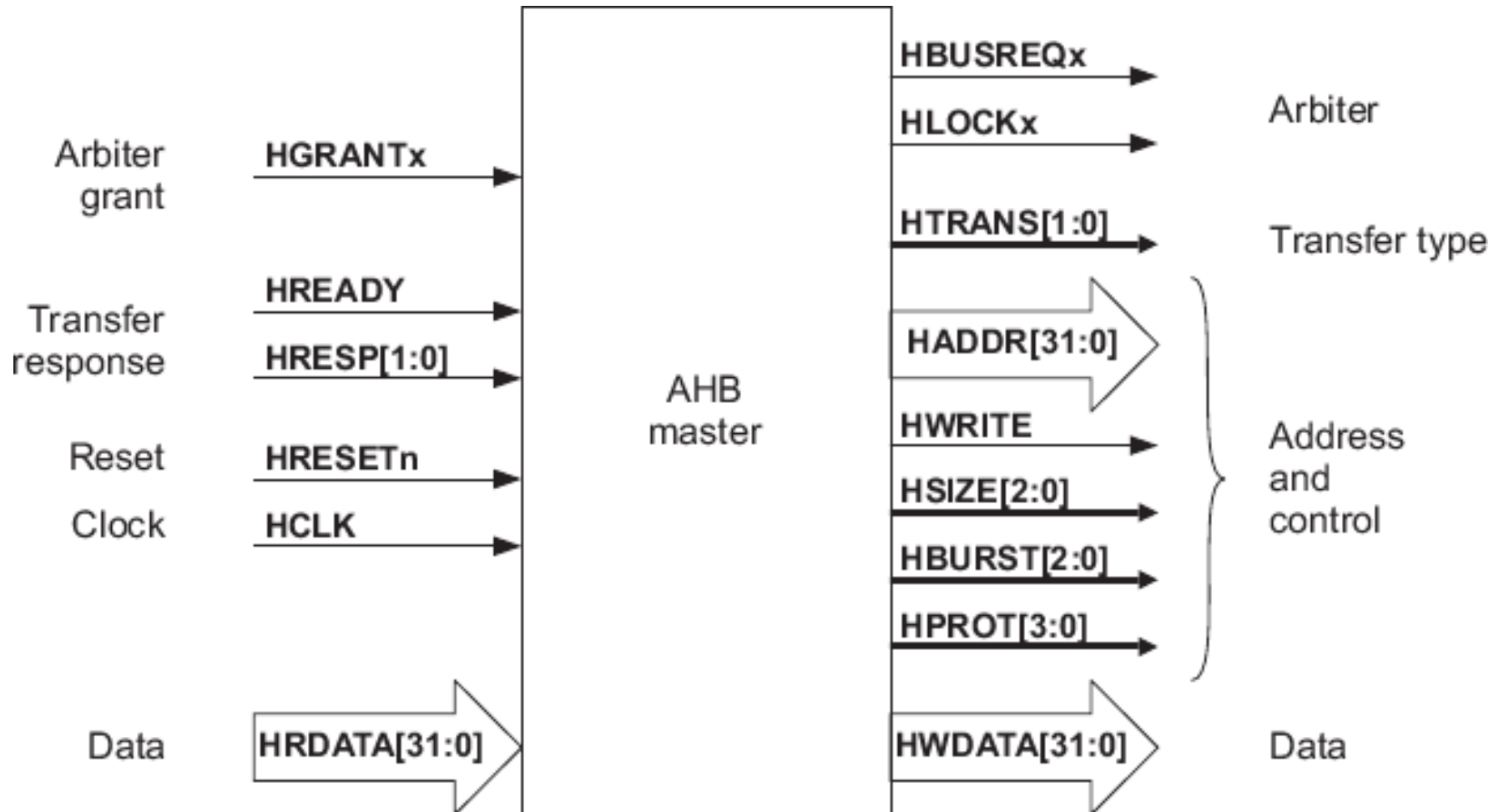


AMBA ne prévoit pas de politique d'arbitrage spécifique

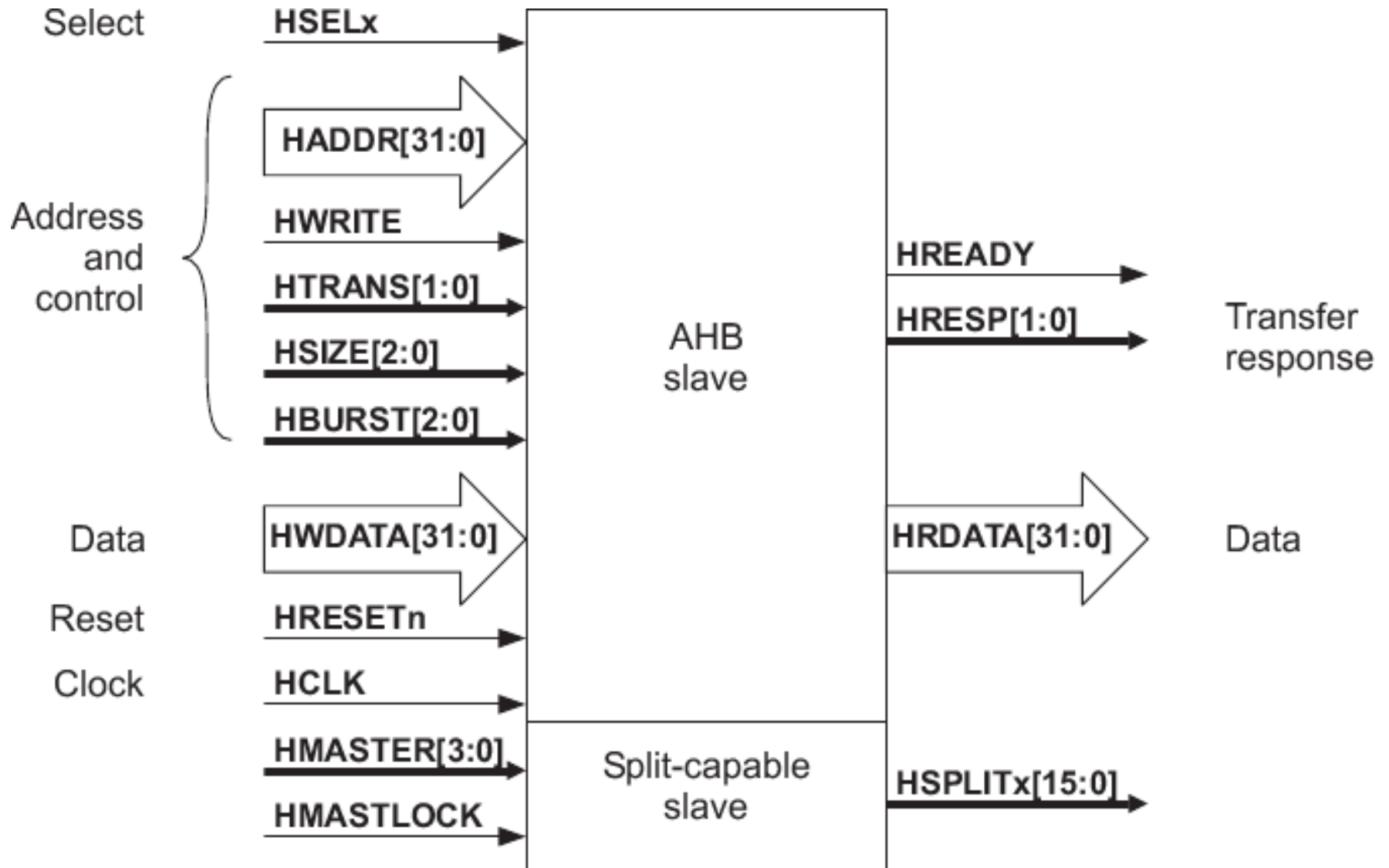
AMBA prévoit la préemption :

- Taille de paquet maximum
- Réponse de l'esclave différée

Exemple: AMBA AHB master



Exemple: AMBA AHB slave



Example: AMBA AHB protection

HPROT[3] cacheable	HPROT[2] bufferable	HPROT[1] privileged	HPROT[0] data/opcode	Description
-	-	-	0	Opcode fetch
-	-	-	1	Data access
-	-	0	-	User access
-	-	1	-	Privileged access
-	0	-	-	Not bufferable
-	1	-	-	Bufferable
0	-	-	-	Not cacheable
1	-	-	-	Cacheable

Résumé

Les bus systèmes implémentent des fonctions systèmes élaborées :

- ⚙ Gestion de la memory-map
- ⚙ Pour les OS et le multi-tâche multi-processeur, gestion des lecture/écriture atomiques, droits
- ⚙ Gestion des interblocages, ré-arbitrages, et fautes
- ⚙ Topologie à base de 'bridge' multi-bus
- ⚙ Gestion des DMA et transferts par paquets (burst) pour réduire le coût d'arbitrage

Plan

- ❑ Bus système
- ❑ Éléments d'un bus système
- ❑ Infrastructure de communication
- ❑ Bus systèmes
- ✖ Network on Chip
 - ✓ AMBA AXI
- ❑ Conclusion

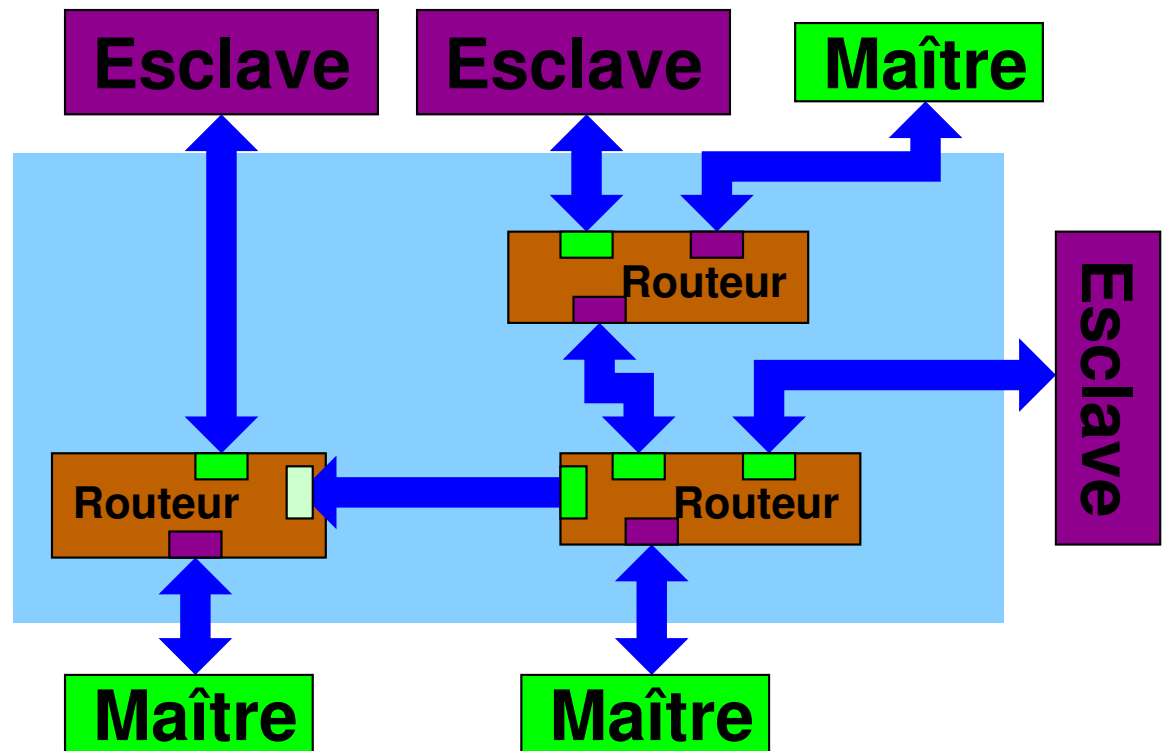
AMBA AXI

Le bus AMBA AXI (Advanced eXtensible Interface), de ARM, est un **bus orienté réseau**. Les transferts sont sur des 'canaux', en **point à point**. Le routage est effectué par des **routeurs**. **Network on Chip, Point to Point, Switch**

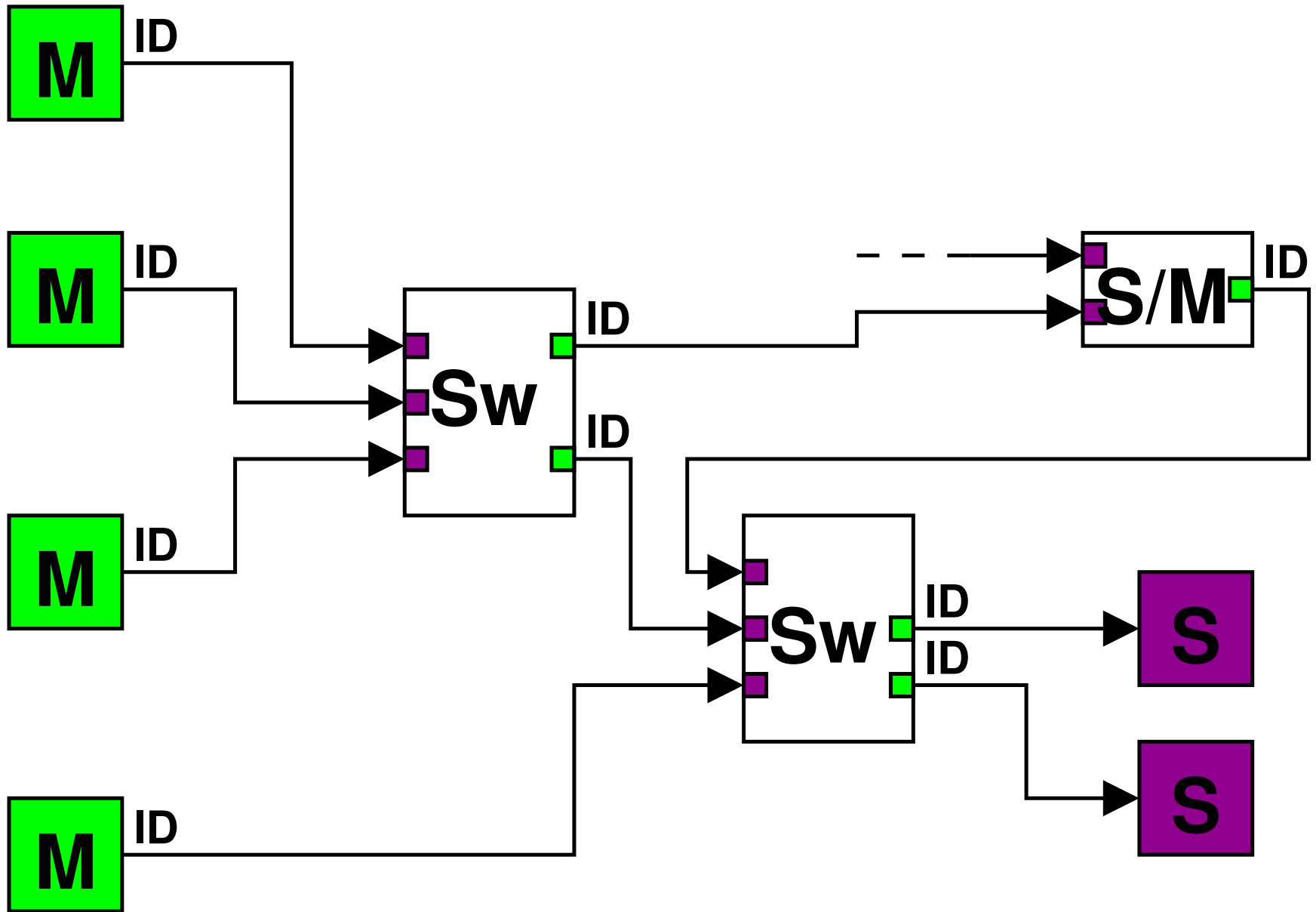
Les **canaux** sont:

- ☆ Transferts en **lecture**
 - ❑ read address
 - ❑ read data
- ☆ Transferts en **écriture**
 - ❑ write address
 - ❑ write data
 - ❑ write response

Les données vont de 8 à 1024 bits.



AXI, réseau point à point



AXI Read

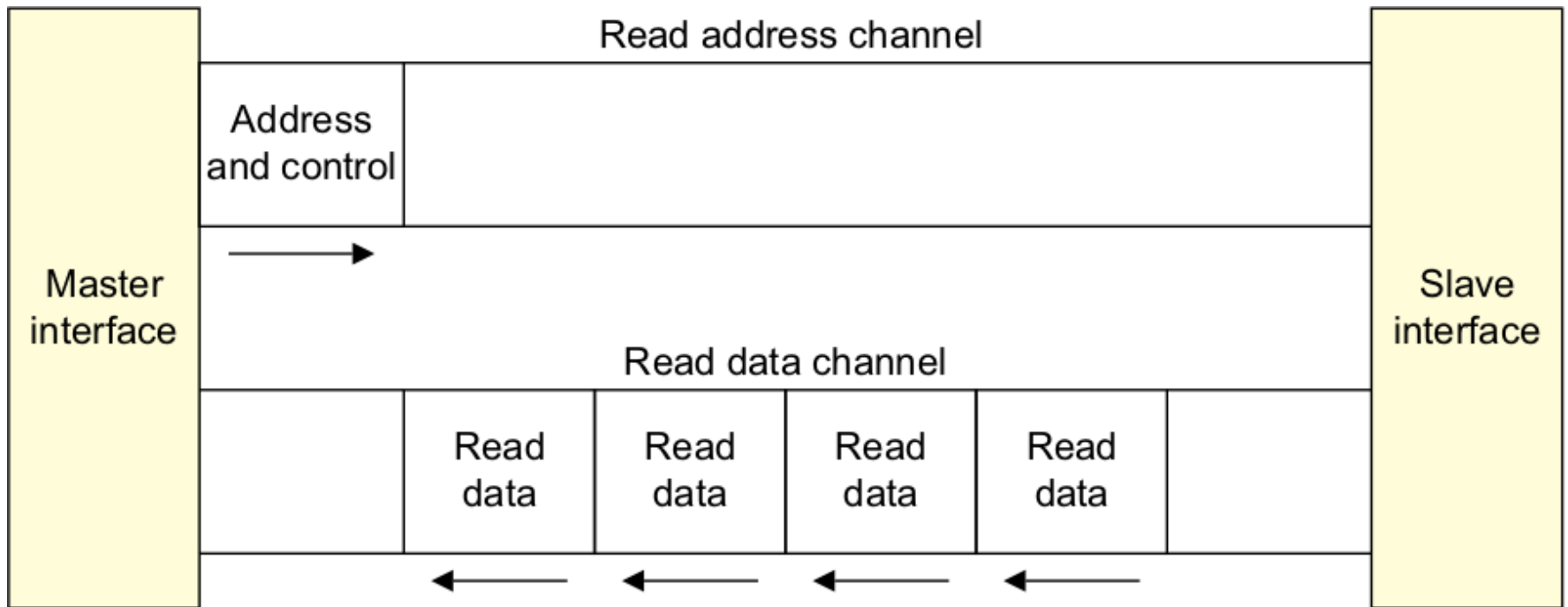


Figure A1-1 Channel architecture of reads

AXI Write

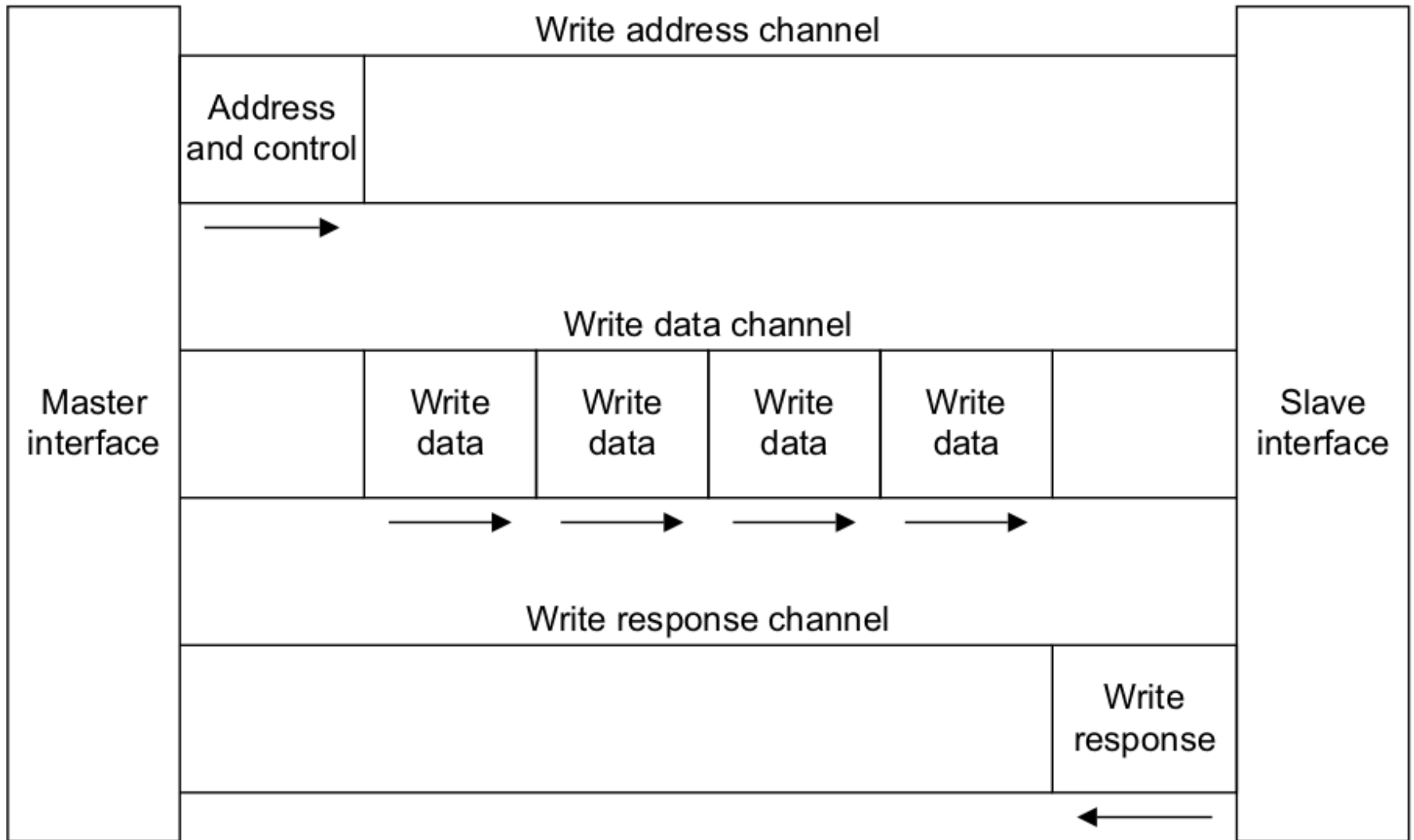


Figure A1-2 Channel architecture of writes

AXI Read, signaux de base

ARID	Master	Read address ID. This signal is the identification tag for the read address group of signals.
ARADDR	Master	Read address. The read address gives the address of the first transfer in a read burst transaction.
ARLEN	Master	Burst length. This signal indicates the exact number of transfers in a burst.
ARSIZE	Master	Burst size. This signal indicates the size of each transfer in the burst.
ARBURST	Master	Burst type. The burst type and the size information determine how the address for each transfer within the burst is calculated.

ARVALID	Master	Read address valid. This signal indicates that the channel is signaling valid read address and control information.
ARREADY	Slave	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals.

AXI Read, signaux complémentaires

ARUSER	Master	User signal. Optional User-defined signal in the read address channel.
ARLOCK	Master	Lock type. This signal provides additional information about the atomic characteristics of the transfer.
ARCACHE	Master	Memory type. This signal indicates how transactions are required to progress through a system.
ARPROT	Master	Protection type. This signal indicates the privilege and security level of the transaction, and whether the transaction is a data access or an instruction access.
ARQOS	Master	Quality of Service, QoS. QoS identifier sent for each read transaction.
ARREGION	Master	Region identifier. Permits a single physical interface on a slave to be used for multiple logical interfaces.

Problématique du routage

- ? Lorsqu'un routeur fait passer un message (read request), la réponse (read data), peut revenir plus tard, après d'autres messages.
- ? ☐ Comment savoir ou faire transiter la réponse?
- ? ☐ Doit on mémoriser tous les messages passés jusqu'au retour des données associées?

Sur un canal, chaque transfert est identifié par son ID:

- ❑ Les transferts de même ID sont effectués dans l'ordre
- ❑ Des transferts d'ID différents peuvent être traités dans le désordre par le périphérique

Exemple de transfert en lecture:

- ❑ Un maître émet une requête en plaçant les signaux de contrôle sur le canal d'émission, avec un ID.
- ❑ L'esclave répond plus tard, en plaçant à nouveau l'ID
- ❑ Entre-temps, le maître peut placer plusieurs requêtes, de même ID, ou différentes

AXI Signals Read Address

A2.5 Read address channel signals

Table A2-5 shows the AXI read address channel signals. Unless the description indicates otherwise, a signal is used by AXI3 and AXI4.

Table A2-5 Read address channel signals

Signal	Source	Description
ARID	Master	Read address ID. This signal is the identification tag for the read address group of signals. See Transaction ID on page A5-77 .
ARADDR	Master	Read address. The read address gives the address of the first transfer in a read burst transaction. See Address structure on page A3-44 .
ARLEN	Master	Burst length. This signal indicates the exact number of transfers in a burst. This changes between AXI3 and AXI4. See Burst length on page A3-44 .
ARSIZE	Master	Burst size. This signal indicates the size of each transfer in the burst. See Burst size on page A3-45 .
ARBURST	Master	Burst type. The burst type and the size information determine how the address for each transfer within the burst is calculated. See Burst type on page A3-45 .
ARLOCK	Master	Lock type. This signal provides additional information about the atomic characteristics of the transfer. This changes between AXI3 and AXI4. See Locked accesses on page A7-95 .
ARCACHE	Master	Memory type. This signal indicates how transactions are required to progress through a system. See Memory types on page A4-65 .
ARPROT	Master	Protection type. This signal indicates the privilege and security level of the transaction, and whether the transaction is a data access or an instruction access. See Access permissions on page A4-71 .
ARQOS	Master	<i>Quality of Service</i> , QoS. QoS identifier sent for each read transaction. Implemented only in AXI4. See QoS signaling on page A8-98 .
ARREGION	Master	Region identifier. Permits a single physical interface on a slave to be used for multiple logical interfaces. Implemented only in AXI4. See Multiple region signaling on page A8-99 .
ARUSER	Master	User signal. Optional User-defined signal in the read address channel. Supported only in AXI4. See User-defined signaling on page A8-100 .
ARVALID	Master	Read address valid. This signal indicates that the channel is signaling valid read address and control information. See Channel handshake signals on page A3-38 .
ARREADY	Slave	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals. See Channel handshake signals on page A3-38 .

AXI Signals Read Data

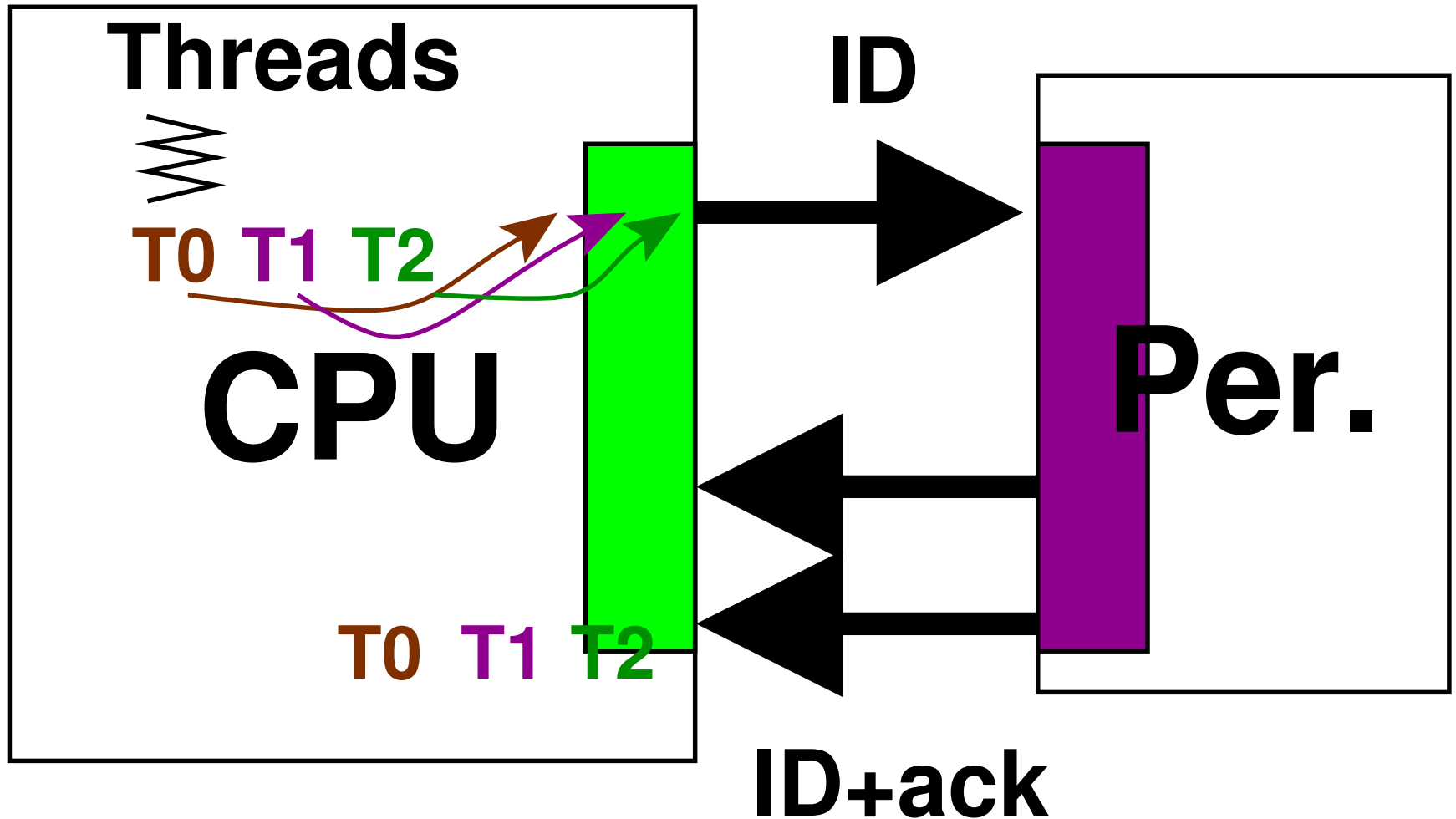
A2.6 Read data channel signals

Table A2-6 shows the AXI read data channel signals. Unless the description indicates otherwise, a signal is used by AXI3 and AXI4.

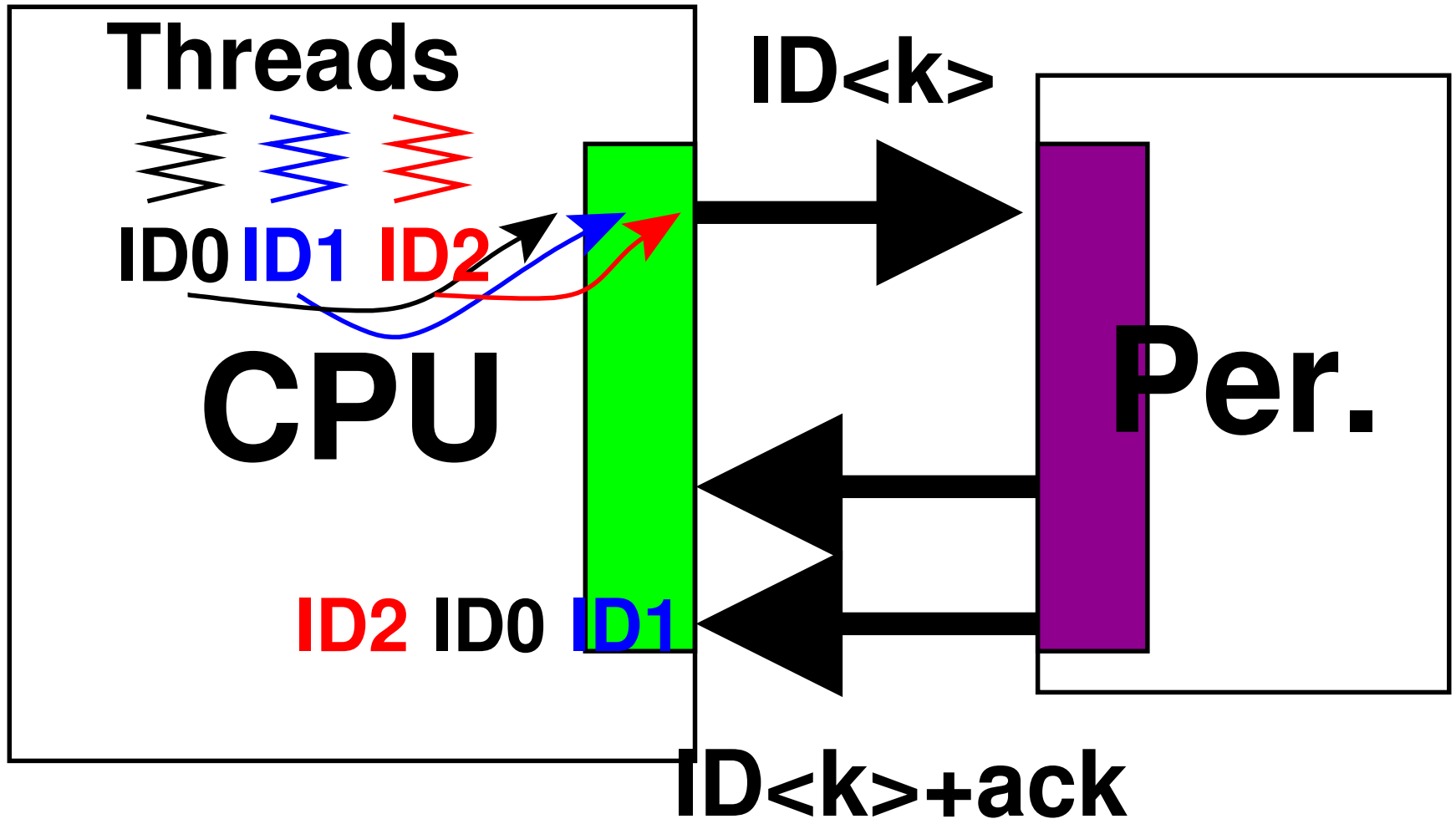
Table A2-6 Read data channel signals

Signal	Source	Description
RID	Slave	Read ID tag. This signal is the identification tag for the read data group of signals generated by the slave. See Transaction ID on page A5-77 .
RDATA	Slave	Read data.
RRESP	Slave	Read response. This signal indicates the status of the read transfer. See Read and write response structure on page A3-54 .
RLAST	Slave	Read last. This signal indicates the last transfer in a read burst. See Read data channel on page A3-39 .
RUSER	Slave	User signal. Optional User-defined signal in the read data channel. Supported only in AXI4. See User-defined signaling on page A8-100 .
RVALID	Slave	Read valid. This signal indicates that the channel is signaling the required read data. See Channel handshake signals on page A3-38 .
RREADY	Master	Read ready. This signal indicates that the master can accept the read data and response information. See Channel handshake signals on page A3-38 .

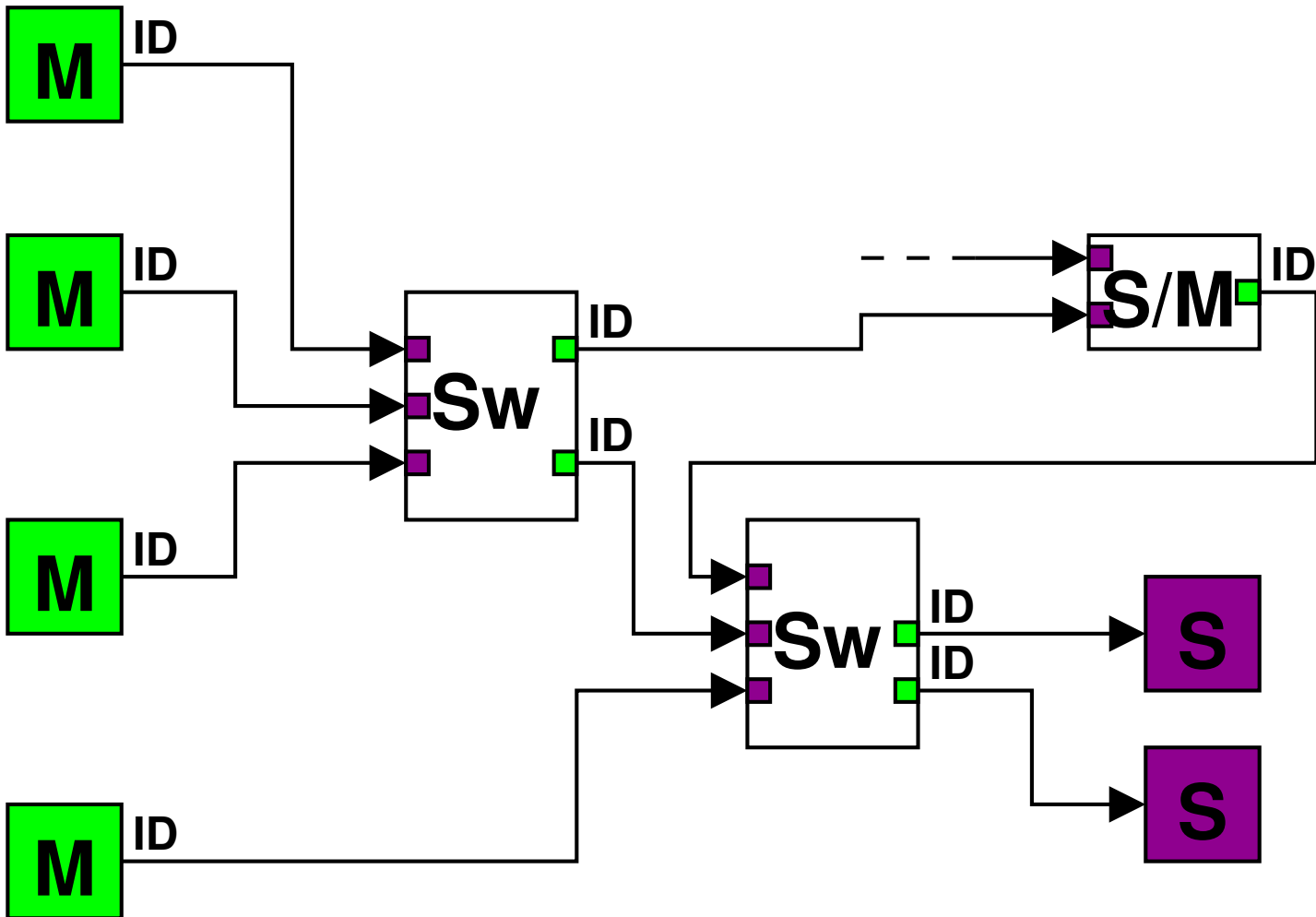
Architecture mono-thread



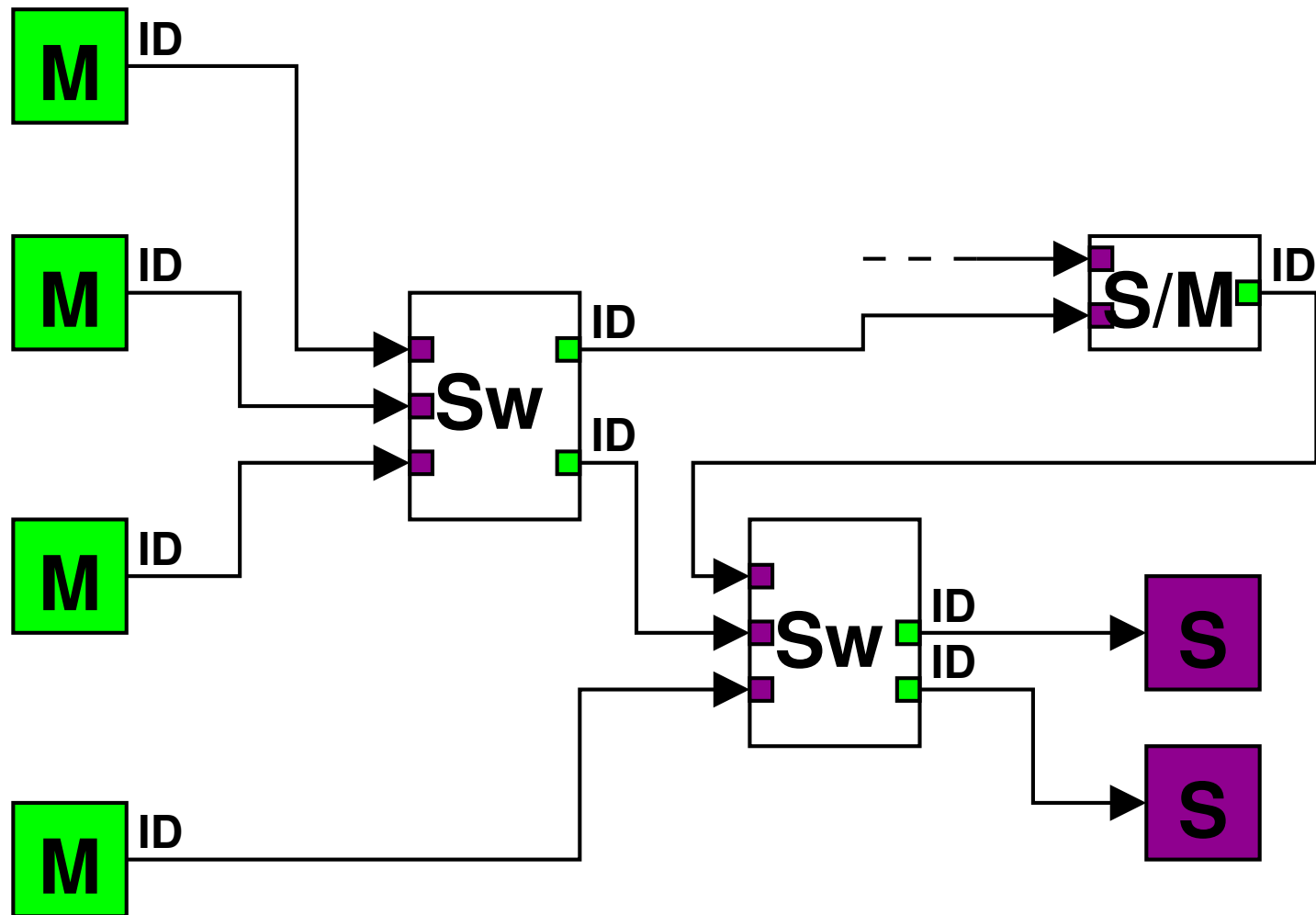
Architecture multi-thread



Topologie complexe

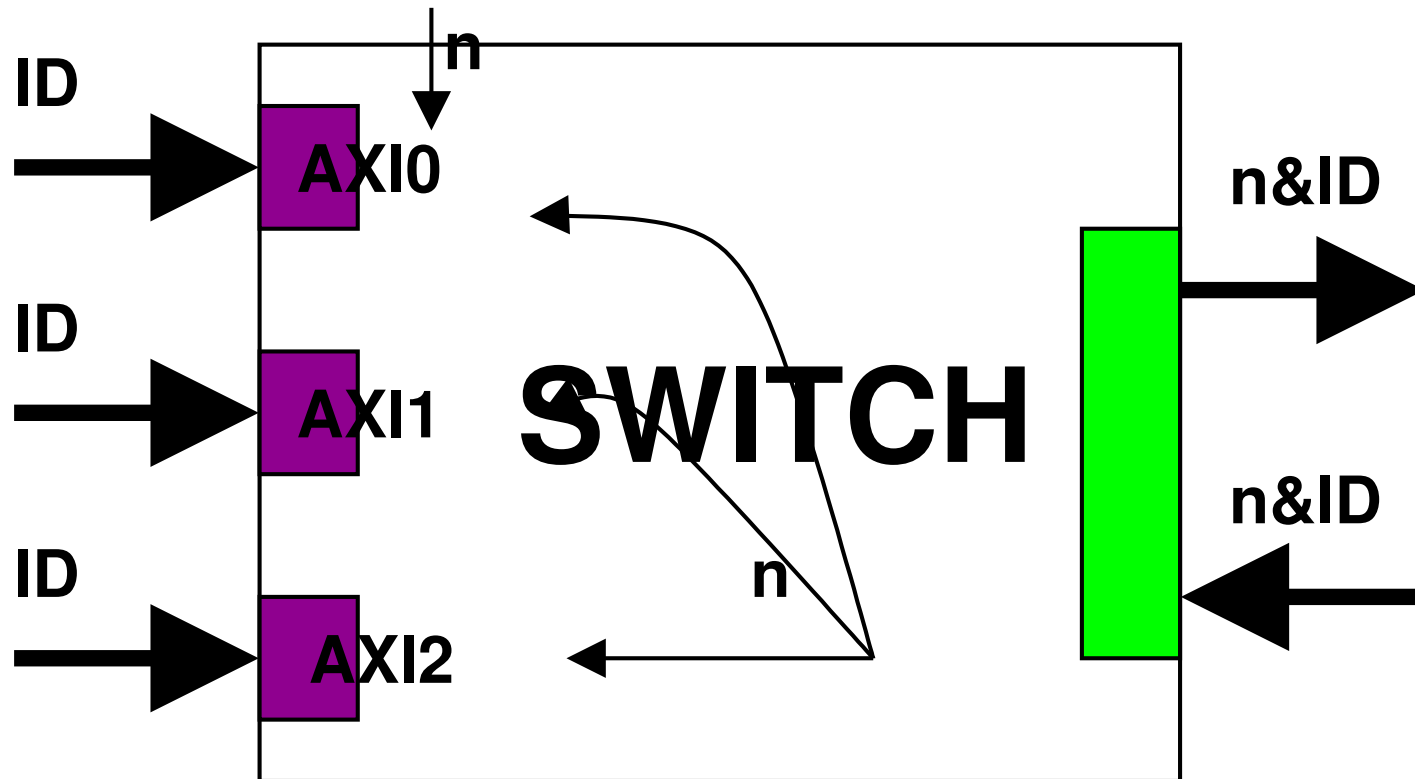


Topologie complexe



? Que se passe-t-il si des transfert de maîtres différents et de même ID arrivent sur un périphérique commun?

AXI, Gestion des ID

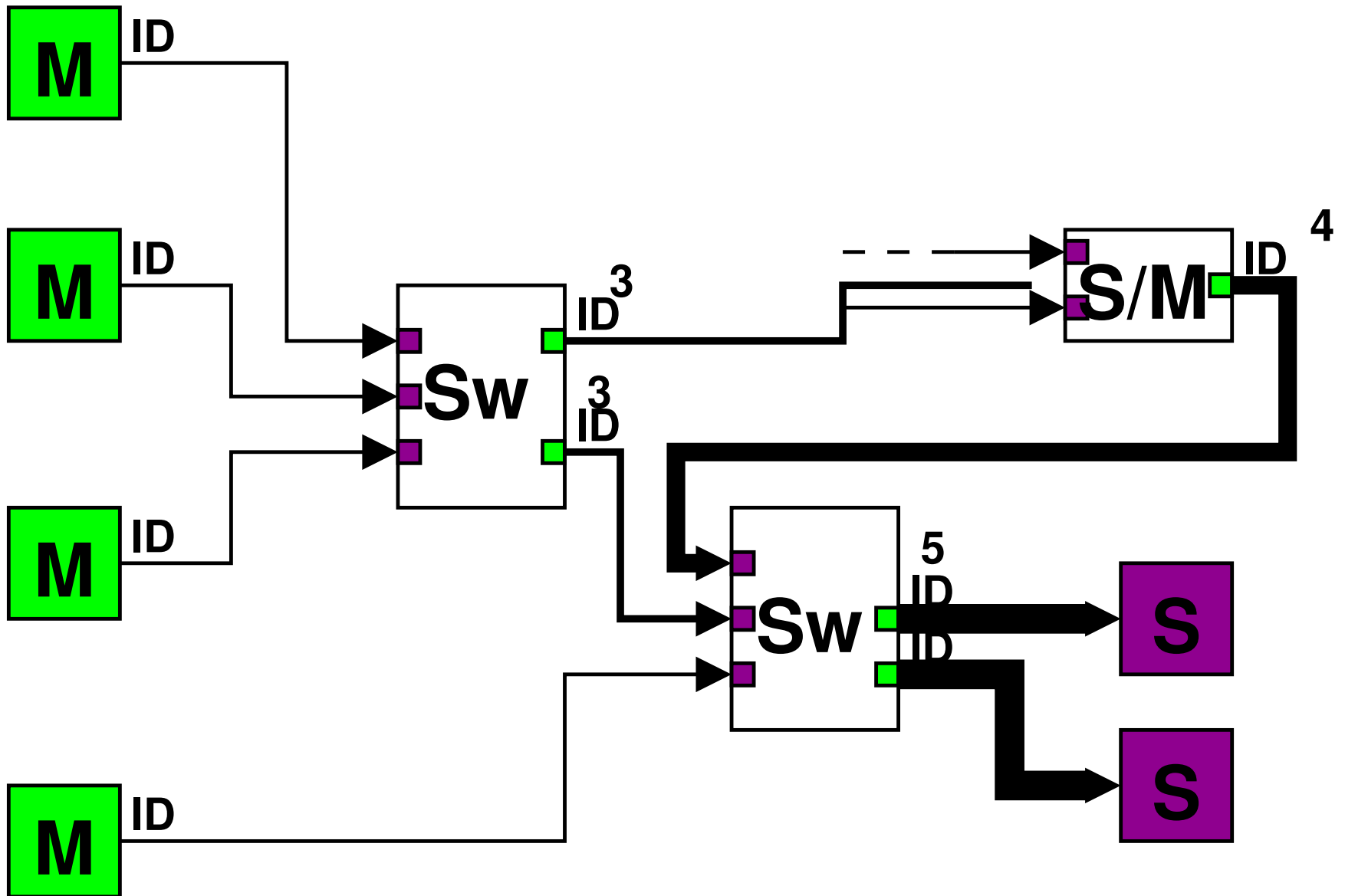


Chaque switch concatène une ID entrante par son numéro de canal d'arrivée.

⚠ Les signaux d'ID s'accroissent au fur et à mesure de la progression de la requête.

➡ La topologie et le routage doivent garantir cette propriété !!

AXI, Gestion des ID



AXI : suppression du lock

 Afin d'éviter la propagation de verrous dans tous le réseau, il n'y a plus de signal **lock** dans les interfaces!!

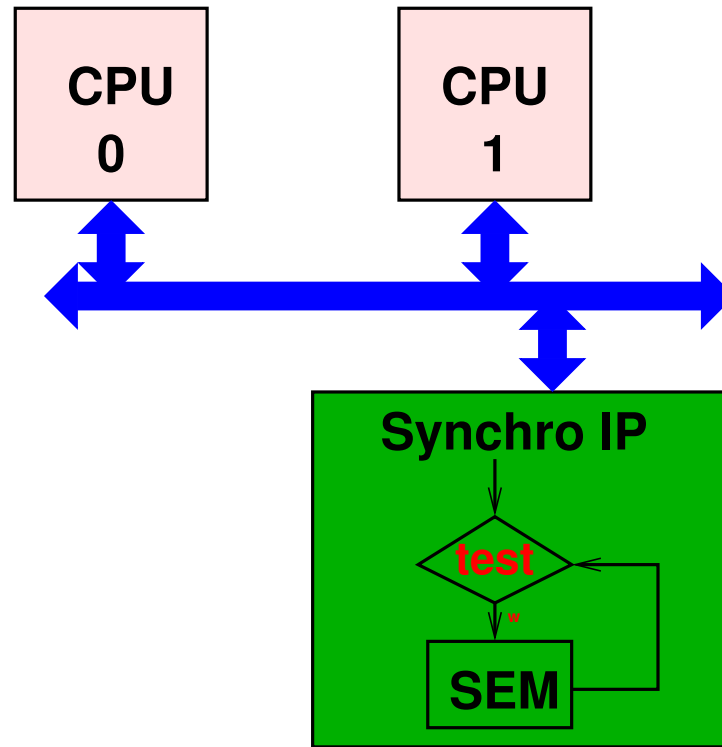
 Il n'est plus possible de faire de lecture/écriture atomique directement sur le bus.



 Mais, pour maintenir les propriété de la programmation concurrente, il faut maintenir la possibilité de réaliser des verrous!

L'absence de synchronisation au niveau matériel du bus est compensée par des périphériques de synchronisation.

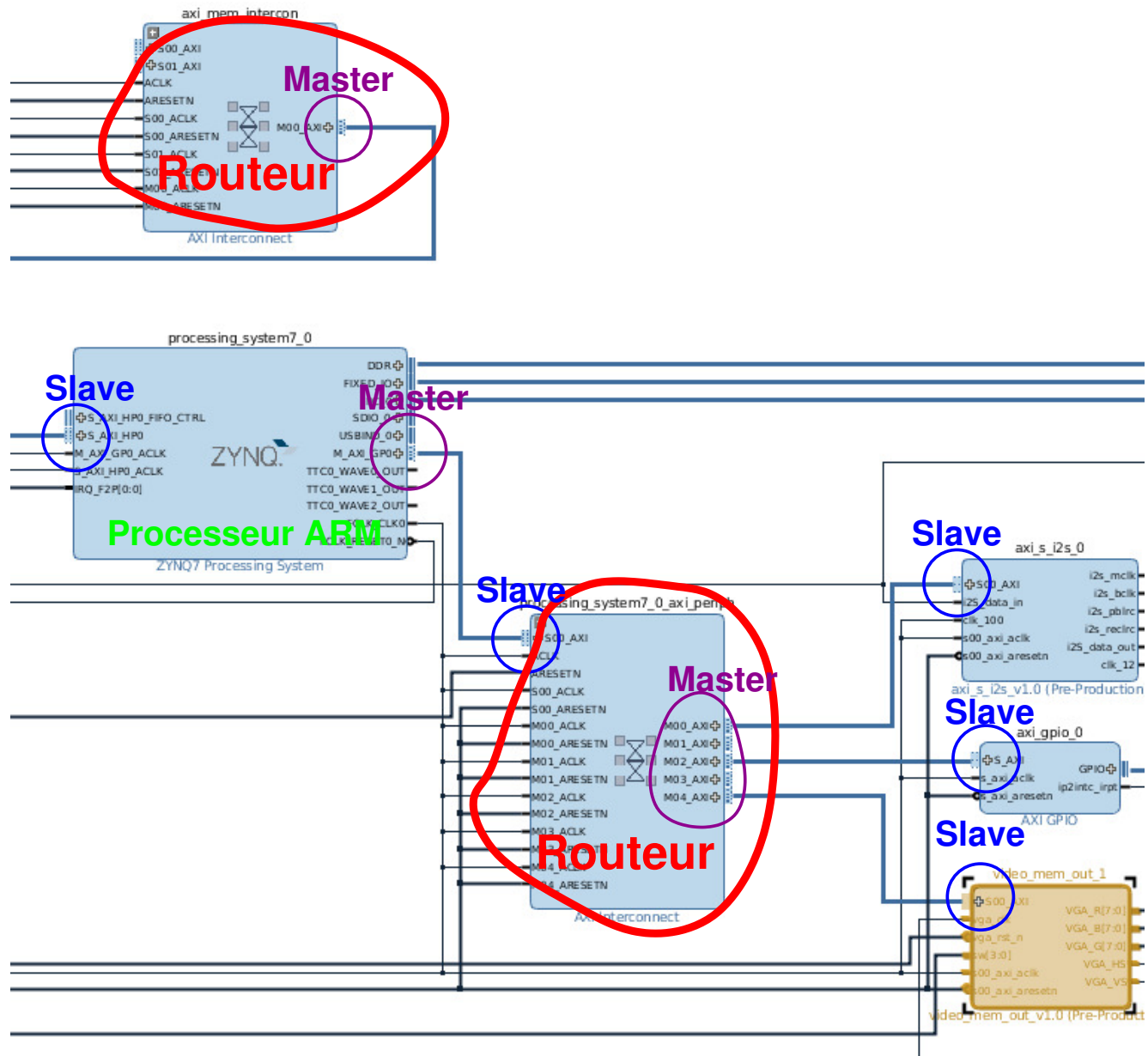
Synchronisation bas niveau



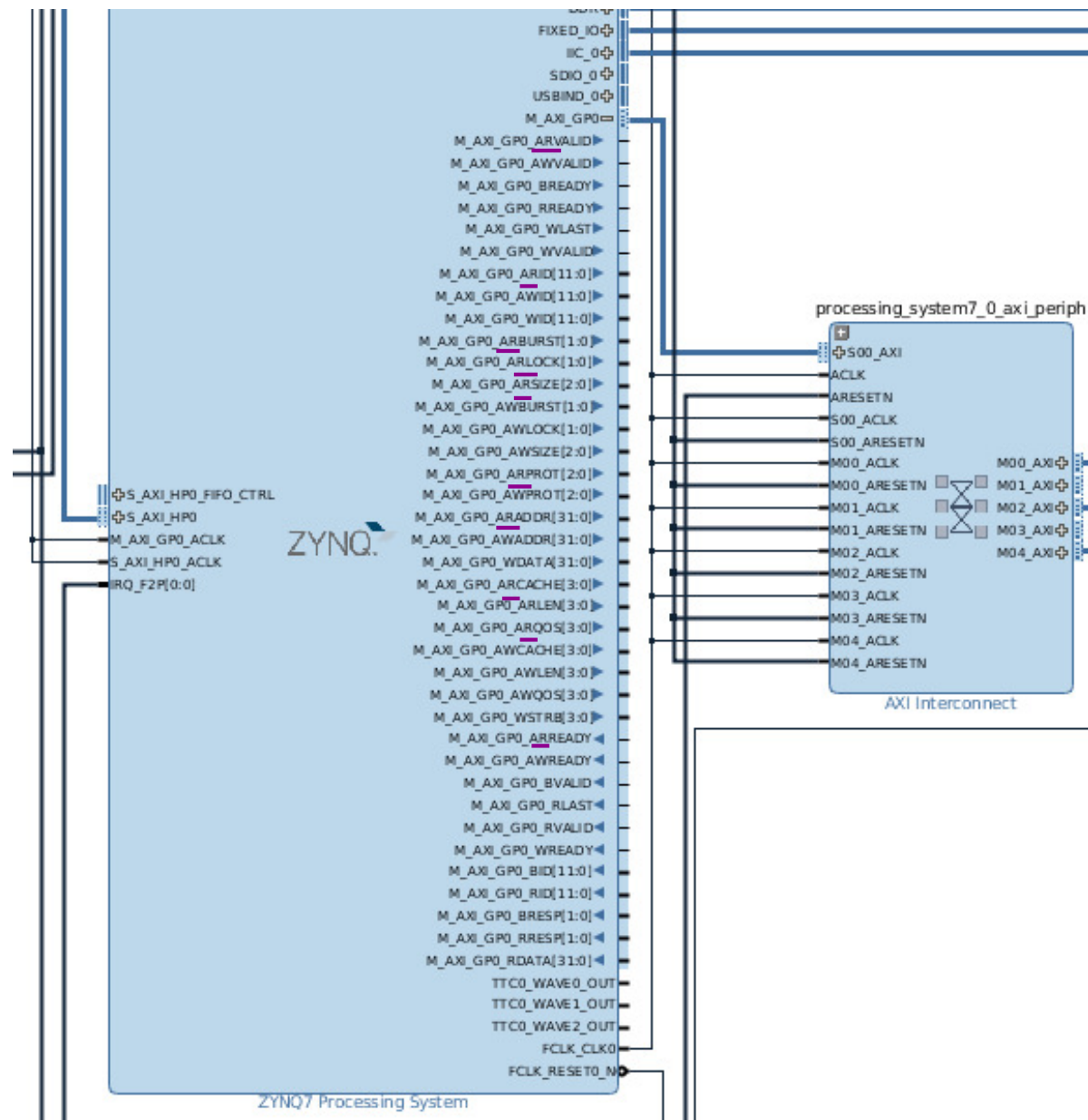
Procédure d'accès:

```
write ID to SEM (use 32 bit store, with ID in 12 LSB)
retrieve SEM (use 32 bit load, it returns 0x00000nnn)
if (SEM = ID) {
  performs a short critical section action
  write 0 to SEM
}
else
  "try again later, or loop back to write"
```

Exemple: Zynq



Exemple: Zynq



Exemple: Zynq SoC

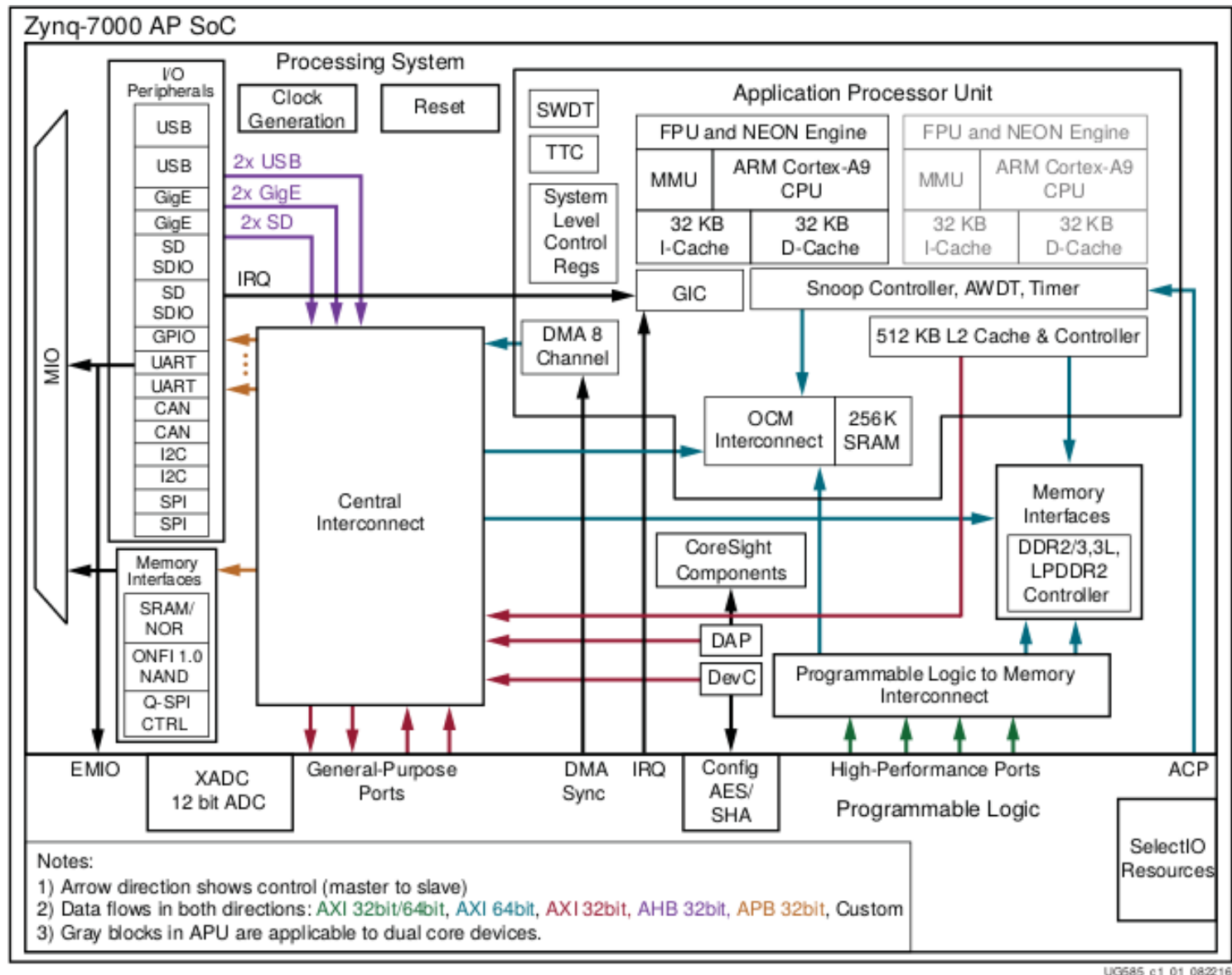


Figure 1-1: Zynq-7000 AP SoC Overview

Exemple: Zynq processor system

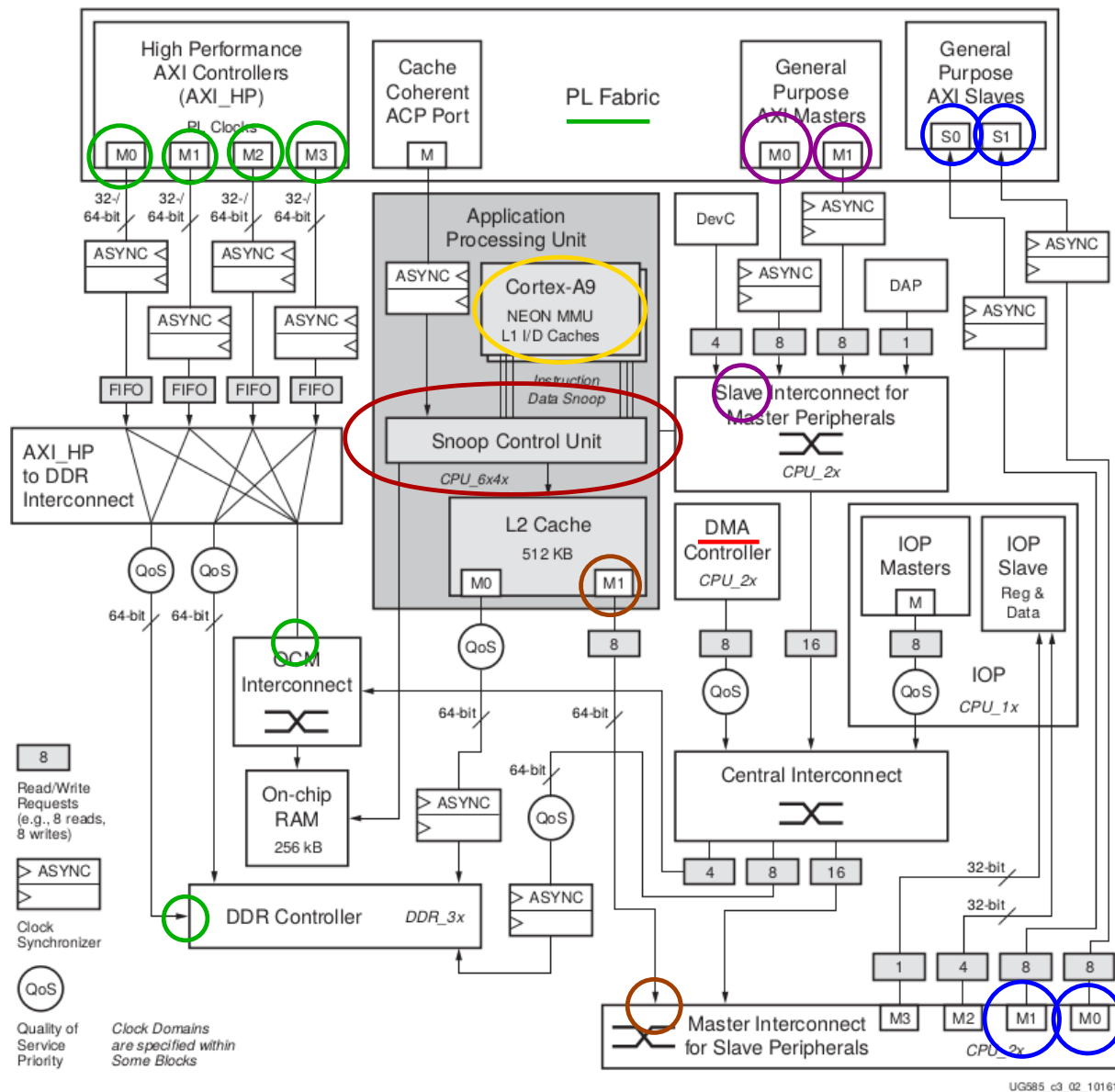


Figure 3-2: APU System View Diagram

Bilan sur AXI4

Le découpage en liens point à point permet de:

- 😊 Simplifier l'arbitrage, local à un routeur et sur-mesure
- 😊 Réduire les blocages en découplant les transferts dans tout le système
- 😊 Avoir des zones d'horloges indépendantes

En contrepartie:

- 😞 La suppression de certains signaux (lock)
- 😞 D'une topologie et de transferts cohérents par construction
- 😞 D'un coût matériel pour:
 - ❑ La complexité des contrôleurs: arbitrage et réordonnancement
 - ❑ Le stockage des requêtes et des données en attente sur les voies montantes et descendantes

Mais le matériel ne coûte plus très cher😊

Plan

- ❑ Bus système
- ❑ Éléments d'un bus système
- ❑ Infrastructure de communication
- ❑ Bus systèmes
- ❑ Network on Chip
- ✗ Conclusion

Conclusion

Le choix d'un bus système se fait en fonction:

- ⚙ De l'application et des ressources/périphériques utilisés
- ⚙ De l'infrastructure logicielle
- ⚙ De la souplesse et diversité des applications
- ⚙ De la performance attendue
- ⚙ De l'antériorité & rétrocompatibilité

Les **bus systèmes** 'usuels' ont l'avantage de consommer moins de ressources mais sont difficilement extensibles.

Les **NoC** sont plus souples et facilement extensibles, en contrepartie de davantage de ressources matérielles, mais modifie la gestion du parallélisme & synchronisation.

La seconde partie du cours portera sur l'environnement logiciel associé aux NoC.

Architecture des SoC

Infrastructure de communication

S. Mancini

Plan Détaillé

✖ Bus système

- ☆ Objectifs
- ☆ Problématique
- ☆ Physiologie
- ☆ Exemple : communication CPU/IP
- ☆ Exemple : communication CPU/IP

✖ Eléments d'un bus système

- Connectique
 - ☆ Bus Trois Etats
 - ☆ Bus "Cross bar"
 - ☆ Bus "NOC"
- Contrôleur de bus
 - ☆ Contrôleur de bus
 - ☆ Arbitrage
 - ☆ Exercice: arbitre de bus
 - ☆ Exercice: arbitre de bus
 - ☆ Exercice: arbitre de bus
 - ☆ Amortissement du coût de l'arbitrage: burst

✖ Infrastructure de communication

□ Topologie

- ☆ Espace d'adressage
- ☆ Hiérarchie de bus
- ☆ Gestion des interblocages

□ DMA

- ☆ DMA: "Direct Memory Access"
- ☆ Caractéristiques d'un DMA
- ☆ De CPU à CPU
- ☆ Contrôle d'un chemin de données
- ☆ Terminal
- ☆ Coopération
- ☆ Accès à une ressource commune

✖ Bus systèmes

□ CoreConnect

- ☆ Architecture
- ☆ CoreConnect - Exemple
- ☆ CoreConnect - Contrôle
- ☆ PLB Signaux
- ☆ PLB Master
- ☆ PLB Slave

- ☆ Exemple de transaction: PLB burst read
- ☆ PLB & pipeline de transferts

□ AMBA

- ☆ AMBA - Architecture
- ☆ Signaux
- ☆ Connectique AHB
- ☆ Contrôle
- ☆ Exemple: AMBA AHB master
- ☆ Exemple: AMBA AHB slave
- ☆ Exemple: AMBA AHB protection
- ☆ Résumé

✖ Network on Chip

□ AMBA AXI

- ☆ AMBA AXI
- ☆ AXI, réseau point à point
- ☆ AXI Read
- ☆ AXI Write
- ☆ AXI Read, signaux de base
- ☆ AXI Read, signaux complémentaires

- ☆ Problématique du routage
- ☆ Gestion des ID
- ☆ AXI Signals Read Address
- ☆ AXI Signals Read Data
- ☆ Architecture mono-thread
- ☆ Architecture multi-thread
- ☆ Topologie complexe
- ☆ AXI, Gestion des ID
- ☆ AXI, Gestion des ID
- ☆ AXI : suppression du lock
- ☆ Synchronisation bas niveau
- ☆ Exemple: Zynq
- ☆ Exemple: Zynq
- ☆ Exemple: Zynq SoC
- ☆ Exemple: Zynq processor system
- ☆ Bilan sur AXI4

✖ Conclusion

- ☆ Conclusion